
특이점이 온 개발자 - Docker & k8s 컨테이너 인프라

컨테이너의 세계로 떠나는 첫 여정

도커가 뭔지, 왜 쓰는지, 어떻게 쓰는지를 이야기로 풀어낸다. 컨테이너의 개념부터 Kubernetes 배포까지, 하나의 프로젝트로 단계별로 따라가며 익힌다.

최주호, 류재성, 김주혁 지음

특이점이 온 개발자 - Docker & k8s 컨테이너 인프라

목차

챕터 0. 프롤로그	4
챕터 1. Docker 이해하기	7
학습 목표	7
1.1 말콤 맥린 : 컨테이너의 아버지	7
1.2 Docker : 작동 원리	10
1.3 Docker Desktop : 설치	14
1.4 Docker CLI : 기본 명령어	14
1.5 Linux : 컨테이너 안에서 쓰는 명령어	17
1.6 컨테이너 : 생명주기와 옵션	24
1.7 이미지 : 직접 만들어보기	30
1.8 마운트 : 컨테이너에 저장소 연결하기	34
이것만은 기억하자	39
챕터 2. Docker 다루기	40
학습 목표	40
2.1 프로비저닝 : 환경을 자동으로 구성하다	40
2.2 NGINX : 웹 서버와 리버스 프록시	48
2.3 Redis : 세션 저장소	62
2.4 MySQL : DB 서버 구축	66
2.5 Docker Compose : 여러 컨테이너를 한 번에	70
2.6 종합 실습 : 웹 사이트 만들기	75
이것만은 기억하자	82
챕터 3. Kubernetes 시작하기	83
맺음말	131
부록: 더 깊이 공부하기	132
Docker	132

Kubernetes	132
이 책의 예제 코드	132
주요 명령어 모음	134

챕터 0. 프롤로그

0.1 문제를 발견하다.

두 달간 밤낮없이 매달렸던 웹사이트의 테스트가 드디어 완료되었습니다. 테스트를 마쳤을 때는 드디어 끝났다는 해방감에 젖어 있었습니다.

하지만 설레는 마음으로 운영 서버의 문을 연 순간, 잔인한 현실이 기다리고 있었습니다. 서버에 코드를 올리자마자 에러 메시지가 비명처럼 쏟아졌습니다.

Java 버전이 맞지 않았고, 꼭 있어야 할 라이브러리는 보이지 않았으며, 설정 파일 경로는 엉뚱한 곳을 가리키고 있었습니다. 하나를 겨우 고쳐놓으면, 기다렸다는 듯 다른 곳에서 또 에러가 터져 나왔습니다.

“제 컴퓨터에서는 분명히 잘 됐는데...”

오늘만 벌써 세 번째, 힘없이 내뱉은 변명이었습니다. 원인은 명확했습니다. 제 컴퓨터와 운영 서버의 ‘환경’이 서로 달랐던 것입니다. 밤을 꼬박 새워 수동으로 버전을 맞추며 배포엔 성공했지만, 다음 업데이트 때 똑같은 짓을 반복해야 한다고 생각하니 앞이 캄캄해졌습니다.

그때였습니다. 멍하니 모니터를 보던 제 옆으로 선배가 다가와 툭 한마디를 던졌습니다.

“오픈아, Docker 한번 알아봐. 네 컴퓨터 환경을 통째로, 어디서든 똑같이 꺼내 쓸 수 있게 해 줄 거야.”

0.2 Docker를 만나다.

지푸라기라도 잡는 심정으로 Docker를 공부하기 시작했습니다. 처음엔 ‘컨테이너’니 ‘이미지’니 하는 말들이 뜬구름 잡는 소리처럼 들렸습니다. 하지만 백 번 읽는 것보다 한 번 해보는 게 낫겠다 싶어 무작정 설치부터 시작했습니다.

직접 컨테이너를 띄워 그 안으로 들어가 보았습니다. 명령어를 치고, 필요한 패키지를 깔고, 웹 서버를 실행해 보고... 그렇게 직접 흠을 묻혀가며 만져보니 그제야 안개가 걷히듯 감이 오기 시작했습니다.

무엇보다 놀라웠던 건, 제가 공들여 세팅한 환경을 ‘이미지’라는 파일 하나로 찰칵 찍어 저장할 수 있다는 사실이었습니다. 이 이미지만 있으면 그게 누구의 컴퓨터든, 어떤 서버든 제가 만든 것과 1%의 오차도 없는 똑같은 환경을 복제해낼 수 있었습니다.

두근거리는 마음으로 이미지를 운영 서버에 올렸습니다. 결과는 놀라웠습니다. 로컬에서 보던 모습 그대로, 서버에서도 완벽하게 숨을 쉬기 시작했습니다. 저를 그토록 괴롭히던 환경 차이라는 유령이 드디어 사라진 순간이었습니다.

0.3 자동화를 배우다.

Docker의 매력에 푹 빠져 지내다 보니, 슬슬 또 다른 욕심이 생겼습니다. 컨테이너를 새로 만들 때마다 필요한 패키지들을 일일이 설치하는 게 은근히 번거로웠기 때문입니다. 매번 똑같은 작업을 반복하는 제 손가락이 조금 억울해 보일 지경이었습니다.

“선배, 이거 매번 똑같이 설치하는 과정을 자동으로 만들 순 없을까요?”

“Dockerfile을 써봐. 레시피를 적어두면 Docker가 알아서 요리해 줄 테니까.”

선배의 조언대로 Dockerfile에 레시피를 적어 내려갔습니다. ‘이 이미지를 베이스로 삼고, 이런 패키지를 깔고, 마지막에 이 명령어를 실행해 줘.’ 이제는 손가락 아프게 타이핑할 필요가 없었습니다. 파일 하나만 던져주면 늘 동일한 환경이 똑딱 완성되었기 때문입니다.

하지만 서비스가 커지면서 또 다른 벽에 부딪혔습니다. 프론트엔드, 백엔드, 데이터베이스... 관리해야 할 컨테이너가 늘어나자 하나하나 켜고 네트워크를 연결하는 것조차 버거운 일이 되었습니다.

“하나씩 실행하려니까 배보다 배꼽이 더 큰 것 같아요.”

“그럴 땐 Docker Compose지. 한 장에 다 적어두고 명령어 한 줄로 싹 다 띄워버려.”

docker-compose.yml 파일에 전체 서비스의 지도를 그렸습니다. 그리고 명령어 한 줄을 입력했습니다. 순식간에 여러 컨테이너가 일사불란하게 깨어나 서로 대화를 나누기 시작했습니다. 복잡한 웹사이트가 단 한 줄로 완성되는 걸 보며, 저는 비로소 자동화가 주는 진정한 해방감을 맛보았습니다.

0.4 Kubernetes 시작하기

모든 게 완벽해 보였던 어느 날 새벽, 요란한 알람 소리에 잠이 깼습니다. 운영 서버의 컨테이너 하나가 알 수 없는 이유로 멈춰버린 것이었습니다. 허겁지겁 서버에 접속해 수동으로 다시 살려냈지만, 등 줄기엔 식은땀이 흘렀습니다.

그뿐만이 아니었습니다. 사용자가 몰리면 직접 컨테이너를 늘려줘야 했고, 새 버전을 배포할 때마다 서비스가 잠깐씩 끊기는 건 어쩔 도리가 없었습니다. Docker가 환경을 담아주긴 했지만, 거친 바다 같은 실전 운영(Operation)의 파도는 여전히 제 손으로 일일이 막아내야 했습니다.

“선배... 컨테이너가 죽으면 지가 알아서 다시 살아날 순 없을까요?”

제 물음에 선배가 빙그레 웃으며 대답했습니다.

“이제 드디어 Kubernetes(쿠버네티스)를 만날 때가 됐구나. 죽은 녀석은 알아서 살려내고, 새 버전도 티 안 나게 속 바꿔치기해 주는 녀석이지.”

그렇게 저의 새로운 항해가 시작되었습니다. 컨테이너를 보호하는 ‘Pod’, 개수를 유지하는 ‘Deployment’, 길을 터주는 ‘Service’... 낯선 용어들이 쏟아졌지만, 이 거대한 오케스트라의 지휘법을 배우며 하나씩 제 것으로 만들었습니다.

테스트 삼아 컨테이너 하나를 강제로 종료해 보았습니다. 그러자 쿠버네티스가 기다렸다는 듯 몇 초 만에 새 컨테이너를 띄워 빈자리를 채웠습니다. 새 버전 배포 역시 사용자들은 눈치채지 못할 만큼 매끄럽게 흘러갔습니다.

뿌듯해하는 저를 보며 선배가 어깨를 툭 쳤습니다.

“필요한 순간에 하나씩 배우면 돼. 처음부터 이 모든 걸 다 알 필요는 없단다.”

이 여정에서 만나게 될 것들

이 책은 오픈이가 걸어온 길을 그대로 따라갑니다. 하나의 웹사이트가 세 개의 챕터를 거치며 단계적으로 진화합니다.

1장 — Docker 이해하기 컨테이너가 무엇인지, 왜 필요한지를 해상 운송의 역사부터 시작해 알아봅니다. Docker를 설치하고 컨테이너 안에 직접 들어가 봅니다. 이미지로 환경을 저장하고, 마운트로 데이터가 사라지지 않게 보호하는 것까지. 상자를 만들고, 찍고, 고정하는 기본기를 익힙니다.

2장 — Docker 다루기 Dockerfile로 환경 구성을 자동화하고, NGINX로 요청을 분배하고, Redis로 세션을 공유하고, MySQL로 데이터를 영구 저장합니다. 마지막으로 Docker Compose로 프론트엔드, 백엔드, DB가 연동되는 풀스택 웹사이트를 명령어 한 줄로 띄워봅니다.

3장 — Kubernetes 시작하기 컨테이너가 죽으면 알아서 살려내고, 트래픽이 몰리면 자동으로 늘려주는 쿠버네티스를 만납니다. Pod, Deployment, Service, ConfigMap, Secret, Volume, Ingress를 하나씩 익히고, 2장에서 만든 웹사이트를 쿠버네티스 위에 다시 배포합니다.

자, 그럼 1장에서 만나겠습니다.

챕터 1. Docker 이해하기

■ 학습 목표

- 컨테이너의 개념과 등장 배경을 이해한다.
- Docker를 설치하고 컨테이너를 실행·조작한다.
- 컨테이너를 이미지로 저장하고 Dockerhub에 공유한다.
- 마운트를 사용하여 컨테이너 삭제 후에도 데이터를 보존한다.

■ 1.1 말콤 맥린 : 컨테이너의 아버지

1.1.1 해상 운송의 초기

Docker를 이해하려면 먼저 ‘컨테이너’라는 개념이 어디서 왔는지 알아야 합니다. 그 시작은 의외로 바다 위에 있습니다.

컨테이너가 등장하기 전, 해상 물류는 온전히 사람의 힘에 의존했습니다. 항구에 배가 도착하면 수백 명의 노동자가 화물을 하나씩 내리고 분류하고 다시 트럭에 옮겨 실어야 했습니다. 이 과정은 길게는 몇 주씩 걸리기도 했습니다.

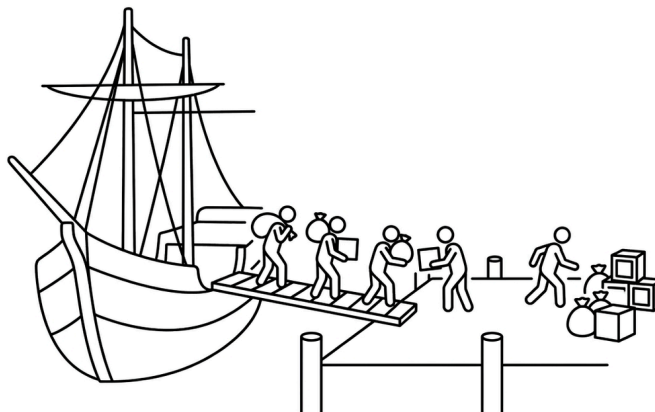


그림 0-1: 초기 해상 물류의 수작업 하역 과정

배로 물건을 보내는 비용보다, 사람이 짐을 싣고 내리는 인건비가 더 비싼 구조였습니다. 트럭 운전사들도 화물을 기다리느라 항구에서 며칠씩 대기하는 일이 다반사였습니다.

1.1.2 컨테이너의 도입

이 비효율을 가장 가까이에서 체감한 사람이 있었습니다. 바로 트럭 운전자 말콤 맥린입니다.

항구에서 화물이 하나씩 옮겨지는 걸 반복적으로 지켜보던 그는 단순하지만 강력한 의문을 품습니다. “왜 짐을 박스째로 통째로 옮기지 않는 걸까?”

이 질문에서 컨테이너라는 개념이 탄생했습니다. 화물을 낱개로 다루는 대신, 표준 규격의 큰 상자 하나에 담아서 통째로 옮기자는 아이디어였습니다.

크레인으로 컨테이너를 그대로 배에 올리고, 도착지에서도 같은 방식으로 내리면 됩니다. 운송 속도와 효율은 이전과는 비교할 수 없을 정도로 올라갔습니다.

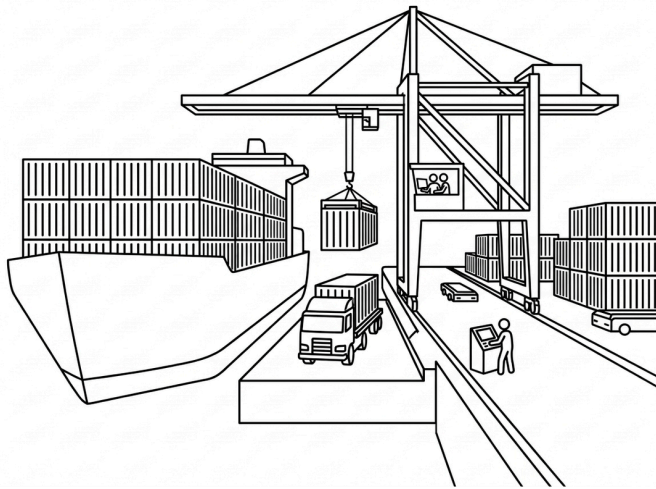


그림 0-2: 표준 컨테이너와 크레인을 활용한 현대 해상 운송

물론 처음부터 순탄하지는 않았습니다. 기존 선박은 컨테이너를 싣는 걸 전제로 만들어진 게 아니었기 때문에 공간 활용이 어려워 효율이 충분히 나오지 않았습니다.

맥린은 여기서 멈추지 않았습니다. 아예 컨테이너 전용 선박을 직접 만들어 버립니다.

상상은 결국 현실이 되었습니다. 컨테이너는 해상 운송의 표준으로 자리 잡았고, 전 세계 물류 체계를 바꿔 놓았습니다.

1.1.3 IT에서의 컨테이너

그렇다면 이 이야기가 IT와 무슨 관련이 있을까요? 사실 IT에서 말하는 ‘컨테이너’도 해상 운송의 컨테이너와 본질은 같습니다.

컨테이너(Container)는 어디서든 동일하게 실행되는 표준화된 실행 환경을 의미합니다.



그림 0-3: 개발 환경과 운영 환경의 차이로 인한 오류 발생

개발 환경에서 아무리 열심히 테스트해도, 결국 운영 환경에 배포하는 순간 또 다른 변수가 기다리고 있는 셈입니다.

컨테이너는 바로 이 문제를 해결합니다. 애플리케이션이 실행되는 데 필요한 라이브러리와 설정 파일, 의존성을 전부 하나의 상자에 담아 버리는 겁니다. 마치 말콤 맥린이 화물을 표준 컨테이너에 넣어 어디서든 똑같이 옮길 수 있게 한 것과 같습니다.

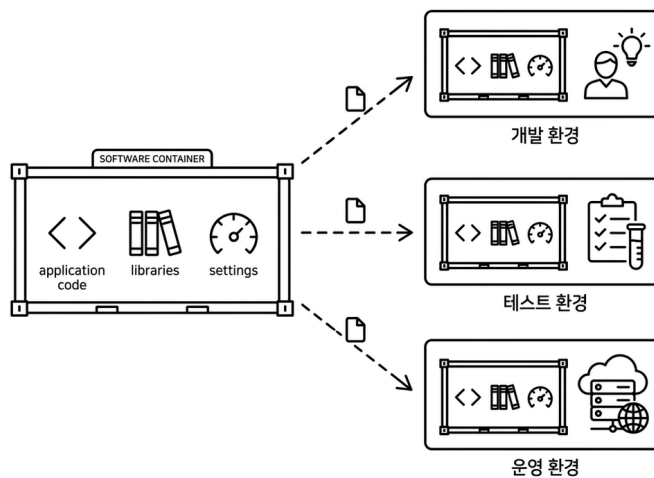


그림 0-4: 컨테이너를 통한 일관된 실행 환경

1.2 Docker : 작동 원리

1.2.1 가상화 : 하나의 컴퓨터를 여러 개로 사용하는 법

컨테이너가 무엇인지는 알았으니, 이제 Docker가 어떤 원리로 동작하는지 살펴봅시다. 그 출발점은 가상화입니다.

가상화(Virtualization)는 하나의 물리 서버를 논리적으로 여러 대처럼 나누어 쓰는 기술입니다. 가상화를 도입하면 서버를 효율적으로 쓸 수 있고 서로 다른 환경을 안전하게 격리할 수 있습니다.

좀 더 쉽게 이해하기 위해 주방에 비유해 보겠습니다.

하나의 주방에 네 명의 요리사가 일하고 있습니다. 그런데 냉장고와 가스레인지가 각각 하나뿐이라 한 번에 한 명만 쓸 수 있는 구조입니다.

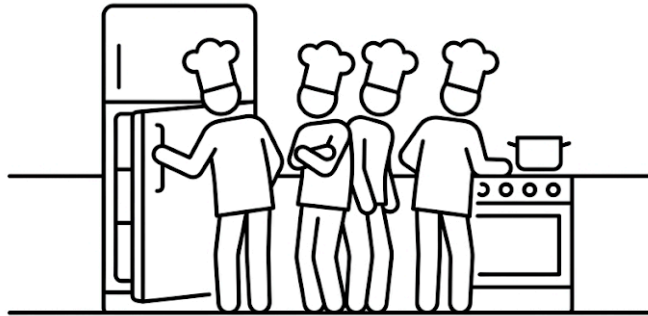


그림 0-5: 하나의 주방을 공유하는 네 명의 요리사

네 명이 동시에 요리하려면 어떻게 해야 할까요? 방법은 크게 두 가지입니다.

첫 번째 방법은 요리사 수만큼 주방을 통째로 복제하는 것입니다. 냉장고, 가스레인지, 싱크대까지 전부 따로 마련해서, 각자 완전히 독립된 주방에서 일하게 합니다.

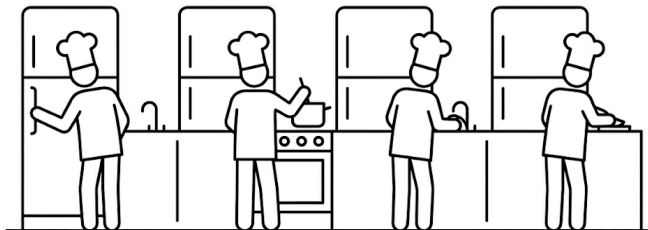


그림 0-6: 요리사별 독립 주방 – 주방 전체를 복제하는 방식

서로 방해받을 일이 없다는 게 장점이지만 설비를 전부 네 벌씩 갖춰야 하니 많은 비용이 발생하게 됩니다.

두 번째 방법은 냉장고나 칼 같은 공용 설비는 함께 쓰되 조리대만 각자 따로 마련하는 것입니다.

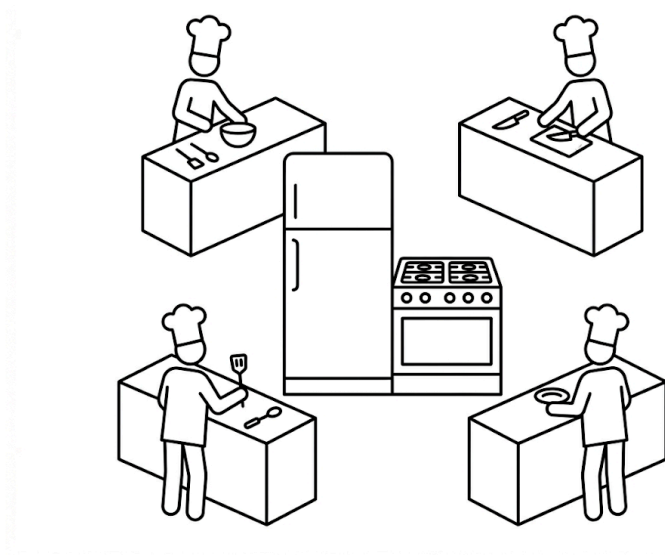


그림 0-7: 기본 설비 공유 + 개별 조리대 – 컨테이너 가상화 비유

이 방식은 자원을 알뜰하게 쓰면서도 각자의 작업 공간은 확보할 수 있습니다. 다만 공용 설비를 함께 쓰는 만큼 서로 방해하지 않기 위한 규칙이 필요합니다.

첫 번째 방식은 격리가 확실하지만 비용이 크고 두 번째 방식은 자원을 알뜰하게 쓰면서도 독립적인 작업 공간을 확보할 수 있습니다. Docker의 컨테이너 가상화가 바로 두 번째 방식입니다.

1.2.2 컨테이너 가상화 : OS 없이 격리하다

컨테이너 가상화(Container Virtualization)는 하나의 운영 체제를 함께 쓰면서 애플리케이션이 실행되는 환경만 따로 분리하는 방식입니다.

하나의 서버에는 OS가 설치되어 있고, 그 핵심에는 커널이 있습니다. 컨테이너 가상화는 이 커널을 여러 애플리케이션이 함께 공유하되, 각 애플리케이션의 실행 공간만 독립적으로 분리하는 방식입니다. 분리된 실행 공간 하나하나가 바로 컨테이너입니다.

컨테이너 안에는 앱 실행에 필요한 라이브러리, 설정, 파일시스템만 들어 있습니다. OS 전체를 복제하는 것이 아니라 커널은 호스트의 것을 함께 쓰고 실행 환경만 격리하는 겁니다.

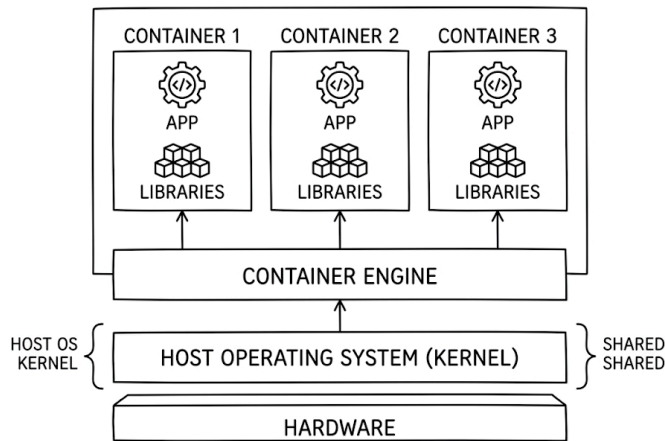


그림 0-8: 컨테이너 가상화 구조

컨테이너마다 파일시스템, 네트워크, 프로세스 공간이 분리되어 있어서 컨테이너 A에서 설치한 라이브러리가 컨테이너 B에 영향을 주지 않습니다.

컨테이너 가상화에서는 컨테이너 엔진(**Container Engine**) 이 컨테이너의 생성, 실행, 중지, 삭제를 담당합니다. **Docker**가 바로 가장 대표적인 컨테이너 엔진입니다. 이미지를 관리하고 컨테이너를 실행하는 등 운영에 필요한 기능을 두루 갖추고 있습니다.

1.2.3 Docker : 이미지에서 컨테이너까지

그렇다면 **Docker**는 어떻게 컨테이너를 만들까요? 간단한 명령어 하나를 따라가며 흐름을 살펴보겠습니다.

지금은 명령어를 실행할 필요 없이 동작 흐름만 이해하면 됩니다.

[참고] **nginx** 컨테이너를 실행하는 명령어입니다.

```
docker run nginx # nginx 컨테이너 실행
```

이 명령어를 실행하면 **Docker** 엔진이 요청을 받습니다. 그런데 **Docker** 엔진이 직접 컨테이너를 만들지는 않습니다. 대신 OS 커널에게 “격리된 프로세스를 하나 만들어 줘”라고 요청합니다.

커널은 건물의 관리사무소 같은 존재입니다. 전기, 수도, 엘리베이터를 관리사무소가 관리하듯이 커널은 컴퓨터의 CPU, 메모리, 프로세스를 관리합니다. 프로세스를 만들고, 격리하고, 자원을 나눠주는 건 오직 커널만 할 수 있는 일입니다.

커널(**Kernel**)은 운영체제(OS)의 핵심 구성 요소로, 프로세스를 만들고 실행 순서를 정하며, 프로세스끼리 서로 간섭하지 않도록 메모리를 보호합니다.

Docker 엔진의 요청을 받은 커널은 격리된 환경에서 새로운 프로세스를 생성합니다. 이것이 바로 컨테이너입니다.

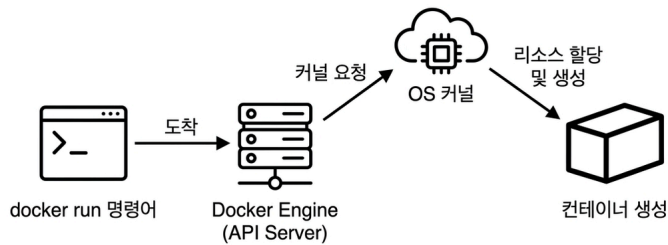


그림 0-9: docker run 실행 흐름

1.2.4 Docker 이미지

앞에서 컨테이너가 격리된 프로세스라는 걸 알았습니다. 그런데 컨테이너는 빈 광통이 아닙니다. 무언가를 담아야 실행할 수 있습니다. 그 “무언가”가 바로 이미지입니다.

이미지는 컨테이너를 만들기 위한 패키지입니다. 안에는 OS, 애플리케이션, 설정 파일이 하나로 묶여 있습니다. 예를 들어 nginx 이미지 안에는 Linux + nginx + 기본 설정이 들어 있습니다.

비유하면, 이미지는 봉어빵 틀입니다. 틀(이미지) 하나로 봉어빵(컨테이너)을 여러 개 찍어낼 수 있습니다. 틀 자체는 변하지 않고, 찍혀 나온 봉어빵은 각각 독립적입니다. 이미지 하나로 똑같은 컨테이너를 몇 개든 만들 수 있습니다.

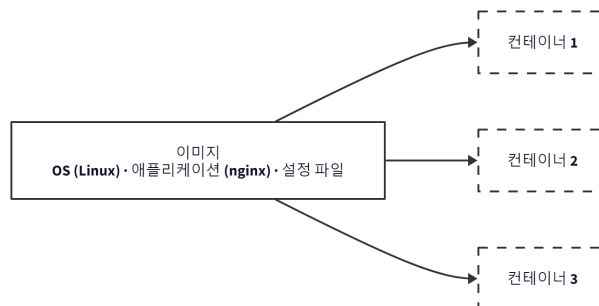


그림 0-10: 하나의 이미지로 여러 컨테이너를 만들 수 있다

그러면 이 이미지는 어디서 올까요? **Docker Hub**라는 저장소에 사람들이 만들어 둔 이미지가 있습니다. 여기서 가져다 쓰면 됩니다. 물론 직접 이미지를 만들어 Docker Hub에 업로드할 수도 있습니다.

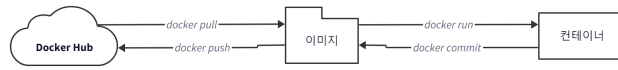


그림 0-11: Docker의 흐름

전체 흐름을 정리하면 이렇습니다. **Docker Hub**에서 이미지를 가져와서 컨테이너를 실행하고, 컨테이너 안에서 작업한 뒤 수정한 내용을 새 이미지로 저장합니다. 이렇게 만든 이미지를 **Docker Hub**에 올리면 다른 사람도 가져다 쓸 수 있습니다. 이 챕터에서는 이 흐름을 직접 따라가며 **Docker**를 익혀보겠습니다.

1.3 Docker Desktop : 설치

1. **Docker 공식 사이트**에 접속하여 자신의 OS에 맞는 **Docker Desktop**을 다운로드합니다.
2. 다운로드한 설치 파일을 실행하고, 안내에 따라 설치를 진행합니다.
3. 설치가 완료되면 **Docker Desktop**을 실행합니다.

Windows의 경우, **Docker Desktop**을 설치하기 전에 **WSL2(Windows Subsystem for Linux 2)**를 먼저 설치해야 합니다. **Docker**는 **Linux** 커널 기능을 사용해 컨테이너를 실행하는데, **Windows**에는 **Linux** 커널이 없기 때문에 **WSL2**를 통해 **Linux** 커널을 제공받아야 합니다.

설치가 완료되면 터미널에서 다음 명령어로 정상 설치 여부를 확인해 봅니다.

[실습] **Docker** 설치 상태를 확인하는 명령어입니다.

```
docker version # Docker 버전 확인
```

Client와 **Server** 정보가 모두 출력되면 **Docker**가 정상적으로 설치된 것입니다.

1.4 Docker CLI : 기본 명령어

1.4.1 Docker Desktop 실행

도커를 사용하기 위해서 먼저 도커 데스크탑을 실행합니다.

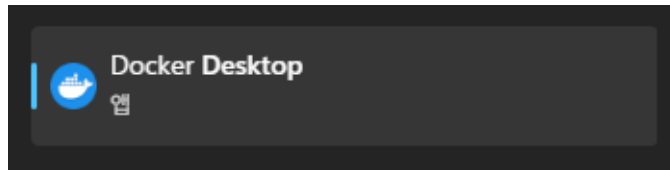


그림 0-12: Docker Desktop 실행 화면

1.4.2 docker pull : 이미지 다운로드

`docker pull` 명령어는 앞에서 살펴본 Docker Hub에서 이미지를 다운로드합니다.

CMD 창을 연 후 명령어를 실행해 봅니다.

[실습] nginx 이미지를 다운로드하는 명령어입니다.

```
docker pull nginx # nginx 이미지 다운로드
```



그림 0-13: nginx 이미지 다운로드

1.4.3 docker run : 컨테이너 실행

[실습] nginx 이미지를 기반으로 컨테이너를 실행하는 명령어입니다.

```
docker run nginx # nginx 컨테이너 실행
```

명령어를 실행하면 컨테이너가 정상적으로 실행되면서 터미널창에 컨테이너 내부 로그가 조회됩니다.

이번 단계의 목표는 컨테이너를 실행하는 것입니다. 실행 후 컨테이너에 접근하려면 포트 포워딩(-p 옵션)이 필요한데, 이 옵션은 리눅스 챕터에서 다룹니다.

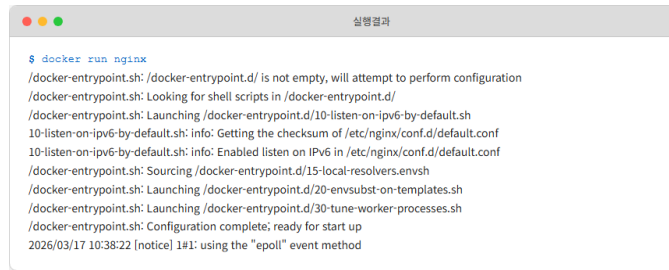


그림 0-14: nginx 컨테이너 실행

지금 상태에서는 터미널창에 추가로 명령어가 입력되지 않습니다. 이 상태를 포그라운드 상태라고 합니다.

포그라운드(Foreground)는 프로세스가 현재 터미널을 직접 점유하여 동작하는 방식입니다. 포그라운드 상태에서는 컨테이너의 실행 로그가 실시간으로 출력되며, 터미널을 종료하면 해당 컨테이너도 함께 종료됩니다.

실행 중인 프로세스는 **CTRL + C** 를 입력하면 빠져나올 수 있습니다.

1.4.4 docker run -d : 백그라운드 실행

백그라운드(Background)는 프로세스가 터미널과 분리된 상태로 동작하는 방식입니다. 백그라운드 상태에서는 컨테이너가 뒤에서 계속 실행되며, 사용자는 터미널을 종료하지 않고도 다른 명령을 자유롭게 실행할 수 있습니다.

[실습] -d 옵션을 사용하여 nginx를 백그라운드에서 실행하는 명령어입니다.

docker run -d nginx # -d : detached 모드

명령어를 실행하면 컨테이너 **ID** 가 출력되고, 터미널은 입력 가능한 상태로 돌아옵니다. 간단합니다.

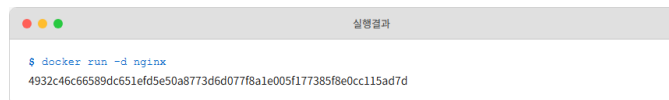


그림 0-15: 백그라운드 실행 결과

1.4.5 docker ps : 컨테이너 목록 출력

[실습] 실행 중인 컨테이너 목록을 출력하는 명령어입니다.

`docker ps` # 실행 중인 컨테이너 출력

현재 실행 중인 컨테이너의 정보를 조회할 수 있습니다.

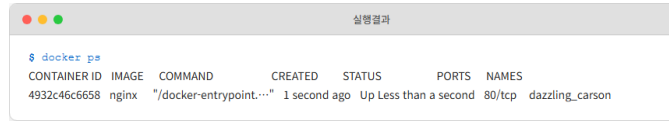


그림 0-16: 컨테이너 목록 조회

1.4.6 자주 사용하는 Docker 명령어

자주 사용하는 Docker 명령어는 다음과 같습니다.

명령어	설명	예시
<code>docker pull <이미지명></code>	Dockerhub에서 이미지 다운로드	<code>docker pull nginx</code>
<code>docker images</code>	로컬에 저장된 이미지 목록 조회	<code>docker images</code>
<code>docker logs <컨테이너ID></code>	컨테이너 로그 출력	<code>docker logs 057c</code>
<code>docker ps -a</code>	종료된 컨테이너 포함 전체 목록 출력	<code>docker ps -a</code>
<code>docker stop <컨테이너ID></code>	실행 중인 컨테이너 종료	<code>docker stop 057c</code>
<code>docker rm <컨테이너ID></code>	종료된 컨테이너 삭제	<code>docker rm 057c</code>
<code>docker rmi <이미지ID></code>	이미지 삭제 (-f 로 강제 삭제)	<code>docker rmi -f fb01</code>

1.5 Linux : 컨테이너 안에서 쓰는 명령어

Docker 명령어를 익혔으니, 이번에는 컨테이너 내부를 살펴보겠습니다. 컨테이너 안은 리눅스 환경이기 때문에 기본적인 리눅스 명령어를 알아야 자유롭게 돌아다닐 수 있습니다. 이후 Dockerfile 작성, 설정 파일 수정, 로그 확인 등 실제 Docker 작업에서 리눅스 명령어는 계속 사용되니, 여기서 한 번 정리하고 넘어가겠습니다.

이미 리눅스 명령어에 익숙하다면 1.6 Docker 컨테이너 다루기로 건너뛰어도 됩니다.

1.5.1 Ubuntu : 환경 세팅

리눅스 명령어 사용을 위해 Ubuntu 환경을 세팅해 보겠습니다.

[실습] Ubuntu 컨테이너를 백그라운드로 실행하는 명령어입니다. `-p 80:80` 은 포트포워딩 설정으로, 호텔 프런트에서 “305호 손님 찾습니다” 하면 내선전화로 연결해주는 것과 같은 원리입니다.

```
docker run -it -p 80:80 ubuntu # 호스트의 80포트로 요청 시 컨테이너의 80포트로 요청 전달
```

포트포워딩(Port Forwarding)은 호스트 PC의 특정 포트에 들어오는 요청을 컨테이너 내부의 포트에 전달하는 기능입니다. `-p 호스트포트:컨테이너포트` 형식으로 사용합니다. 예를 들어, `-p 9000:8080` 은 호스트의 9000번 포트에 들어온 요청을 컨테이너의 8080번 포트에 전달합니다.

`-it` 옵션은 `-i` (입력 가능), `-t` (터미널 환경 제공)를 조합한 옵션입니다.



그림 0-18: Ubuntu 컨테이너 실행

1.5.2 탐색 명령어 : pwd, cd, ls, clear

컨테이너 내부에서 현재 위치를 확인하고 원하는 디렉토리로 이동하며 파일 목록을 조회하는 기본 명령어입니다.

명령어	설명	예시
<code>pwd</code>	현재 위치 경로 출력	<code>/root</code>
<code>cd <경로></code>	해당 폴더로 이동	<code>cd home</code>
<code>cd ..</code>	상위 폴더로 이동	
<code>ls</code>	현재 폴더의 파일/폴더 목록 출력	
<code>ls -l</code>	상세 정보와 함께 목록 출력	
<code>ls -a</code>	숨김 파일 포함 전체 출력	
<code>ls -la</code>	숨김 파일 포함 상세 출력	
<code>clear</code>	터미널 화면 비우기	

절대 경로와 상대 경로

절대 경로와 상대 경로는 최상위 폴더(/)로부터 폴더를 찾아가는 방식을 절대 경로라고 하며, 현재 위치에서 폴더를 찾는 방식을 상대 경로라고 합니다.

예를 들어, /home/ubuntu 경로에서 /bin 폴더로 이동하려면 상대 경로 `cd bin` 으로는 찾을 수 없고, 절대 경로 `cd /bin` 으로 이동해야 합니다.

먼저 `cd /home/ubuntu` 로 이동한 뒤, 상대 경로와 절대 경로의 차이를 확인해 보겠습니다.

```

실행 결과

root@c0b6dd5261d8:/home/ubuntu# cd bin
bash: cd: bin: No such file or directory

root@c0b6dd5261d8:/home/ubuntu# cd /bin
root@c0b6dd5261d8:/bin#
  
```

그림 0-20: 절대 경로로 이동

실행 후 `cd /` 로 루트 경로로 돌아갑니다.

1.5.3 파일/폴더 관리 : `mkdir`, `touch`, `rm`, `cp`, `mv`

컨테이너 내부에서 폴더를 만들고, 파일을 생성하거나 복사·이동·삭제하는 명령어입니다.

명령어	설명	예시
<code>mkdir <폴더명></code>	폴더 생성	<code>mkdir hello</code>
<code>touch <파일명></code>	빈 파일 생성	<code>touch a.txt</code>
<code>rm <파일명></code>	파일 삭제	<code>rm a.txt</code>
<code>rm -r <폴더명></code>	폴더 삭제 (하위 포함)	<code>rm -r hello</code>
<code>cp <원본> <사본></code>	파일 복사	<code>cp a.txt b.txt</code>
<code>mv <원본> <대상></code>	파일 이동 또는 이름 변경	<code>mv a.txt /tmp</code>

`mv` 는 같은 경로에서 파일명만 변경하는 용도로도 사용할 수 있습니다. 예: `mv b.txt c.txt`

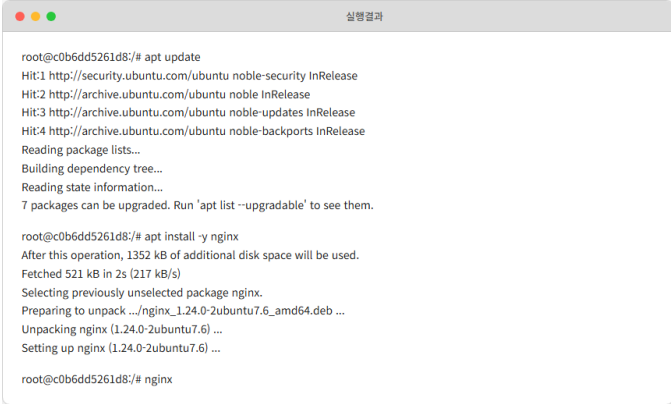
1.5.4 패키지 관리 : apt

컨테이너에는 기본적으로 최소한의 프로그램만 설치되어 있습니다. 이때 **apt** 명령어는 필요한 도구(nginx, vim 등)를 직접 설치할 때 사용하는 패키지 관리 명령어입니다.

명령어	설명
<code>apt update</code>	설치 가능한 패키지 목록을 최신 상태로 갱신
<code>apt list grep <키워드></code>	패키지 검색
<code>apt install -y <패키지명></code>	패키지 설치 (-y: 자동 승인)

[실습] 패키지 목록 갱신 후 **nginx**를 설치하고 실행하는 명령어입니다.

```
apt update      # 패키지 목록 갱신
apt install -y nginx # 패키지 설치
nginx          # 패키지 실행
```



```

root@c0b6dd5261d8:/# apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists...
Building dependency tree...
Reading state information...
7 packages can be upgraded. Run 'apt list --upgradable' to see them.

root@c0b6dd5261d8:/# apt install -y nginx
After this operation, 1352 kB of additional disk space will be used.
Fetched 521 kB in 2s (217 kB/s)
Selecting previously unselected package nginx.
Preparing to unpack ../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx (1.24.0-2ubuntu7.6) ...

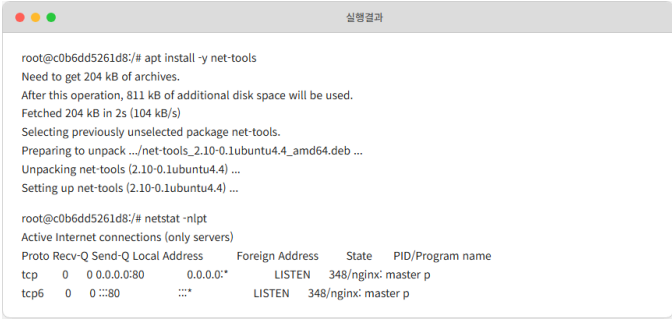
root@c0b6dd5261d8:/# nginx
  
```

그림 0-23: nginx 설치 및 실행

nginx 실행 후 포트 확인을 위해 **net-tools** 를 설치합니다.

[실습] net-tools를 설치하고 포트 상태를 확인하는 명령어입니다.

```
apt install -y net-tools
netstat -nlpt
```



```

root@c0b6dd5261d8:/# apt install -y net-tools
Need to get 204 kB of archives.
After this operation, 811 kB of additional disk space will be used.
Fetched 204 kB in 2s (104 kB/s)
Selecting previously unselected package net-tools.
Preparing to unpack .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4.4) ...
Setting up net-tools (2.10-0.1ubuntu4.4) ...

root@c0b6dd5261d8:/# netstat -nlpt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address   Foreign Address State    PID/Program name
tcp        0      0 0.0.0.0:*        LISTEN      348/nginx: master p
tcp6       0      0 :::*            LISTEN      348/nginx: master p

```

그림 0-24: 포트 상태 확인

80 포트가 실행되고 있는 것을 확인할 수 있습니다. 브라우저에 `localhost:80` 으로 접속하면 nginx 페이지가 응답합니다.



그림 0-25: nginx 페이지 응답

1.5.5 텍스트 편집 : vim

vim은 리눅스에서 가장 흔히 사용되는 텍스트 편집기로, 서버 환경에서 설정 파일을 다룰 때 사용됩니다. 먼저 vim 패키지를 설치합니다.

[실습] vim 패키지를 설치하는 명령어입니다.

```
apt install -y vim
```



그림 0-26: vim 패키지 설치

설치 도중 Geographic area와 Time zone을 선택하는 화면이 나타납니다. 이때 **Asia , Seoul** 을 각각 선택하면 됩니다.

vim의 핵심 사용 흐름은 다음과 같습니다.

단계	동작	키
1	파일 열기/생성	<code>vim <파일명></code>
2	입력 모드 전환	<code>i</code>
3	내용 편집	자유롭게 입력
4	일반 모드로 복귀	<code>ESC</code>
5	저장 후 종료	<code>:wq</code> 입력 후 Enter

[실습] vim으로 파일을 생성하는 명령어입니다.

```
vim test1.txt # test1.txt 파일 생성
```

생성된 txt 파일에서 `i` 를 눌러 입력 모드로 전환하고 내용을 작성한 뒤, `ESC` → `:wq` 로 저장합니다.



그림 0-28: vim 편집 화면

이제 `cat` 명령어로 파일 내용을 확인해 봅니다.

[실습] 파일 내용을 출력하는 명령어입니다.

```
cat test1.txt # 파일 내용 출력
```



그림 0-29: 파일 내용 출력

vim 명령 행 모드에서 **:q** 는 종료, **:q!** 는 저장하지 않고 강제 종료입니다.

1.5.6 프로세스 관리 : ps, kill

컨테이너 내부에서 실행 중인 프로세스를 확인하고, 필요 없는 프로세스를 종료할 때 사용하는 명령어입니다.

명령어	설명	예시
<code>ps -ef</code>	실행 중인 전체 프로세스 출력	
<code>ps -ef grep <키워드></code>	특정 프로세스 검색	<code>ps -ef grep nginx</code>
<code>kill <PID></code>	프로세스 안전 종료 (SIGTERM)	<code>kill 357</code>
<code>kill -9 <PID></code>	프로세스 강제 종료 (SIGKILL)	<code>kill -9 357</code>

1.5.7 파일 검색과 로그 확인 : find, tail

컨테이너 내부에서 설정 파일의 위치를 찾거나, 로그 파일의 최근 내용을 확인할 때 사용하는 명령어입니다.

명령어	설명	예시
<code>find <경로> -name <파일명></code>	파일 이름으로 위치 검색	<code>find / -name index.html</code>
<code>find <경로> -name <패턴></code>	패턴으로 검색 (* 사용)	<code>find / -name index*</code>
<code>tail <파일></code>	파일 마지막 10줄 출력	<code>tail access.log</code>
<code>tail -n <숫자> <파일></code>	마지막 N줄 출력	<code>tail -n 50 access.log</code>

[실습] 파일 이름으로 위치를 검색하는 명령어입니다.

```
find / -name index.html # index.html 파일 위치 검색
```



그림 0-32: 파일 검색 결과

리눅스 명령어가 익숙해졌으니, 이제 컨테이너 자체를 좀 더 자유자재로 다루는 법을 알아보겠습니다.

1.6 컨테이너 : 생명주기와 옵션

1.6.1 exit vs detach : 빠져나올 때 살아있는 경우 vs 죽은 경우

실행 중인 컨테이너에서 빠져나올 때, 방법에 따라 컨테이너가 종료될 수도 있고 계속 살아있을 수도 있습니다.

죽은 경우

다음 명령어를 실행해 컨테이너를 실행합니다. `--name` 옵션을 사용해 컨테이너의 이름을 지정할 수 있습니다.

[실습] 이름을 지정하여 `ubuntu` 컨테이너를 실행하는 명령어입니다.

```
docker run -it --name dead ubuntu # dead라는 이름의 ubuntu 컨테이너 실행
```

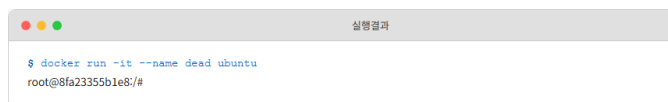


그림 0-33: 컨테이너 실행

컨테이너를 빠져나오기 위해 `exit` 를 입력합니다.

[실습] `exit`로 빠져나온 후 컨테이너 상태를 확인하는 명령어입니다.

```
exit
```

```
docker ps # 실행 중인 컨테이너 확인
```



```

root@8fa23355b1e8:/# exit
exit
$ docker ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES

```

그림 0-34: exit 후 컨테이너 종료

`exit` 명령어를 사용하면 컨테이너를 빠져나오는 동시에 프로세스가 종료됩니다.

살아있는 경우

이번에는 살아있는 경우를 확인해보겠습니다. 다음 명령어를 통해 컨테이너를 실행합니다.

[실습] 새로운 ubuntu 컨테이너를 실행하는 명령어입니다.

`docker run -it --name alive ubuntu` # alive라는 이름의 ubuntu 컨테이너 실행

```

$ docker run -it --name alive ubuntu
root@17943f60b5cd:/#

```

그림 0-35: alive 컨테이너 실행

이번에는 `CTRL + P` 입력 후 `CTRL + Q` 를 입력해 컨테이너를 빠져나옵니다.

프로세스를 유지한 채 빠져나오는 방법입니다.

[실습] 프로세스를 유지한 채 컨테이너에서 빠져나오는 단축키입니다.

CTRL + P -> CTRL + Q
docker ps

이번에는 컨테이너를 빠져나와도 컨테이너가 그대로 실행되는 것을 볼 수 있습니다.

```

$ docker ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
17943f60b5cd  ubuntu  "/bin/bash"  Less than a second ago  Up Less than a second  alive

```

그림 0-36: 프로세스 유지 확인

1.6.2 -dit : 백그라운드 인터랙티브 실행

`docker run` 사용 시에 `-dit` 옵션은 다음과 같습니다.

옵션	역할
<code>-d</code>	백그라운드 실행 (detached)
<code>-i</code>	입력 가능한 상태 유지 (interactive)
<code>-t</code>	터미널 환경 제공 (TTY)
<code>-dit</code>	위 세 옵션을 조합하여 백그라운드에서 터미널 입력이 가능한 상태로 실행

-d 옵션

하나씩 명령어를 실행해 보겠습니다. 먼저 **nginx** 컨테이너를 실행합니다.

[실습] -d 옵션으로 **nginx**를 백그라운드에서 실행하는 명령어입니다.

```
docker run -d nginx # nginx 백그라운드 실행
docker ps          # 실행 중인 컨테이너 확인
```

```

$ docker run -d nginx
46ef77d616b43617d5306e528e9b79f8d8b245b1e4a9e6777b9685d780699bf

$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS      NAMES
46ef77d616b4   nginx    "/docker-entrypoint..." 1 second ago Up Less than a second 80/tcp      vibrant_darwin

```

그림 0-38: nginx 백그라운드 실행

-d 옵션으로 **nginx** 컨테이너를 실행하면 백그라운드에서 프로세스가 실행됩니다.

이번에는 **ubuntu** 컨테이너를 실행해 봅니다.

[실습] -d 옵션으로 **ubuntu**를 실행하는 명령어입니다.

```
docker run -d ubuntu # ubuntu 백그라운드 실행
docker ps            # 실행 중인 컨테이너 확인
```

```

$ docker run -d ubuntu
0b74e66f5b0f3f4cb30b353503f2ef0c162e63f19be512e938f28e07d01dc1e7

$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS      NAMES
46ef77d616b4   nginx    "/docker-entrypoint..." 3 seconds ago Up 2 seconds 80/tcp      vibrant_darwin

```

그림 0-39: ubuntu -d 실행 후 종료

현재 실행 중인 컨테이너 중에 **ubuntu**는 없습니다.

ubuntu 컨테이너를 **-d** 옵션만 사용하면 백그라운드에서 실행되었다가 즉시 종료된 것입니다.

Docker 컨테이너는 메인 프로세스가 살아있는 동안만 유지됩니다. nginx는 웹 서버이므로 요청을 기다리며 혼자서 계속 실행됩니다. 반면 ubuntu의 메인 프로세스는 **bash** (사용자 명령을 받아 실행하는 프로그램)입니다. bash는 사용자가 명령을 입력해야 동작하기 때문에, **-it** 옵션으로 터미널을 연결하지 않으면 할 일이 없어 즉시 종료됩니다.

-dit 옵션

자, 그럼 **-dit** 옵션을 입력하면 어떻게 될까요?

[실습] **-dit** 옵션으로 백그라운드에서 실행하는 명령어입니다.

docker run -dit ubuntu # ubuntu 백그라운드 + 인터랙티브 실행

docker ps # 실행 중인 컨테이너 확인

```

$ docker run -dit ubuntu
462ed99bd1fa1b63a6cedf23aa8a61f5a1e32b631a3d9ebf54b30263c8913c9c

$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
462ed99bd1fa   ubuntu   "/bin/bash"             1 second ago  Up Less than a second   reverent_lehmann
46ef77d616b4   nginx    "/docker-entrypoint..." 4 seconds ago  Up 3 seconds        80/tcp         vibrant_darwin
  
```

그림 0-40: ubuntu -dit 실행

이번에는 ubuntu 컨테이너가 백그라운드에서 잘 실행되고 있습니다.

1.6.3 CMD : 컨테이너 시작 명령어 덮어쓰기

CMD(COMMAND) 는 컨테이너가 시작될 때 실행되는 기본 프로세스를 정의하는 명령어입니다.

docker run <이미지명> <CMD명령> 으로 직접 명령을 주면 **<CMD명령>** 옵션이 실행되며, 명령을 주지 않으면 이미지에 설정된 기본 CMD가 실행됩니다.

이번에는 CMD 명령을 추가하겠습니다. **sleep 1000** 은 프로세스가 1000초 동안 대기하도록 만드는 설정입니다.

[실습] CMD 명령을 지정하여 ubuntu를 실행하는 명령어입니다.

docker run -d ubuntu sleep 1000 # sleep 명령으로 ubuntu 컨테이너 유지

docker ps # 실행 중인 컨테이너 확인

`sleep 1000` 이 메인 프로세스가 되어 1000초 동안 대기하므로, `ubuntu` 컨테이너도 `-d` 옵션만으로 종료되지 않고 유지됩니다.

```

$ docker run -d ubuntu sleep 1000
7aca4e791643d3a39917aecc97868a62f5c1f6fa3df1ac63248e703650835cd3

$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS   NAMES
7aca4e791643   ubuntu   "sleep 1000"            Less than a second ago   Up Less than a second   goofy_bose
462ed99bd1fa   ubuntu   "/bin/bash"             2 seconds ago   Up 1 second         reverent_lehmann
46ef77d616b4   nginx    "/docker-entrypoint..." 5 seconds ago   Up 4 seconds       80/tcp   vibrant_darwin
  
```

그림 0-41: CMD로 프로세스 유지

1.6.4 attach : 실행 중인 컨테이너에 접근

이번에는 실행 중인 컨테이너 내부로 접근하는 방법을 살펴보겠습니다. 접근 방법에는 **attach** 와 **exec** 명령어가 있습니다.

attach 명령어 는 현재 실행 중인 메인 프로세스(PID 1) 에 직접 접근하는 명령어입니다.
`docker attach <컨테이너ID>` 명령어로 실행합니다.

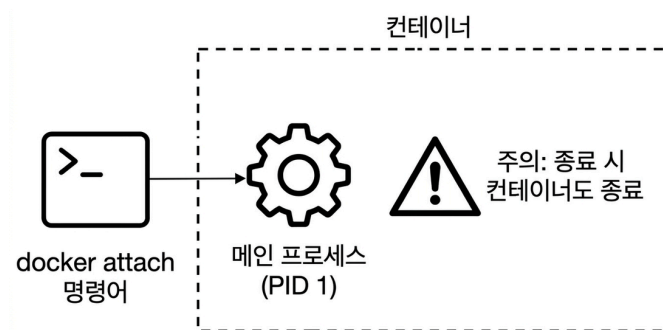


그림 0-42: attach 명령어의 동작 방식

백그라운드로 `ubuntu` 컨테이너를 실행합니다.

실행 중인 `ubuntu` 컨테이너에 `attach` 명령어로 접근하면 `ubuntu`의 입력 터미널창이 나타납니다.

[실습] `attach` 명령어로 실행 중인 컨테이너의 메인 프로세스에 접근하는 명령어입니다.

```

docker run -dit ubuntu # ubuntu 백그라운드 실행
docker ps             # 실행중인 컨테이너 확인
docker attach d2b1    # attach로 접근
  
```

```

$ docker run -dit ubuntu
d2b18acc2cf05a40644f0ca8536cb50a0ff560a1be0c9ce2fbf5ddbbba0e729c

$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
d2b18acc2cf0   ubuntu   "/bin/bash"   Less than a second ago   Up Less than a second   angry_engelbart

$ docker attach d2b1
root@d2b18acc2cf0:/#

```

그림 0-43: attach로 접근

attach 명령어는 메인 프로세스(**PID 1**)에 직접 연결되기 때문에, 여러 터미널에서 접속해도 모두 같은 화면과 같은 프로세스를 공유하게 됩니다.

다만 이 상태에서 잘못 빠져나오면서 메인 프로세스를 종료해 버리면 컨테이너 전체가 종료되니 주의해야 합니다.

CTRL + P 입력 후 **CTRL + Q**를 입력해 프로세스를 빠져나옵니다.

1.6.5 exec : 컨테이너에 새 프로세스로 접근

exec 명령어는 현재 실행 중인 메인 프로세스가 아닌, 메인 프로세스와 동일한 환경의 새로운 프로세스를 생성하여 접근하는 방식입니다. `docker exec <옵션> <컨테이너ID> <CMD>` 명령어로 실행합니다.

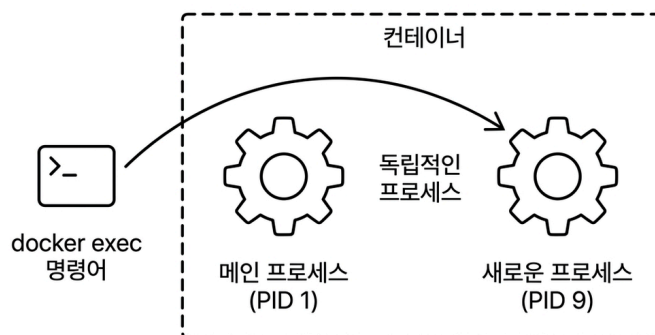


그림 0-44: exec 명령어의 동작 방식

아래 명령어를 실행해 프로세스 내부로 접근합니다.

[실습] **exec** 명령어로 컨테이너 내부에 새 프로세스를 생성하여 접근하는 명령어입니다.

docker exec -it d2b1 bash # 실행 중인 컨테이너에 새 **bash** 접속



그림 0-45: exec로 접근

exec 명령어로 프로세스에 접근 후 새로운 터미널 창을 실행합니다.



그림 0-46: 새 터미널 창 실행

새롭게 실행한 터미널 창에 아래 명령어를 실행합니다. 특정 컨테이너 내부에 실행 중인 프로세스 정보를 출력합니다.

[실습] 컨테이너 내부에서 실행 중인 프로세스 목록을 출력하는 명령어입니다.

docker exec d2b1 ps aux # 컨테이너 내부 프로세스 목록 확인

결과를 조회해보면 PID가 1인 프로세스가 메인 프로세스입니다. 그리고 **PID**가 **12**인 프로세스가

exec 명령어로 접근했을 때 생성된 프로세스입니다.



그림 0-47: 프로세스 목록 확인

exec 명령어로 실행된 프로세스는 메인 프로세스(**PID 1**)에 영향을 주지 않고 독립적으로 명령을 실행합니다.

다만 환경변수는 프로세스마다 따로 관리되기 때문에 메인 프로세스의 셸에서 새로 추가한 환경 변수는 **exec** 프로세스에서 보이지 않습니다.

1.7 이미지 : 직접 만들어보기

컨테이너를 실행하고 내부를 조작하는 법까지 익혔습니다. 이제 컨테이너 안에서 작업한 내용을 다른 사람과 공유하려면 어떻게 해야 할까요? 바로 컨테이너를 이미지로 만들면 됩니다.

Tomcat 컨테이너를 직접 수정하고, 그 상태를 이미지로 구워 보겠습니다.

1.7.1 Tomcat : 이미지 내려받기 & 실행하기

아래 명령어로 Tomcat 컨테이너를 실행합니다.

[실습] Tomcat 컨테이너를 실행하는 명령어입니다.

```
docker run -d -p 8080:8080 tomcat # Tomcat 컨테이너 백그라운드 실행
```



그림 0-48: Tomcat 컨테이너 실행

`docker ps` 명령어를 통해 현재 실행 중인 Tomcat 컨테이너를 확인해 봅니다. 현재 컨테이너 ID 는 `5fcd` 입니다.

브라우저에 `localhost:8080` 으로 요청을 하면 Tomcat이 응답한 화면이 나타납니다.

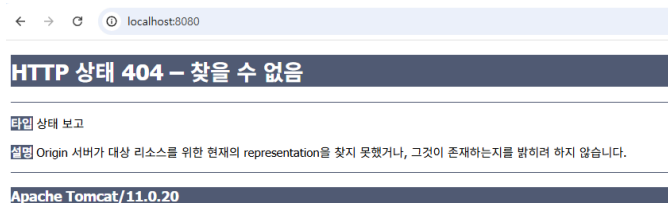


그림 0-49: Tomcat 404 에러

웹 서버에서 별도 컨트롤러가 없다면 슬래시(/)로 받는 요청은 서버 내의 `index.html`을 응답합니다. 그런데 Tomcat 이미지에 기본 제공되는 `index.html`이 없어서 요청에 응답할 페이지가 없어 404 에러가 발생합니다.

1.7.2 Tomcat 컨테이너 수정 : index.html 만들기

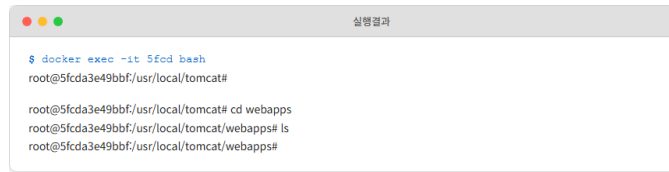
Tomcat의 `index.html` 파일은 `webapps/ROOT` 폴더에 위치합니다. 한번 Tomcat 프로세스 내부로 접근해서 해당 경로로 이동해 보겠습니다.

[실습] Tomcat 컨테이너 내부로 접근하는 명령어입니다.

```
docker exec -it 5fcd bash # Tomcat 프로세스 연결
cd /usr/local/tomcat/webapps # webapps 경로 이동
```

webapps 경로는 `find / -name webapps` 등 검색을 통해 찾을 수 있습니다.

webapps 폴더에서 `ls` 명령어로 확인하면 내부가 비어있습니다.



```

$ docker exec -it 5fcd bash
root@5fcd3e49bbf:/usr/local/tomcat#
root@5fcd3e49bbf:/usr/local/tomcat# cd webapps
root@5fcd3e49bbf:/usr/local/tomcat/webapps# ls
root@5fcd3e49bbf:/usr/local/tomcat/webapps#
  
```

그림 0-50: webapps 폴더 비어있음

webapps 폴더에 ROOT 폴더를 생성하고, 내부에 index.html 파일을 만들어 보겠습니다.

index.html 파일을 생성하려면 vim 패키지를 설치해야 합니다. 아래 명령어를 실행해 패키지 설치 후 vim 편집 에디터를 실행합니다.

[실습] vim 패키지를 설치하고 index.html 파일을 생성하는 명령어입니다.

```

mkdir ROOT      # ROOT 폴더 생성
cd ROOT        # ROOT 폴더로 이동

apt update      # 패키지 최신화
apt install -y vim # vim 패키지 설치
vim index.html  # index.html 파일 생성
  
```

편집 에디터에서 `i` 키를 눌러 입력 모드로 전환한 후, 내용 입력이 완료되면 `ESC` 를 눌러 입력 모드에서 일반 모드 전환 후 `:wq` 명령어로 내용을 저장합니다.



```

<h1>Hello Docker</h1>
<h3>index.html</h3>
~
~
~
:wq
  
```

그림 0-51: index.html 작성

다시 브라우저에 `localhost:8080` 으로 요청을 하면 우리가 만든 index.html 페이지가 응답됩니다.



그림 0-52: index.html 응답 확인

1.7.3 docker commit : 이미지 굽기

이제 수정한 Tomcat의 이미지를 저장해보겠습니다.

우선 **CTRL + P** 입력 후 **CTRL + Q** 를 입력해 실행 중인 컨테이너 프로세스를 빠져나옵니다.

그리고 다음 명령어로 commit을 실행합니다.

docker commit 형태로 명령어를 작성합니다. Dockerhub에서 계정과 리포지토리를 찾는 데 사용되므로, 본인의 Dockerhub 아이디로 변경해야 합니다.

[실습] 컨테이너의 현재 상태를 새 이미지로 저장하는 명령어입니다.

docker commit 5fcd coderyu5523/tomcat # 컨테이너 현재 상태를 이미지로 저장

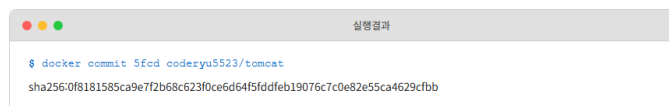


그림 0-53: 이미지 커밋 완료

1.7.4 docker push : Dockerhub에 저장

먼저 **docker login** 명령어를 실행해 Dockerhub에 로그인합니다.

[실습] Dockerhub에 로그인하는 명령어입니다.

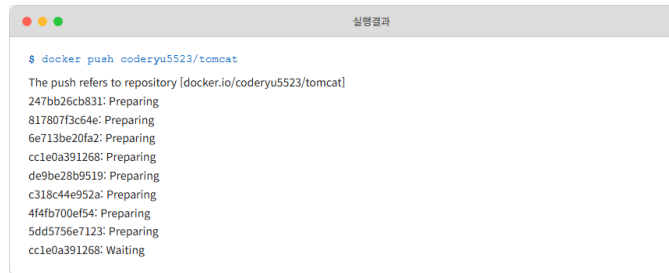
docker login # Dockerhub 로그인
ENTER 입력

아래 명령어를 사용해 내 Dockerhub에 저장합니다.

docker push <이미지명> 형태로 명령어를 실행합니다.

[실습] 이미지를 Dockerhub에 업로드하는 명령어입니다.

`docker push coderyu5523/tomcat` # 이미지를 Dockerhub에 업로드



```
$ docker push coderyu5523/tomcat
The push refers to repository [docker.io/coderyu5523/tomcat]
247bb26cb831: Preparing
817807f3c64e: Preparing
6e713be20fa2: Preparing
cc1e0a391268: Preparing
de9be28b9519: Preparing
c318c44e952a: Preparing
4f4fb700e154: Preparing
5dd5756e7123: Preparing
cc1e0a391268: Waiting
```

그림 0-54: 이미지 푸시 완료

Dockerhub의 Repositories 탭에서 업로드한 이미지를 확인할 수 있습니다.

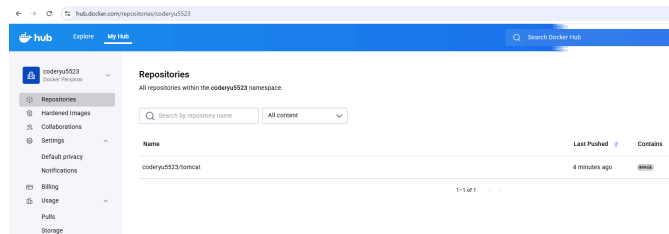


그림 0-55: Dockerhub 저장소 확인

1.8 마운트 : 컨테이너에 저장소 연결하기

컨테이너는 삭제 시 내부 데이터가 모두 사라지는 휘발성 구조입니다. 따라서 중요 데이터나 로그를 영구적으로 저장하기 위해서는 마운트를 통해 외부 저장소와 연결하는 과정이 필요합니다.

마운트(Mount)는 컨테이너 내부의 경로를 외부 저장소에 연결하는 기능입니다. 마운트를 사용하면 컨테이너가 삭제되어도 데이터는 외부에 그대로 남아 있습니다.

Docker에서는 두 가지 마운트 방식을 제공합니다. 로컬 PC의 폴더에 직접 연결해 데이터를 저장하는 바인드 마운트(Bind Mount)와 Docker가 관리하는 저장소를 사용하는 볼륨 마운트(Volume Mount)입니다.

1.8.1 바인드 마운트 : 호스트 폴더를 직접 연결

바인드 마운트(Bind Mount)는 호스트 PC의 실제 폴더를 그대로 컨테이너 내부에 연결하는 방식입니다.

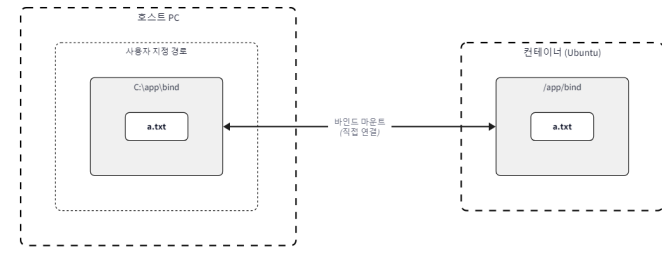


그림 0-56: 호스트 PC의 폴더와 컨테이너 내부 폴더가 직접 연결

실습해보기

테스트를 위해 아래 Ubuntu 환경에서 파일을 생성해 로컬 PC에 저장해 보겠습니다.

`docker run -it --mount type=bind,src=<로컬PC 경로>,dst=<컨테이너 경로> <이미지>` 형태로 명령어를 작성합니다.

[실습] 바인드 마운트를 설정하여 ubuntu 컨테이너를 실행하는 명령어입니다.

`docker run -it --mount type=bind,src=C:/app/bind,dst=/app/bind ubuntu` # 바인드 마운트로 ubuntu 실행

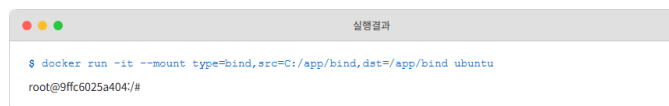


그림 0-57: 바인드 마운트 실행

명령어 실행 시 로컬 PC와 컨테이너 내부에 마운트한 `bind` 폴더가 생성됩니다.

[실습] `app` 폴더의 내부를 확인하는 명령어입니다.

`ls /app`



그림 0-58: bind 폴더 확인

이제 `/app/bind` 폴더에 `a.txt` 파일을 생성하겠습니다.

[실습] 마운트된 폴더에 파일을 생성하는 명령어입니다.

```
touch /app/bind/a.txt
```

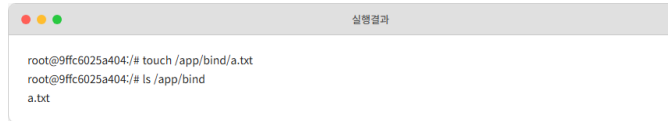


그림 0-59: 파일 생성 확인

생성한 파일이 PC의 하드디스크에 저장된 것을 확인할 수 있습니다.

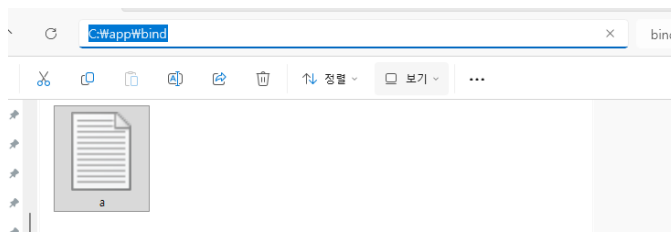


그림 0-60: 호스트 PC에 저장 확인

실습 후 `exit` 명령어로 컨테이너를 빠져나옵니다.

1.8.2 볼륨 마운트 : Docker가 관리하는 저장소

볼륨 마운트(**Volume Mount**)는 Docker 엔진 내부의 전용 저장 공간(Volume)을 컨테이너 폴더와 연결하는 방식입니다.

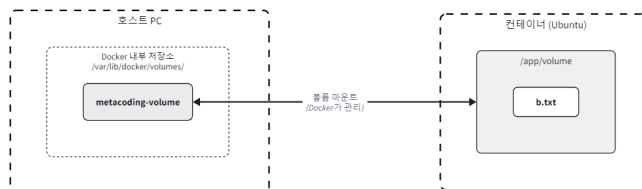


그림 0-61: Docker 엔진이 관리하는 내부 저장 공간에 데이터 저장

실습해보기

한번 테스트를 해봅시다. 다음 명령어를 실행해 현재 연결된 볼륨 정보를 확인합니다.

[실습] 현재 볼륨 목록을 확인하는 명령어입니다.

```
docker volume ls # 볼륨 목록 확인
```



그림 0-62: 볼륨 목록 조회

현재 연결된 볼륨은 없습니다. 아래 명령어를 실행해 볼륨이 연결된 컨테이너를 생성합니다.

`docker run -it --mount type=volume,src=<볼륨명>,dst=<컨테이너경로> <이미지>` 형태로 명령어를 실행합니다.

[실습] 볼륨 마운트를 설정하여 `ubuntu` 컨테이너를 실행하는 명령어입니다.

```
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu # 볼륨 마운트로 ubuntu 실행
```



그림 0-63: 볼륨 마운트 실행

`/app/volume` 폴더가 자동으로 생성됩니다.



그림 0-64: volume 폴더 생성

`/app/volume` 폴더 내부에 `b.txt` 파일을 생성합니다.

[실습] 볼륨 마운트된 폴더에 파일을 생성하는 명령어입니다.

```
touch /app/volume/b.txt
```



그림 0-65: b.txt 파일 생성

`b.txt` 파일이 정상적으로 생성되었다면 볼륨에 저장된 것입니다.

실행 중인 컨테이너를 `exit` 명령어로 종료한 후 볼륨 정보를 확인해 봅니다.

[실습] 컨테이너 종료 후 볼륨이 유지되는지 확인하는 명령어입니다.

```
exit
docker volume ls
```



그림 0-66: 볼륨 유지 확인

컨테이너는 종료되었지만 볼륨은 유지되고 있습니다.

다시 명령어를 실행해 새로운 컨테이너와 볼륨을 연결해 보겠습니다.

[실습] 새로운 컨테이너에서 기존 볼륨의 데이터를 확인하는 명령어입니다.

```
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu # 볼륨 마운트로
ubuntu 재실행
ls /app/volume
```



그림 0-67: 볼륨 데이터 재사용

새로 생성한 컨테이너에서도 이전 컨테이너가 사용하던 볼륨의 데이터를 그대로 확인할 수 있습니다. 볼륨이 컨테이너와 독립적으로 유지되기 때문입니다.

구분	바인드 마운트	볼륨 마운트
저장 위치	호스트 PC의 지정 폴더	Docker 엔진 내부 저장 공간
관리 주체	사용자 직접 관리	Docker 엔진이 관리
경로 지정	절대 경로 필요	볼륨 이름만 지정
사용 사례	개발 중 소스 코드 공유, 설정 파일 연결	DB 데이터 보존, 컨테이너 간 데이터 공유

■ 이것만은 기억하자

- 상자에 담으면 어디서든 똑같다. 말콤 맥린이 화물을 표준 컨테이너에 넣어 전 세계 어디든 보낸 것처럼, **Docker**는 애플리케이션을 컨테이너에 담아 어디서든 동일하게 실행합니다.
- 이미지는 레시피, 컨테이너는 요리. 하나의 이미지로 여러 컨테이너를 만들 수 있고, 수정한 컨테이너는 **commit** 으로 새 이미지를 만들어 **Dockerhub**에 공유할 수 있습니다.
- 마운트는 외부 **USB**. 컨테이너가 사라져도 마운트로 연결한 데이터는 외부에 안전하게 남아있습니다.

컨테이너를 만들고 실행하는 것까지는 할 수 있게 되었습니다. 아직 해결하지 못한 문제가 있습니다. 컨테이너를 새로 만들 때마다 매번 같은 패키지를 수동으로 설치해야 하고 사용자가 늘어나도 서버 한 대뿐이라 트래픽을 분산할 방법이 없습니다. 여러 컨테이너를 하나씩 따로 실행하고 관리하는 것도 한계가 있습니다.

다음 챕터에서는 이 문제들을 해결합니다. **Dockerfile**로 환경 구성을 자동화하고, **NGINX**로 트래픽을 분산합니다. **Redis**로 서버 간 세션을 공유하고, **MySQL**로 데이터를 영구 저장합니다. 그리고 **Docker Compose**로 이 모든 컨테이너를 한 줄의 명령어로 띄우는 풀스택 웹사이트를 만들어 봅니다.

챕터 2. Docker 다루기

■ 학습 목표

- Dockerfile로 컨테이너 환경 구성을 자동화한다.
- NGINX로 경로 기반 라우팅, 로드밸런싱, 캐싱을 구현한다.
- Redis로 여러 서버 간 세션을 공유한다.
- MySQL 컨테이너로 데이터를 영구 저장한다.
- Docker Compose로 여러 컨테이너를 한 번에 실행하고 관리한다.
- 프론트엔드, 백엔드, DB가 연동되는 풀스택 웹사이트를 만든다.

■ 2.1 프로비저닝 : 환경을 자동으로 구성하다

챕터 1에서 Ubuntu 컨테이너를 실행할 때마다 vim 패키지를 직접 설치해야 했습니다. 컨테이너를 새로 만들 때마다 같은 작업을 반복하는 건 비효율적입니다. 필요한 환경을 미리 준비해두고 컨테이너 실행 시 자동으로 적용되면 이런 반복을 줄일 수 있습니다. 이를 프로비저닝이라 합니다.

예를 들면, 매번 장을 보고 재료를 손질하고 양념을 만드는 과정을 반복하는 대신 밀키트를 쓰는 것과 같습니다. 밀키트는 재료와 양념이 이미 준비되어 있어서 봉지만 뜯으면 바로 조리할 수 있습니다. 프로비저닝도 마찬가지입니다. 필요한 패키지와 설정을 미리 정의해두면 컨테이너를 만들 때마다 자동으로 환경이 구성됩니다.

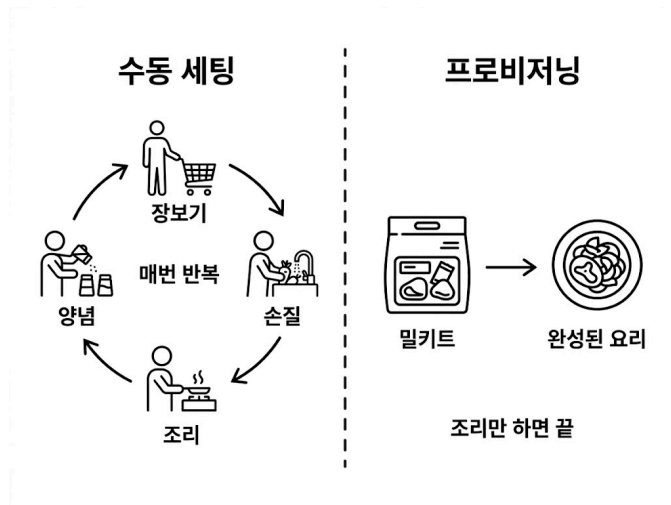


그림 0-1: 수동 세팅과 프로비저닝

프로비저닝(Provisioning): 도커에서 프로비저닝은 컨테이너가 처음 생성될 때 필요한 환경을 자동으로 세팅하는 과정을 의미합니다. 컨테이너가 실행되면 쓸 수 있는 설정, 패키지, 파일 등을 미리 준비하는 작업입니다.

2.1.1 Dockerfile : 프로비저닝 설계도

Docker에서는 프로비저닝을 위해 **Dockerfile** 을 사용합니다.

Dockerfile: 컨테이너가 실행될 때 필요한 환경을 자동으로 구성해주는 이미지를 생성하기 위한 스크립트로, 컨테이너를 만들기 위한 레시피입니다.

Dockerfile은 한 번 작성하면 누가, 어디서 실행해도 같은 환경이 만들어지는 자동 조립 설명서입니다. Dockerfile에 필요한 설정을 작성하고 `docker build` 명령을 실행하면 Docker 엔진이 Dockerfile을 읽고 이미지를 생성합니다.

이렇게 만들어진 이미지를 `docker run` 으로 실행하면 새로운 컨테이너가 생성되고, Dockerfile에서 구성한 설정이 적용됩니다.

Dockerfile에서 컨테이너가 실행되기까지 세 단계를 거칩니다.

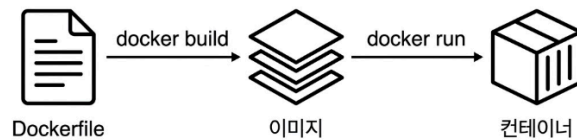


그림 0-2: Dockerfile에서 이미지 생성 및 컨테이너 실행 과정

1단계 – Dockerfile 작성. 텍스트 파일에 환경 구성을 적습니다. 베이스 이미지, 설치할 패키지, 복사할 파일, 실행할 명령을 순서대로 기록합니다.

2단계 – docker build. Docker 엔진이 Dockerfile을 위에서 아래로 읽으며 각 줄을 실행합니다. 결과물이 이미지(Image)로 저장됩니다. 이미지는 읽기 전용이며 한 번 만들어지면 변하지 않습니다.

3단계 – docker run. 이미지를 기반으로 컨테이너(Container)를 생성하고 실행합니다. 하나의 이미지에서 컨테이너를 몇 개든 만들 수 있습니다. 컨테이너를 삭제해도 이미지는 그대로 남아 있으므로 다시 실행하면 동일한 환경이 만들어집니다.

아래는 Dockerfile에서 사용하는 주요 설정입니다.

[참고] Dockerfile 주요 설정 구조

FROM

WORKDIR <기본 작업 경로 설정>

COPY <파일을 컨테이너 내부로 복사>

RUN <이미지 빌드 시 실행할 리눅스 명령 (패키지 설치 등)>

CMD <컨테이너 시작 시 메인 프로세스에 실행되는 명령어>

ENTRYPOINT <메인 프로세스를 지정하는 명령어>

2.1.2 빌드 시점과 실행 시점

Dockerfile의 설정은 실행되는 시점이 다릅니다. 이 구분을 알아야 각 역할이 명확해집니다.

빌드 시점 – **docker build** 를 실행할 때 처리

이 설정은 실행된 결과가 이미지에 기록됩니다.

지시어	역할	예시
<code>FROM</code>	사용할 이미지를 선택합니다. 모든 Dockerfile의 첫 줄입니다.	<code>FROM ubuntu:24.04</code>
<code>WORKDIR</code>	이후 지시어가 실행될 기본 경로를 지정합니다. 경로가 없으면 자동 생성됩니다.	<code>WORKDIR /app</code>
<code>RUN</code>	이미지 안에서 명령어를 실행합니다. 패키지 설치, 설정 변경 등에 사용합니다.	<code>RUN apt install -y vim</code>
<code>COPY</code>	호스트(내 PC)의 파일을 이미지 내부로 복사합니다.	<code>COPY index.html /app/</code>

실행 시점 — `docker run` 으로 컨테이너가 시작될 때 실행

`docker build` 로 이미지를 만들 때는 무시되고, `docker run` 으로 컨테이너를 띄우는 순간 비로소 실행됩니다.

지시어	역할	예시
<code>CMD</code>	컨테이너가 시작될 때 실행할 명령을 지정합니다. Dockerfile에 작성하지 않으면 베이스 이미지에 설정된 CMD가 실행됩니다.	<code>CMD ["/bin/bash"]</code>
<code>ENTRYPOINT</code>	컨테이너가 시작될 때 항상 실행되는 고정 명령어입니다. <code>CMD</code> 와 함께 쓰면 <code>CMD</code> 가 <code>ENTRYPOINT</code> 뒤에 붙어서 실행됩니다.	<code>ENTRYPOINT ["nginx"]</code>

2.1.2 Dockerfile : 스크립트 작성

이제 Dockerfile을 직접 작성해보겠습니다. Ubuntu 컨테이너가 실행될 때 자동으로 vim이 설치된 상태로 시작하도록 만들어 보겠습니다.

CURSOR IDE에서 `Dockerfile` 을 생성 후, 아래의 스크립트를 작성합니다.(별도 확장자 없이 파일명만 **Dockerfile**로 입력하면 됩니다.)

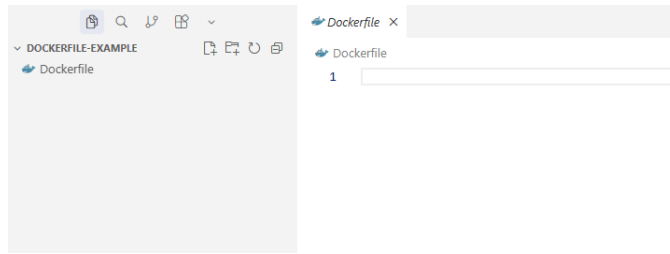


그림 0-5: Dockerfile 생성 완료

[작성] **Dockerfile** 을 아래와 같이 작성합니다.

```
FROM ubuntu:24.04          # Ubuntu 24.04 이미지 사용
RUN apt update && apt install -y vim # vim 패키지 설치
CMD ["/bin/bash"]          # 컨테이너 시작 시 bash 실행
```

이제 **docker build** 명령어를 실행해보겠습니다. **docker build <옵션> <이미지명(선택)> <경로>** 형태로 명령어를 작성합니다.

이미지 이름을 지정하려면 **docker build** 명령에 반드시 **-t** 옵션을 사용해야 합니다. 또 실행 경로와 **Dockerfile**이 동일한 경로에 있다면 **.** 을 사용합니다.

실습해보기

다음 명령어를 실행해 **Dockerfile** 기반 이미지를 생성합니다.

[실습] 터미널을 Dockerfile이 위치한 폴더로 이동 후, 다음 명령어를 실행합니다.

```
docker build -t ubuntu-vim . # .은 현재 경로를 기준으로 Dockerfile을 읽어옴
```



그림 0-6: docker build 실행 결과

생성된 이미지를 기반으로 컨테이너를 생성합니다.

[실습] **ubuntu-vim** 이미지로 컨테이너를 실행하는 명령어입니다.

```
docker run -it ubuntu-vim # ubuntu-vim 이미지로 컨테이너 실행
```



그림 0-7: 컨테이너 실행 및 접속

컨테이너 내부 터미널창에서 **vim** 명령어를 실행하면 **vim** 편집 에디터 창이 뜹니다.

[실습] vim 에디터로 파일을 생성하는 명령어입니다.

vim a.txt # a.txt 파일 생성

이렇게 Dockerfile 하나면 컨테이너의 원하는 환경을 동일하게 재현할 수 있습니다.



그림 0-8: vim 에디터 실행 확인

2.1.3 WORKDIR, COPY : 작업 경로와 파일 복사

WORKDIR: 기본 작업 폴더를 설정하는 옵션입니다. **COPY**: 파일을 컨테이너 내부로 복사하는 옵션입니다.

2.1.2에서 생성한 폴더 내부에 **index.html** 파일을 생성합니다. index.html의 내부는 비어있는 빈 파일입니다.

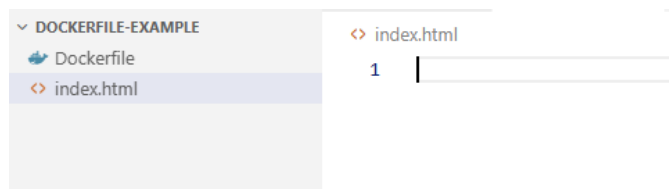


그림 0-9: 폴더 및 파일 구조

Dockerfile에 **WORKDIR**와 **COPY** 설정을 추가합니다. **WORKDIR**로 작업 디렉토리를 **/app** 으로 지정하고 **COPY**로 로컬의 이미지 파일을 컨테이너로 복사하면 됩니다.

[작성] **Dockerfile** 에 아래와 같이 **html** 파일을 복사하도록 수정합니다.

```

FROM ubuntu:24.04          # Ubuntu 24.04 이미지 사용
WORKDIR /app                # 기본 작업 경로를 /app으로 설정
COPY ./index.html ./index.html  # 로컬의 파일을 컨테이너의 /app으로 복사
RUN apt update && apt install -y vim # vim 패키지 설치
CMD ["/bin/bash"]          # 컨테이너 시작 시 bash 실행

```

실습해보기

이미지를 빌드합니다.

[실습] WORKDIR와 COPY가 적용된 이미지를 빌드하는 명령어입니다.

```
docker build -t ubuntu-html .
```

. 은 현재 경로를 기준으로 Dockerfile을 읽어옴

컨테이너를 생성합니다. 터미널에 접속하면 기본 경로가 `/app` 으로 설정되어 있고, 이 경로 안에서 `index.html` 파일을 확인할 수 있습니다.

[실습] 컨테이너를 실행하고 파일 목록을 확인하는 명령어입니다.

```
docker run -it ubuntu-html # ubuntu-html 이미지로 컨테이너 실행
ls
```



그림 0-10: 실행 결과 확인

실습 후 **exit** 명령어를 입력해 컨테이너에서 빠져나옵니다.

2.1.4 CMD, ENTRYPOINT : 기본 명령과 고정 명령

CMD는 `docker run` 에서 다른 명령을 지정하면 그 명령이 대신 실행됩니다. 반면 **ENTRYPOINT**는 무슨 옵션을 주든 항상 실행됩니다. 식당에 비유하면 CMD는 “기본 메뉴”라서 손님이 바꿀 수 있지만, ENTRYPOINT는 “수저, 물 같은 기본 세팅”과 같습니다.

CMD : 컨테이너가 시작될 때 실행할 명령을 지정합니다. Dockerfile에 작성하지 않으면 베이스 이미지에 설정된 CMD가 실행됩니다. **ENTRYPOINT**: 컨테이너가 시작될 때 반드시 실행되어야 하는 메인 프로세스를 지정하는 명령어입니다.

Dockerfile에 **ENTRYPOINT**를 추가해 `echo` 명령으로 메시지를 출력하도록 설정합니다.

[작성] `Dockerfile` 에 **ENTRYPOINT**를 추가합니다.

```
FROM ubuntu:24.04          # Ubuntu 24.04 이미지 사용
WORKDIR /app               # 기본 작업 경로를 /app으로 설정
COPY ./index.html ./index.html  # 로컬의 index.html을 컨테이너의 /app으로 복사
RUN apt update && apt install -y vim # vim 패키지 설치
CMD ["/bin/bash"]          # ENTRYPOINT가 있으면 CMD는 뒤에 붙어서 실행됨
ENTRYPOINT ["echo", "컨테이너 실행"] # 컨테이너 시작 시 항상 실행되는 명령
```

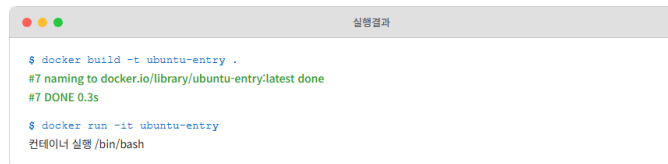
실습해보기

다음 명령어를 실행해 컨테이너를 실행합니다.

[실습] **ENTRYPOINT**가 적용된 이미지를 빌드 및 실행하는 명령어입니다..

```
docker build -t ubuntu-entry .    # 이미지 생성
docker run -it ubuntu-entry       # 컨테이너 실행
```

결과를 확인하면 **ENTRYPOINT**에 작성한 컨테이너 실행 `/bin/bash` 이 출력되고 프로세스는 즉시 종료됩니다. 왜 그럴까요?



```
$ docker build -t ubuntu-entry .
#7 naming to docker.io/library/ubuntu-entry:latest done
#7 DONE 0.3s
$ docker run -it ubuntu-entry
컨테이너 실행 /bin/bash
```

그림 0-11: ENTRYPOINT 실행 결과

ENTRYPOINT가 있으면 **CMD**는 독립적으로 실행되지 않고, **ENTRYPOINT** 뒤에 붙어서 함께 실행됩니다. 즉 `echo "컨테이너 실행"` + `/bin/bash` 가 합쳐져서 `echo "컨테이너 실행" /bin/bash` 가 된 것입니다.

`echo` 는 뒤에 오는 내용을 그대로 출력하고 끝나는 명령이라, “컨테이너 실행 `/bin/bash`”라는 글자만 화면에 찍고 종료됩니다. 메인 프로세스가 끝났으니 컨테이너도 즉시 종료됩니다.

다음 실습을 위해 실행한 컨테이너를 종료합니다.

[실습] 터미널 - 실행 중인 모든 컨테이너를 중지하고 삭제합니다.

```
docker stop $(docker ps -q) # 실행 중인 컨테이너 모두 중지
docker rm $(docker ps -aq)  # 중지된 컨테이너 모두 삭제
```

2.2 NGINX : 웹 서버와 리버스 프록시

이전 섹션에서는 Dockerfile로 컨테이너 환경을 자동화하는 방법을 배웠습니다. 하지만 서비스를 운영하다 보면 사용자가 늘어나고 백엔드 서버 한 대로는 트래픽을 감당하기 어려운 상황이 오게 됩니다. 챕터 1에서 `nginx` 이미지를 실행해봤는데, 사실 NGINX는 단순히 HTML을 보여주는 것 외에도 강력한 기능을 가지고 있습니다. 이번에는 NGINX의 다양한 기능들을 알아보겠습니다.

NGINX: 웹 서버이자 프록시 서버로, 정적 파일을 매우 빠르게 처리하고 백엔드 서버에 요청을 중계하는 역할을 합니다. 로드밸런싱과 HTTPS 처리는 물론 캐싱 기능까지 제공해서 대규모 트래픽을 안정적으로 처리하는 데 널리 사용됩니다.

2.2.1 NGINX : 동작 원리

NGINX는 안내데스크에서 방문객을 적절한 담당자에게 연결해주는 것과 같은 역할을 합니다. 클라이언트가 서버로 요청을 보내면 NGINX가 가장 앞단에서 요청을 분석한 뒤 상황에 맞게 로드밸런싱, 정적 파일 제공, 캐싱 처리 등을 수행합니다.

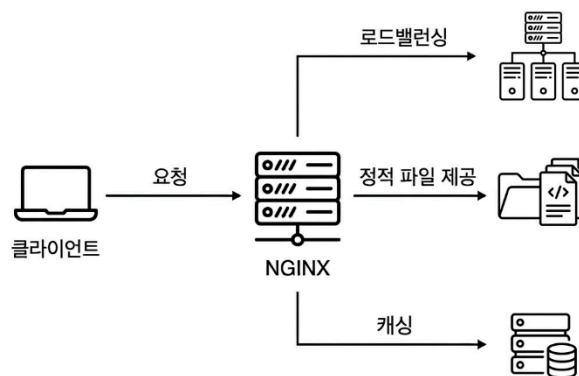


그림 0-12: NGINX의 주요 기능

리버스 프록시: NGINX처럼 서버를 대신해 요청을 받는 서버를 리버스 프록시라고 합니다. 리버스 프록시는 서버를 외부에 직접 노출하지 않도록 보호하고, 들어오는 트래픽을 효율적으로 분산해 서버의 부하를 줄여줍니다.

2.2.2 경로 기반 라우팅 : URL로 요청을 나누다

경로 기반 라우팅: NGINX의 경로 기반 라우팅은 클라이언트가 요청한 URL 경로를 기준으로, URL 경로에 해당하는 서버나 서비스로 트래픽을 전달하는 방식입니다.

아래 그림처럼 클라이언트가 `/users`, `/products` 경로로 API 요청을 보내면, NGINX는 해당 경로에 매핑된 서버로 요청을 전달합니다.

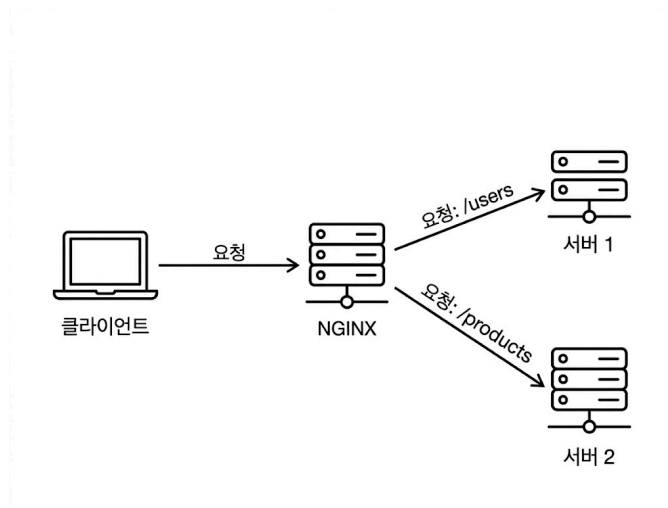


그림 0-13: 경로 기반 라우팅 구조

실습해보기

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex01> 에서 확인할 수 있습니다.

app1, app2, lb 폴더에 있는 Dockerfile은 개별 이미지를 생성하며, 각각 독립적인 컨테이너로 실행됩니다.

[EX01 패키지 구조]

```

ex01/
├── app1/           # 첫 번째 웹 서버
│   ├── Dockerfile
│   └── index.html
└── app2/           # 두 번째 웹 서버
  
```

```

|   ├── Dockerfile
|   ├── index.html
|   └── lb/          # 로드밸런서 (NGINX)
|       ├── Dockerfile
|       └── nginx.conf # 라우팅 설정

```

app 이미지

app1과 app2는 nginx 이미지를 기반으로 각각의 index.html을 복사하여 서버를 구성합니다.

파일	설명
<code>app1/Dockerfile</code> , <code>app2/Dockerfile</code>	nginx 이미지 기반, index.html 복사 후 포그라운드 실행
<code>app1/index.html</code> , <code>app2/index.html</code>	각 서버를 구분하는 HTML (Server1, Server2)

lb 이미지

lb 폴더의 Dockerfile은 로컬에 있는 nginx.conf를 컨테이너 내부로 복사하여 이미지를 생성합니다.

[참고] [lb/Dockerfile](#)

ex01/lb/Dockerfile

```

FROM nginx                # NGINX 이미지 사용
COPY nginx.conf /etc/nginx/conf.d/default.conf # 로컬의 nginx.conf를 컨테이너의 NGINX 설정 경로로 복사
ENTRYPOINT ["nginx", "-g", "daemon off;"]      # NGINX를 포그라운드로 실행

```

nginx.conf 파일은 다음과 같습니다. nginx.conf는 nginx 서버가 어떤 방식으로 요청을 처리할지 정의하는 파일입니다. `/app1` 요청은 app1(8000 포트)으로, `/app2` 요청은 app2(9000 포트)로 전달합니다.

upstream: 요청을 전달할 백엔드 서버 그룹을 정의하는 블록입니다. `proxy_pass` 에서 upstream 이름을 지정하면, NGINX가 해당 그룹에 등록된 서버로 요청을 전달합니다.

[참고] [lb/nginx.conf](#)

ex01/lb/nginx.conf

```

upstream app1 {                # app1 서버 그룹 정의
    server host.docker.internal:8000; # 호스트의 8000번 포트로 연결
}

upstream app2 {                # app2 서버 그룹 정의
    server host.docker.internal:9000; # 호스트의 9000번 포트로 연결
}

server {
    listen 80;                # 80포트로 요청시 설정 적용
    server_name localhost;

    location /app1 {          # 요청 url 기준으로 요청 전달
        proxy_pass http://app1; # upstream app1로 전달
    }

    location /app2 {          # 요청 url 기준으로 요청 전달
        proxy_pass http://app2; # upstream app2로 전달
    }
}

```

`host.docker.internal` 은 Docker 컨테이너 내부에서 호스트 PC에 접근하기 위한 특수 DNS 주소입니다. 컨테이너는 격리된 네트워크 환경에서 동작하기 때문에, `localhost` 로는 호스트 PC에 접근할 수 없습니다. 대신 Docker가 제공하는 `host.docker.internal` 주소를 사용하면 호스트 PC의 서비스에 접근할 수 있습니다.

이제 이미지를 실행해보겠습니다. 다음 명령어를 순차적으로 실행합니다.

챕터 1에서 배운 포트포워딩을 `-p 호스트포트:컨테이너포트` 형식으로 설정합니다.

[실습] EX01 폴더로 이동 후, 터미널에서 app1, app2, lb 이미지를 빌드하고 컨테이너를 실행합니다.

```

#서버 1 실행
docker build -t app1 ./app1 # app1 이미지 빌드
docker run -dit -p 8000:80 app1 # NGINX가 host.docker.internal:8000으로 접근하므로 호스트 8000번 포트를 열어줌

#서버 2 실행
docker build -t app2 ./app2 # app2 이미지 빌드

```

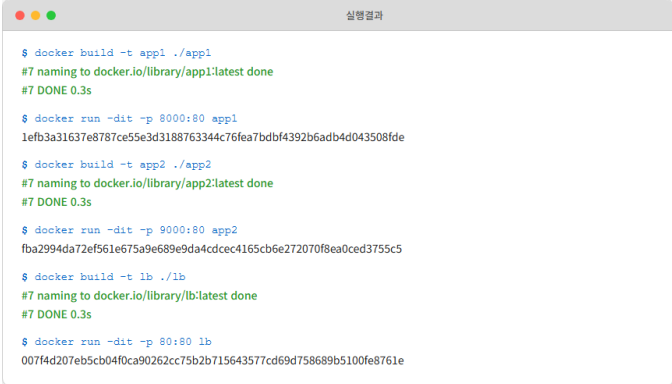
`docker run -dit -p 9000:80 app2` # NGINX가 `host.docker.internal:9000`으로 접근하므로 호스트 9000번 포트를 열어줌

#lb 실행

`docker build -t lb ./lb`

`docker run -dit -p 80:80 lb`

명령어를 실행하면 Docker Desktop에서 생성된 컨테이너를 확인할 수 있습니다.



```

$ docker build -t app1 ./app1
#7 naming to docker.io/library/app1:latest done
#7 DONE 0.3s

$ docker run -dit -p 8000:80 app1
1efb3a31637e8787ce55e3d318876344c76fea7bdf4392b6adb4d043508fde

$ docker build -t app2 ./app2
#7 naming to docker.io/library/app2:latest done
#7 DONE 0.3s

$ docker run -dit -p 9000:80 app2
fba2994da72ef561e675a9e689e9da4cdcec4165cb6e272070f8ea0ced3755c5

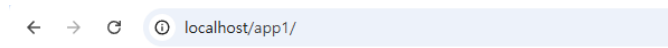
$ docker build -t lb ./lb
#7 naming to docker.io/library/lb:latest done
#7 DONE 0.3s

$ docker run -dit -p 80:80 lb
007f4d207eb5cb04f0ca90262cc75b2b715643577cd69d758689b5100fe8761e
  
```

그림 0-15: Docker Desktop에서 컨테이너 확인

브라우저에서 `localhost:80/app1` 으로 요청을 보내면 `app1` 서버가 응답합니다.

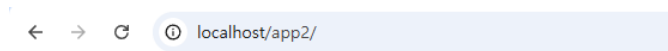
URL 주소는 NGINX 설정의 `location` 에 설정한 경로입니다.



Nginx Server 1

그림 0-16: /app1 경로 응답 결과

`localhost:80/app2` 로 요청을 보내면 `app2` 서버가 응답합니다.



Nginx Server2

그림 0-17: /app2 경로 응답 결과

다음 실습을 위해 실행한 서버를 종료합니다.

[실습] 터미널 - 실행 중인 모든 컨테이너를 중지하고 삭제합니다.

```
docker stop $(docker ps -q) # 실행 중인 컨테이너 모두 중지
docker rm $(docker ps -aq) # 중지된 컨테이너 모두 삭제
```

2.2.3 라운드 로빈 : 요청을 골고루 나누다

놀이공원 매표소가 3개 있을 때, 손님을 1번→2번→3번→1번 순서로 배정하는 것처럼 라운드 로빈 라우팅(**Round-Robin Routing**)은 여러 서버에 요청을 순차적으로 번갈아 가며 분배하는 로드 밸런싱 방식입니다.

API 요청이 있다면 순서대로 서버 1, 서버 2, 서버 3, 서버 1...의 순으로 트래픽이 분배됩니다.

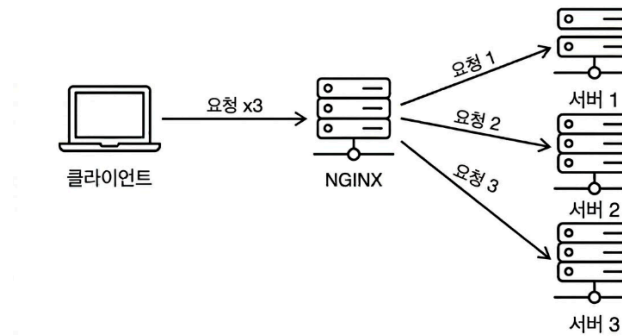


그림 0-18: 라운드 로빈 로드밸런싱 구조

실습해보기

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex02> 에서 확인할 수 있습니다.

[EX02 패키지 구조]

```
ex02/
├── app1/      # 웹 서버 (2개 컨테이너로 실행)
│   ├── Dockerfile
│   └── index.html
└── lb/        # 로드밸런서 (NGINX)
```

└─ Dockerfile
└─ nginx.conf # 라운드 로빈 설정

EX02는 EX01과 `lb/nginx.conf` 파일을 제외하고 동일한 구조이므로 코드는 생략하겠습니다.

lb 이미지

nginx.conf는 다음과 같습니다. `upstream` 설정에 두 개의 서버가 등록되어 있습니다.

NGINX는 별도의 로드 밸런싱 알고리즘을 지정하지 않으면 기본적으로 라운드 로빈 방식을 사용해 트래픽을 자동 분산합니다.

[참고] `lb/nginx.conf`

ex02/lb/nginx.conf

```
upstream app1 {                # app1 서버 그룹 정의
    server host.docker.internal:8000;    # 호스트의 8000번 포트로 연결
    server host.docker.internal:8001;    # 호스트의 8001번 포트로 연결
}

server {
    listen 80;                  # 80포트로 요청시 설정 적용
    server_name localhost;

    location /app1 {           # /app1 요청을 upstream으로 전달
        proxy_pass http://app1;      # 라운드 로빈으로 분배
    }
}
```

파일 작성 후 다음 명령어를 실행해 `app1`과 `lb`의 컨테이너를 생성합니다. 동일한 `app1` 이미지로 컨테이너 2개를 서로 다른 포트(8000, 8001)로 실행하는 것입니다.

[실습] EX02 폴더로 이동 후, 터미널에서 `app1` 이미지를 빌드하고, 2개의 컨테이너와 `lb`를 실행합니다.

```
#서버 1 2개 생성
docker build -t app1 ./app1    # app1 이미지 빌드

docker run -dit -p 8000:80 app1 # app1 서버 1 실행 (8000번 포트)
docker run -dit -p 8001:80 app1 # app1 서버 2 실행 (8001번 포트)
```

#nginx 실행

docker build -t lb ./lb**docker run -dit -p 80:80 lb** # 로드밸런서 실행 (80번 포트)

실행결과

```
$ docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
5d55f8e2db95	lb	Up Less than a second	0.0.0.0:80->80/tcp, [::]:80->80/tcp	lb
cla3b0bbe62c	app1	Up 3 seconds	0.0.0.0:8001->80/tcp, [::]:8001->80/tcp	app1-2
7a15eb92d23a	app1	Up 4 seconds	0.0.0.0:8000->80/tcp, [::]:8000->80/tcp	app1-1

그림 0-19: 라운드 로빈 컨테이너 실행 확인

이제 테스트를 해보겠습니다. **localhost:80/app1** 주소로 동일한 요청을 반복하면 요청이 두 서버로 번갈아 전달되는 것을 확인할 수 있습니다.

loving_galileo

ff8e235c79a2 app1:1ctsd

STATUS Running (3 minutes ago)

0000.00

Logs	Inspect	Bind mounts	Exec	Files	Stats
<pre> 2025/11/25 15:09:44 [notice] #1: getrlimit(RLIMIT_NOFILE): 1048576:1048576 2025/11/25 15:09:44 [notice] #1: start worker processes 2025/11/25 15:09:44 [notice] #1: start worker process 16 2025/11/25 15:09:44 [notice] #1: start worker process 17 2025/11/25 15:09:44 [notice] #1: start worker process 18 2025/11/25 15:09:44 [notice] #1: start worker process 19 2025/11/25 15:09:44 [notice] #1: start worker process 20 2025/11/25 15:09:44 [notice] #1: start worker process 21 2025/11/25 15:09:44 [notice] #1: start worker process 22 2025/11/25 15:09:44 [notice] #1: start worker process 23 2025/11/25 15:09:44 [notice] #1: start worker process 24 2025/11/25 15:09:44 [notice] #1: start worker process 25 2025/11/25 15:09:44 [notice] #1: start worker process 26 2025/11/25 15:09:44 [notice] #1: start worker process 27 172.17.0.1 - - [25/Nov/2025:15:11:23 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "172.17.0.1" 172.17.0.1 - - [25/Nov/2025:15:11:52 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "172.17.0.1" remote: Counting objects: 60% (2/3) remote: Counting objects: 100% (3/3) remote: Compressing objects: 100% (2/2) </pre>					

RAM 0.94 GB CPU 0.50% Disk: 19.95 GB used (limit 1006.85 GB)

그림 0-20: 8000 포트 서버 요청

Containers / festive_lichterman

festive_lichterman

22f65c73f3c5 app1:1ctsd

STATUS Running (4 minutes ago)

0001.00

Logs	Inspect	Bind mounts	Exec	Files	Stats
<pre> 2025/11/25 15:09:33 [notice] #1: start worker process 20 2025/11/25 15:09:33 [notice] #1: start worker process 21 2025/11/25 15:09:33 [notice] #1: start worker process 22 2025/11/25 15:09:33 [notice] #1: start worker process 23 2025/11/25 15:09:33 [notice] #1: start worker process 24 2025/11/25 15:09:33 [notice] #1: start worker process 25 2025/11/25 15:09:33 [notice] #1: start worker process 26 2025/11/25 15:09:33 [notice] #1: start worker process 27 2025/11/25 15:09:33 [notice] #1: start worker process 28 172.17.0.1 - - [25/Nov/2025:15:11:33 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "172.17.0.1" 172.17.0.1 - - [25/Nov/2025:15:12:08 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "172.17.0.1" remote: Counting objects: 60% (2/3) remote: Counting objects: 100% (3/3) remote: Compressing objects: 100% (2/2) remote: Compressing objects: 100% (2/2), done. Receiving objects: 60% (2/3) Receiving objects: 100% (3/3) Receiving objects: 100% (3/3), done. </pre>					

RAM 0.94 GB CPU 0.75% Disk: 19.95 GB used (limit 1006.85 GB)

그림 0-21: 8001 포트 서버 요청

다음 실습을 위해 실행한 서버를 종료합니다.

[실습] 터미널에서 실행 중인 모든 컨테이너를 중지하고 삭제합니다.

```
docker stop $(docker ps -q) # 실행 중인 컨테이너 모두 중지
docker rm $(docker ps -aq) # 중지된 컨테이너 모두 삭제
```

2.2.4 캐싱 : 정적 파일을 빠르게 전달하다

자주 입는 옷을 박스에 보관하지 않고 옷걸이에 걸어놓는 것처럼 NGINX의 정적 서버는 HTML, CSS, 이미지와 같은 정적 파일을 보관했다가 클라이언트에게 직접 제공합니다.

정적 서버와 캐싱: 정적 서버는 한 번 제공한 정적 파일을 일정 기간 동안 저장해 둡니다. 이후 동일한 요청이 오면 서버에 다시 조회하지 않고 저장된 파일을 즉시 반환합니다. 이를 캐싱이라고 합니다.

클라이언트가 서버에 처음 이미지 파일을 요청하면 NGINX는 서버에 요청을 전달해 이미지 파일을 응답받습니다.

일정 시간 이내에 동일한 요청이 다시 들어오면 NGINX는 서버 대신 캐시에 저장된 파일을 즉시 반환합니다.



그림 0-22: 첫 번째 요청 (MISS) - 캐시가 비어있어 백엔드 서버에 요청 후 응답을 캐시에 저장

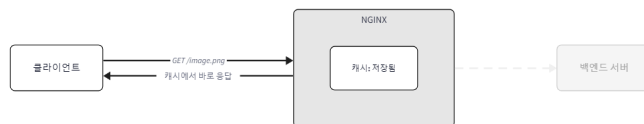


그림 0-23: 두 번째 요청 (HIT) - 캐시에 저장된 응답을 바로 반환, 백엔드 서버 접근 없음

실습해보기

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex03> 에서 확인할 수 있습니다.

[EX03 패키지 구조]


```

ex03/
├── api/           # 백엔드 서버 (Flask)
│   ├── app.py    # API 코드
│   └── Dockerfile
├── image.png      # 캐싱 테스트용 이미지
└── nginx/         # NGINX (캐싱 + 프록시)
    ├── Dockerfile
    └── nginx.conf # 캐싱 설정

```

api 이미지

Dockerfile은 다음과 같이 구성되어 있습니다.

[참고] [api/Dockerfile](#)

ex03/api/Dockerfile

```

FROM python:3.10-alpine    # Python 3.10 경량 이미지 사용
WORKDIR /app               # 작업 경로를 /app으로 설정
COPY . /app                # 현재 폴더의 모든 파일을 컨테이너의 /app으로 복사
RUN pip install flask      # Flask 웹 프레임워크 설치
CMD ["python", "app.py"]   # Flask 서버 실행

```

API 서버는 파이썬으로 구성되어 있습니다.

슬래시(/) 주소는 HTML을 응답하고, [/image.png](#) 주소는 이미지 파일을 응답합니다.

[참고] [api/app.py](#)

ex03/api/app.py

```

# Flask: 파이썬의 경량 웹 프레임워크 (스프링의 @RestController와 유사한 역할)
from flask import Flask, Response, send_file
import os

app = Flask(__name__)
# HTML 응답
@app.route("/")
def index():
    html = """
    <!DOCTYPE html>

```

```

    <h2>NGINX CACHE</h2>
  </body>
</html>
"""

return Response(html, mimetype="text/html")

# 이미지 요청
@app.route("/image.png")
def get_image():
    image_path = os.path.join(os.path.dirname(__file__), "image.png")
    return send_file(image_path, mimetype="image/png")

# ... 생략

```

NGINX 이미지

nginx의 Dockerfile은 다음과 같습니다.

[참고] [nginx/Dockerfile](#)

ex03/nginx/Dockerfile

```

FROM nginx                # NGINX 이미지 사용
COPY nginx.conf /etc/nginx/conf.d/default.conf # 로컬의 nginx.conf를 컨테이너의 NGINX 설정 경로로
복사
ENTRYPOINT ["nginx", "-g", "daemon off;"] # NGINX를 포그라운드로 실행

```

다음으로 nginx.conf 파일입니다. 이 파일에는 캐시 저장 경로를 지정하는 `proxy_cache_path` 와 캐시 사용 여부를 제어하는 `proxy_cache` 가 포함되어 있습니다.

`proxy_cache_path` 로 캐시 저장 경로와 메모리 공간을 지정하고 `location` 블록의 `proxy_cache` 로 캐시 적용 여부를 제어합니다.

[참고] [nginx/nginx.conf](#)

ex03/nginx/nginx.conf

```

# 캐시 관련 설정(캐시 저장 디렉토리, 캐시 이름 설정)
proxy_cache_path /var/cache/nginx keys_zone=my_cache:10m;

server {
    listen 80;

```

```

location / {
    proxy_pass http://host.docker.internal:5000;
    proxy_cache off;          # 캐시 사용하지 않음
}

location = /image.png {
    proxy_pass http://host.docker.internal:5000;
    proxy_cache my_cache;      # proxy_cache_path에서 설정한 캐시 사용
    proxy_cache_valid 200 1m;  # 200 상태 응답 1분 동안 캐시
    add_header X-Cache-Status $upstream_cache_status always; # response 헤더에 캐시 여부 표시
    proxy_ignore_headers Cache-Control Expires; # 백엔드의 캐시 정책을 무시하고 Nginx가 캐시
}
}

```

아래 명령어를 실행해 컨테이너를 생성합니다. API 서버를 먼저 실행한 뒤, NGINX 캐시 서버를 실행합니다.

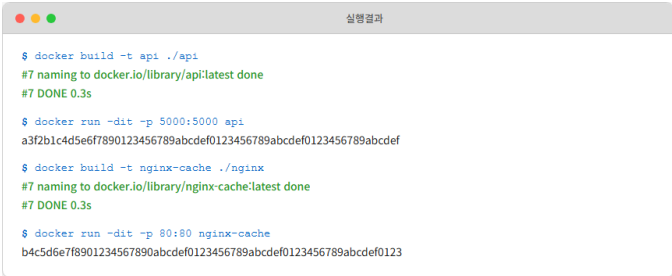
[실습] EX03 폴더로 이동 후, api 서버와 NGINX 캐시 서버를 빌드하고 실행합니다.

```

# api 서버 실행
docker build -t api ./api          # API 이미지 빌드
docker run -dit -p 5000:5000 api    # API 서버 실행 (5000번 포트)

# nginx 실행
docker build -t nginx-cache ./nginx # NGINX 캐싱 이미지 빌드
docker run -dit -p 80:80 nginx-cache # NGINX 캐싱 서버 실행 (80번 포트)

```



```

$ docker build -t api ./api
#7 naming to docker.io/library/api:latest done
#7 DONE 0.3s

$ docker run -dit -p 5000:5000 api
a3f2b1c4d5e6f7890123456789abcdef0123456789abcdef

$ docker build -t nginx-cache ./nginx
#7 naming to docker.io/library/nginx-cache:latest done
#7 DONE 0.3s

$ docker run -dit -p 80:80 nginx-cache
b4c5d6e7f8901234567890abcdef0123456789abcdef0123

```

그림 0-24: 캐싱 실습 - HTML 응답

컨테이너 실행 후 `localhost:80/image.png` 로 요청하면 저장된 이미지 파일이 응답됩니다.

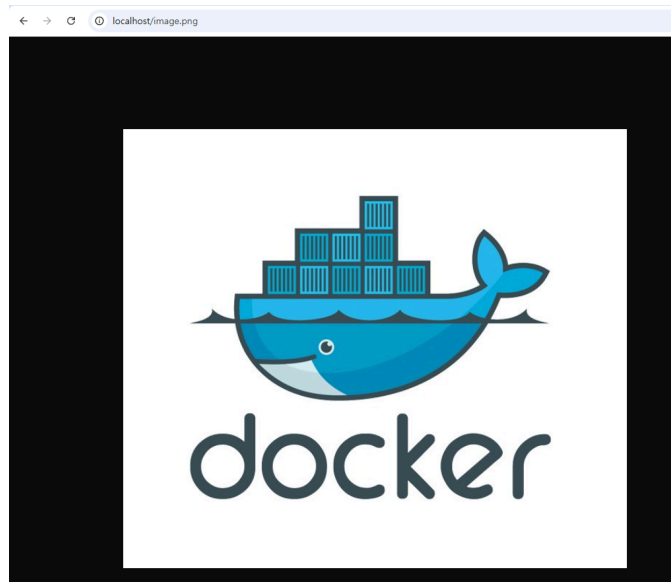


그림 0-25: 캐싱 실습 - 이미지 응답

브라우저 **F12 > Network > Headers** 탭을 확인합니다.

브라우저 자체도 캐시 기능이 있어 확인이 어려울 수 있습니다. 이때는 브라우저의 **Disable cache**를 체크해 브라우저의 캐싱을 비활성화해야 합니다.

이 중 Response Headers의 **X-Cache-Status** 값이 **MISS** 입니다. **MISS** 는 캐시에 요청 데이터가 없어 원본 서버에 요청을 보내 값을 가져온 상태입니다.

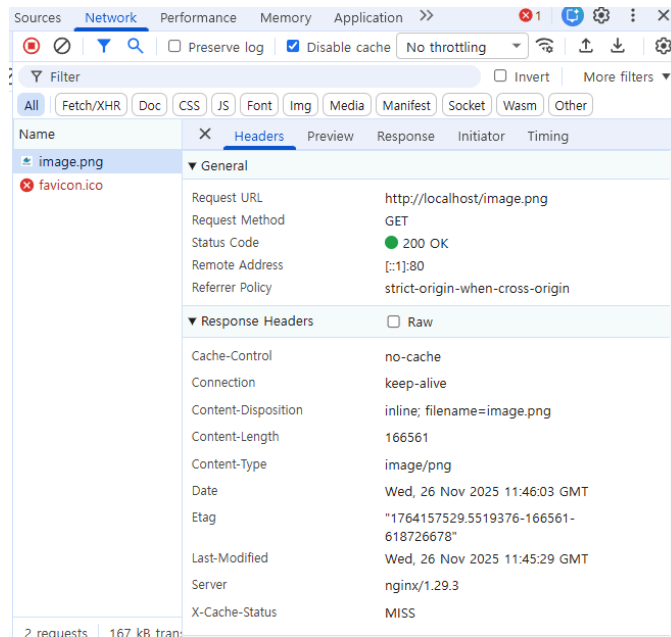


그림 0-26: X-Cache-Status: MISS 확인

다시 새로고침을 해보겠습니다. 이번에는 **X-Cache-Status** 값이 **HIT** 입니다.

HIT 는 캐시에 저장된 파일을 서버 요청 없이 그대로 응답한 상태입니다.

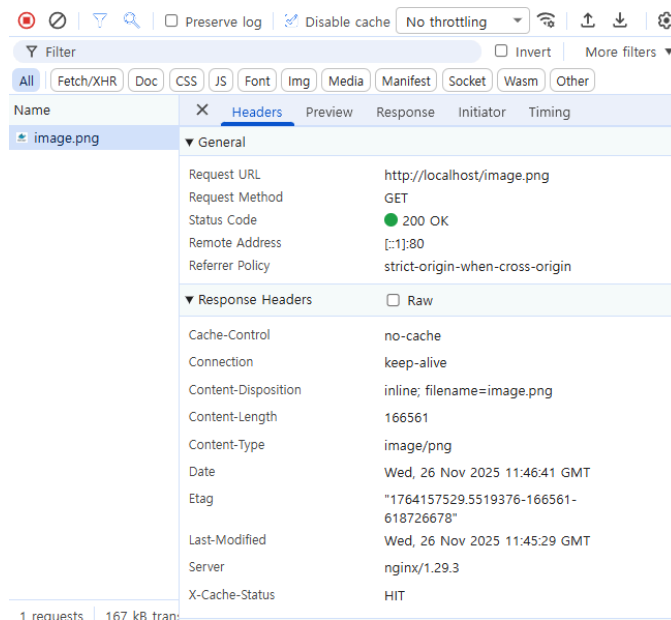


그림 0-27: X-Cache-Status: HIT 확인

다음 실습을 위해 실행한 서버를 종료합니다.

[실습] 터미널에서 실행 중인 모든 컨테이너를 중지하고 삭제합니다.

```
docker stop $(docker ps -q) # 실행 중인 컨테이너 모두 중지
docker rm $(docker ps -aq) # 중지된 컨테이너 모두 삭제
```

2.3 Redis : 세션 저장소

NGINX를 통해 여러 서버로 트래픽을 분산하는 방법을 배웠습니다. 그런데 로드밸런싱에는 숨겨진 함정이 있습니다. 사용자가 서버 1에 로그인했는데 다음 요청이 서버 2로 전달되면? 서버 2는 이 사용자가 로그인한 사실을 모르기 때문에 요청을 처리할 수 없습니다. 바로 여기서 Redis가 필요합니다.

Redis는 여러 서버가 함께 사용하는 공용 사물함과 같습니다. 서버 1이 사물함에 데이터를 넣어두면 서버 2도 같은 사물함을 열어 그 데이터를 꺼낼 수 있습니다. 어떤 서버가 요청을 처리하든 동일한 데이터에 접근할 수 있습니다.

레디스(Redis): 메모리 기반의 데이터베이스로, 관계형 데이터베이스가 아닌 키-값(Key-Value) 구조로 데이터를 저장합니다. 디스크가 아닌 메모리에 데이터를 저장하기 때문에 속도가 매우 빨라 데이터 캐싱, 사용자의 세션 저장 등 고성능 처리가 필요한 곳에 주로 사용됩니다.

2.3.1 세션 : 왜 외부 저장소가 필요한가

클라이언트가 로그인 요청을 보내면 NGINX는 해당 요청을 **서버 1**으로 전달합니다. 이때 로그인에 성공한 사용자의 세션 정보는 **서버 1**에서 생성되어 저장됩니다. 그런데 다음 요청이 NGINX에 의해 **서버 2**로 전달되면? **서버 2**는 해당 사용자의 세션 정보를 가지고 있지 않기 때문에 요청을 처리할 수 없게 됩니다.



그림 0-28: 세션 불일치 - 서버 1에 저장된 세션이 서버 2에는 없어 요청이 실패

Redis를 사용하면 이 문제를 해결할 수 있습니다. [서버 1](#) 이 세션을 자체 메모리가 아닌 Redis에 저장하면 이후 요청이 [서버 2](#) 로 전달되더라도 Redis에서 동일한 세션 정보를 조회할 수 있어 요청을 문제없이 처리할 수 있습니다.



그림 0-29: Redis로 해결 - 세션을 공유 저장소에 보관하여 어떤 서버에서든 조회 가능

서버에 세션을 저장하는 방식은 서버가 여러 대로 늘어날 때 세션이 분산되어 관리가 어렵다는 단점이 있습니다. 반면 Redis를 사용하면 세션을 한 곳에서 관리할 수 있어 서버가 확장되더라도 세션 공유에 문제가 없습니다.

2.3.2 Redis : 실습

아래 Github 주소를 참고합니다.

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex04> 에서 확인할 수 있습니다.

Dockerfile은 파이썬 이미지 기반으로 Flask와 Redis 패키지를 설치하고 app.py를 실행합니다.

[참고] [api/Dockerfile](#)

ex04/api/Dockerfile

```

FROM python:3.10-alpine          # Python 3.10 경량 이미지 사용
WORKDIR /app                    # 작업 경로를 /app으로 설정
COPY app.py .                   # app.py를 컨테이너의 /app으로 복사
RUN pip install flask && pip install redis    # Flask + Redis 패키지 설치
CMD ["python", "app.py"]        # Flask 서버 실행
  
```

[app.py](#) 는 아래와 같이 구성되어 있습니다.

[/save](#) 요청이 들어오면 Redis 서버와 연결된 객체 `r` 을 통해 값을 저장하고, [/read](#) 요청이 들어오면 동일한 객체를 사용해 Redis에 저장된 값을 조회합니다.

[참고] [api/app.py](#)

ex04/api/app.py

```
# Flask: 파이썬의 경량 웹 프레임워크 (스프링의 @RestController와 유사한 역할)
# @app.route()는 스프링의 @GetMapping과 같은 역할

# Redis 연결 -- 컨테이너 이름 'redis'를 호스트명으로 사용
r = redis.Redis(host='redis', port=6379, db=0)

@app.route("/save")
def save_name():
    r.set("name", "metacoding") # Redis에 값 저장
    return "이름이 저장되었습니다."

@app.route("/read")
def read_name():
    value = r.get("name") # Redis에서 값 조회
    return f"name = {value.decode('utf-8')}"

# ... 생략
```

다음 명령어를 실행합니다. **-network** 옵션으로 동일 네트워크에 연결하면 컨테이너 이름으로 통신할 수 있게 됩니다.

Docker 컨테이너는 각각 독립된 환경에서 실행되기 때문에, 서로 다른 컨테이너는 기본적으로 통신할 수 없습니다. API 서버와 Redis 서버도 별도의 컨테이너이므로 같은 네트워크에 연결해야 컨테이너 이름으로 서로 통신할 수 있습니다. 이때 **-network** 옵션을 사용하면 서로 다른 컨테이너를 하나의 네트워크로 묶을 수 있습니다.

[실습] EX04 폴더로 이동 후, 네트워크를 생성하고 Redis, API 컨테이너를 실행합니다.

```
# 네트워크 만들기
docker network create myNetwork # myNetwork 네트워크 생성

# redis 앱 빌드 및 실행
docker run -d --name redis --network myNetwork -p 6379:6379 redis # Redis 컨테이너 실행, myNetwork
네트워크로 연결

# api 실행
docker build -t api ./api # API 이미지 빌드
docker run -d --name api1 --network myNetwork -p 5001:5000 api # API 서버 1 실행 (5001번 포트),
myNetwork 네트워크로 연결
```



```
docker run -d --name api2 --network myNetwork -p 5002:5000 api # API 서버 2 실행 (5002번 포트),  
myNetwork 네트워크로 연결
```

3개의 컨테이너가 생성되었습니다.



```
실행결과

$ docker network create myNetwork
4e7a42fac6321f9506d97760c8767f1a77ebf919732b916b0a19b6be801d21b

$ docker run -d --name redis --network myNetwork -p 6379:6379 redis
0b116ddfa8d4679945612f7e3ce762f2e96bb1fb5e497a2d615aac388a329ea1

$ docker build -t api ./api
#9 DONE 2.2s

View build details: docker-desktop://dashboard/build/desktop-linux/mlpex2lh75w1anbwf08wonk1

$ docker run -d --name api1 --network myNetwork -p 5001:5000 api
a66a05566b78b8e654509906f9c9e64aed6ef6d10c85250202df039a6a64062

$ docker run -d --name api2 --network myNetwork -p 5002:5000 api
7245c10e125a3a2c2f9ba308e1e090e6509b6d2d022d1801df93c987fe0ef4b7
```

그림 0-30: Redis 실행 컨테이너 확인

브라우저에서 `api1` 서버의 `localhost:5001/save` 로 요청을 보내면 이름이 잘 저장되었다는 응답을 확인할 수 있습니다.

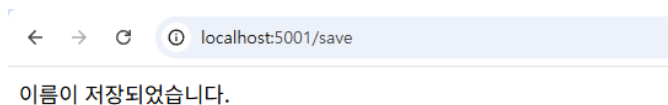


그림 0-31: api1에서 데이터 저장

그리고 `api2` 서버인 `localhost:5002/read` 로 요청을 보내면 `api1` 서버에서 저장했던 값을 확인할 수 있습니다. `api1`에서 저장한 데이터를 `api2`에서 읽을 수 있습니다.

Redis를 통해 컨테이너 간 데이터 공유가 제대로 이루어지고 있다는 뜻입니다.

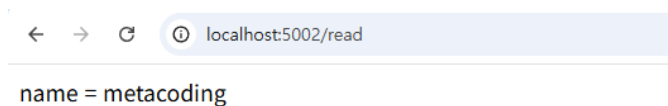


그림 0-32: api2에서 데이터 조회

실습이 끝나면 컨테이너와 네트워크를 정리합니다.

[실습] 터미널에서 컨테이너를 중지하고 삭제한 뒤 네트워크를 제거합니다.

```
docker stop $(docker ps -q) # 실행 중인 컨테이너 모두 중지
docker rm $(docker ps -aq) # 중지된 컨테이너 모두 삭제
docker network rm myNetwork # 네트워크 삭제
```

2.4 MySQL : DB 서버 구축

Redis로 세션 데이터를 관리하는 방법을 배웠습니다. 하지만 Redis는 메모리 기반이라 서버가 종료되면 데이터가 날아갈 수 있습니다. 회원 정보, 게시글, 주문 내역처럼 반드시 보관해야 하는 데이터는 어떻게 저장해야 할까요? 이때 필요한 것이 바로 DB 서버입니다.

2.4.1 MySQL : 컨테이너로 DB 서버 생성

실습해보기

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex05> 에서 확인할 수 있습니다.

[EX05 패키지 구조]

```
ex05/
├── db/          # MySQL 데이터베이스
│   ├── Dockerfile
│   └── init.sql # 초기 테이블 및 데이터 생성 SQL
```

Dockerfile은 MySQL 이미지를 기반으로 환경변수와 초기화 SQL을 설정합니다.

[참고] [db/Dockerfile](#)

ex05/db/Dockerfile

```
FROM mysql          # MySQL 이미지

COPY init.sql /docker-entrypoint-initdb.d # 로컬의 init.sql을 MySQL 초기화 폴더로 복사

ENV MYSQL_USER=metacoding # 생성할 사용자 이름
ENV MYSQL_PASSWORD=metacoding1234 # 사용자 비밀번호
ENV MYSQL_ROOT_PASSWORD=root1234 # root 비밀번호
```

```
ENV MYSQL_DATABASE=metadb # 생성할 데이터베이스 이름
```

```
CMD ["--character-set-server=utf8mb4", "--collation-server=utf8mb4_unicode_ci"] # 한글 인코딩 설정
```

`init.sql` 은 `user_tb` 테이블을 생성한 뒤, 초기 데이터를 입력합니다.

`init.sql`: 컨테이너가 처음 생성될 때 한 번 실행되는 스크립트입니다. MySQL 공식 이미지는 `/docker-entrypoint-initdb.d` 디렉토리에 있는 SQL 파일을 최초 실행 시 자동으로 실행합니다.

[참고] [db/init.sql](#)

ex05/db/init.sql

```
CREATE TABLE user_tb (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50)
);

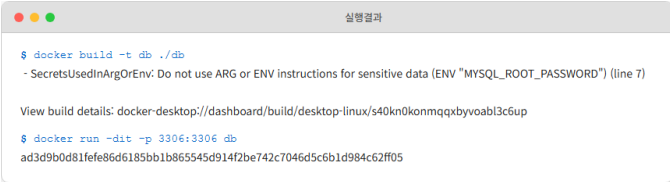
INSERT INTO user_tb (name) VALUES ('ssar');
INSERT INTO user_tb (name) VALUES ('cos');
```

다음 명령어를 실행합니다. MySQL의 기본 포트인 3306으로 포트 포워딩합니다.

[실습] EX05 폴더로 이동 후, DB 이미지를 빌드하고 컨테이너를 실행합니다.

```
# 이미지 빌드 및 실행
docker build -t db ./db # DB 이미지 빌드
docker run -dit -p 3306:3306 db # MySQL 컨테이너 실행 (3306번 포트)
```

컨테이너 로그를 확인해보면 MySQL이 잘 실행된 것을 확인할 수 있습니다.



```
실행결과
$ docker build -t db ./db
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_ROOT_PASSWORD") (line 7)

View build details: docker-desktop://dashboard/build/desktop-linux/s40kn0konmqxbyvoabl3c6up
$ docker run -dit -p 3306:3306 db
ad3d9b0d81fe8e6d6185bb1b865545d914f2be742c7046d5c6b1d984c62ff05
```

그림 0-33: MySQL 컨테이너 실행 로그

2.4.2 MySQL : 데이터 확인

2.4.1에서 DB 서버를 백그라운드로 실행했습니다. 이제 내부 터미널로 접속해 확인해보겠습니다.

`docker exec -it <컨테이너ID> bash` 명령어를 사용해 터미널로 접속합니다.

[실습] 터미널에서 실행 중인 컨테이너를 확인하고 내부에 접속합니다.

```
docker ps          # 실행 중인 컨테이너 확인
docker exec -it 1fc2 bash # MySQL 컨테이너 내부 접속
```



그림 0-34: MySQL 컨테이너 접속

터미널 창에 다음 명령어를 실행 후 MySQL에 접속합니다. 입력할 사용자명과 비밀번호는 Dockerfile에 설정한 환경 변수입니다.

`mysql -u <사용자명> -p` 형식으로 접속하며, 우리는 Dockerfile에 사용자를 설정했습니다.

[실습] 터미널에서 MySQL 클라이언트에 접속합니다.

```
mysql -u metacoding -p # MySQL 접속
```

비밀번호를 입력할 때는 보안상 화면에 표시되지 않지만 제대로 입력되고 있으니 그대로 입력하면 됩니다.

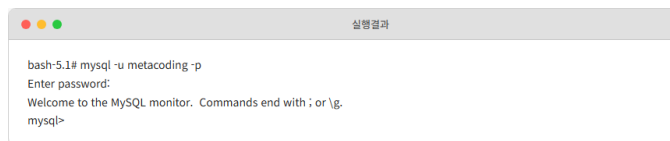
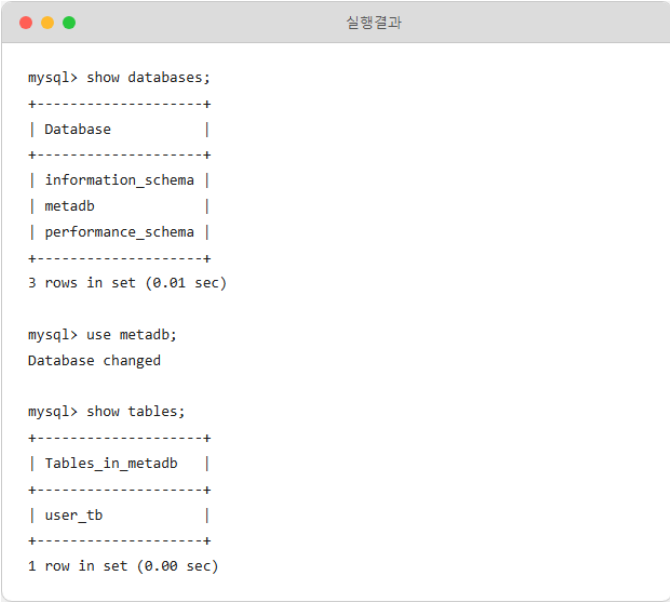


그림 0-35: MySQL 접속 성공

다음 SQL문을 사용해 DB 정보를 확인합니다.

[실습] MySQL에서 데이터베이스와 테이블 정보를 조회합니다.

```
show databases;
use metadb;
show tables;
select * from user_tb;
```



```

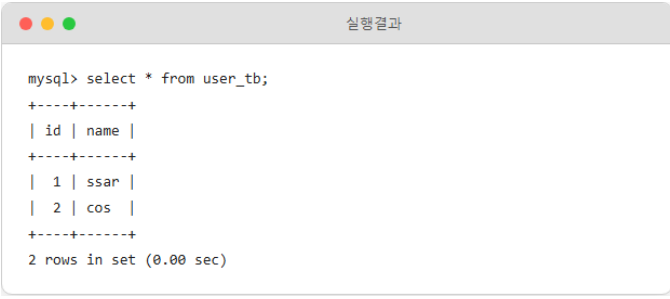
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| metadb    |
| performance_schema |
+-----+
3 rows in set (0.01 sec)

mysql> use metadb;
Database changed

mysql> show tables;
+-----+
| Tables_in_metadb |
+-----+
| user_tb          |
+-----+
1 row in set (0.00 sec)

```

그림 0-36: 데이터베이스 목록 확인



```

mysql> select * from user_tb;
+----+-----+
| id | name |
+----+-----+
|  1 | ssar |
|  2 | cos  |
+----+-----+
2 rows in set (0.00 sec)

```

그림 0-37: user_tb 데이터 조회

`init.sql`에 작성한 데이터가 잘 조회됩니다.

실습 후 터미널에서 빠져나와 컨테이너를 종료합니다.

[실습] 컨테이너를 종료 후 삭제하는 명령어입니다.

```

exit          # db 컨테이너에서 빠져나옴
docker stop $(docker ps -q)
docker rm $(docker ps -aq)

```

2.5 Docker Compose : 여러 컨테이너를 한 번에

컨테이너가 하나일 때는 `docker run` 한 줄이면 충분합니다. 그런데 실제 서비스는 프론트엔드, 백엔드, DB처럼 여러 컨테이너가 함께 동작합니다. 컨테이너가 5개라면 `docker run` 도 5번, 네트워크 생성도 따로, 내릴 때도 하나씩 내려야 합니다. 시작 순서를 틀리면 에러가 발생합니다. DB가 올라오기 전에 백엔드를 먼저 실행하면 연결에 실패합니다.

Docker Compose는 이 문제를 해결합니다. `docker-compose.yml` 이라는 YAML 파일 하나에 모든 컨테이너의 구성을 정의합니다.

2.5.1 Docker Compose : 왜 필요한가

Docker Compose는 오케스트라 악보와 같습니다. 각 악기(컨테이너)의 역할은 다르지만 하나의 악보로 동시에 연주할 수 있습니다.

도커 컴포즈(Docker Compose): 하나의 스크립트 파일로, 여러 컨테이너를 하나의 환경으로 묶어 관리할 수 있는 도구입니다.

지금까지 우리는 Dockerfile에 필요한 환경과 설정을 작성한 후 이미지를 생성, 실행해 컨테이너를 생성했습니다.

여러 컨테이너를 함께 실행하려면 Dockerfile마다 이미지를 따로 빌드하고 컨테이너도 하나씩 실행해야 했습니다. 컨테이너끼리 통신하려면 네트워크도 `docker network` 명령어로 직접 만들어야 했습니다.

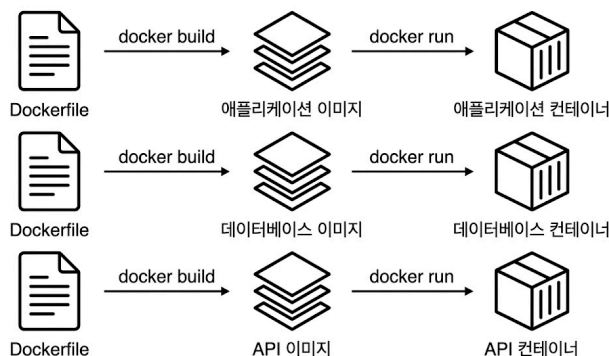


그림 0-38: 기존 방식 - Dockerfile 개별 빌드 및 실행

반면 Docker Compose 스크립트에 이미지 관리에 대한 작업을 정의하면 `docker compose up` 명령 한 번으로 여러 이미지를 동시에 실행하고 필요한 환경을 자동으로 구성할 수 있습니다.

Compose가 해결하는 것은 세 가지입니다.

순서 : 어떤 컨테이너를 먼저 띄울지 `depends_on` 으로 지정합니다. DB가 먼저 올라온 뒤 백엔드가 시작되도록 순서를 보장합니다.

네트워크 : 같은 Compose 파일에 정의된 컨테이너는 자동으로 하나의 네트워크에 묶입니다.

`docker network create` 를 직접 실행할 필요가 없습니다. 컨테이너끼리 서비스 이름으로 통신할 수 있습니다.

일괄 관리 : `docker compose up` 한 줄이면 모든 컨테이너가 시작됩니다. `docker compose down` 한 줄이면 모든 컨테이너와 네트워크가 정리됩니다.

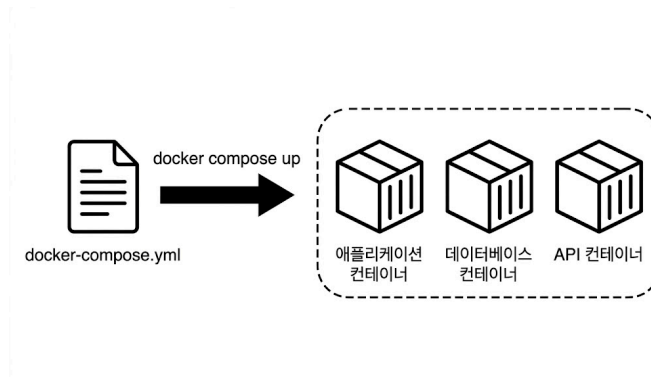


그림 0-39: Docker Compose 방식 — 한 번에 생성 및 연결

아래는 `docker-compose.yml`의 기본 구조입니다.

image 옵션 은 Dockerhub에서 가져올 이미지 이름을 지정합니다. 단, **build** 옵션 과 함께 사용하면 Dockerfile로 빌드한 이미지에 해당 이름을 지정할 수 있습니다.

[참고] `docker-compose.yml` 기본 구조

```
# 실행할 컨테이너
services:
  <서비스명>:
    container_name: <컨테이너명> # 컨테이너 이름 지정
    image: <이미지명>          # 이미지 이름 지정
    build: <경로>              # Dockerfile 경로 (이미지를 직접 빌드)
    ports:
      - "호스트포트:컨테이너포트" # 포트 매핑
    depends_on:
      - <다른서비스명>          # 이 서비스보다 먼저 시작해야 하는 서비스
    environment:
```

```

- KEY=VALUE           # 환경 변수 설정
volumes:
- <호스트경로:컨테이너경로> # 데이터 저장소 연결
networks:
- <네트워크명>          # 연결할 네트워크

# 저장소 지정
volumes:
<볼륨명>:

# 컨테이너간 통신을 위한 네트워크 선언
networks:
<네트워크명>:

```

2.5.2 Docker Compose : 실습

아래의 Github 주소를 참고합니다.

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex06> 에서 확인할 수 있습니다.

코드 내용은 EX01과 동일하며 docker-compose.yml 파일이 추가된 형태입니다.

[EX06 패키지 구조]

```

ex06/
├── app1/           # 첫 번째 웹 서버
│   ├── Dockerfile
│   └── index.html
├── app2/           # 두 번째 웹 서버
│   ├── Dockerfile
│   └── index.html
├── lb/             # 로드밸런서 (NGINX)
│   ├── Dockerfile
│   └── nginx.conf  # 라우팅 설정
└── docker-compose.yml # 전체 컨테이너 통합 실행

```

EX06의 각 Dockerfile은 EX01과 동일한 구조입니다.

[참고] [app1/Dockerfile](#) (app2도 동일)

ex06/app1/Dockerfile


```
FROM nginx                # NGINX 이미지 사용
COPY index.html /usr/share/nginx/html    # index.html을 NGINX 기본 경로로 복사
ENTRYPOINT ["nginx", "-g", "daemon off;"] # NGINX를 포그라운드로 실행
```

[참고] [lb/Dockerfile](#)

ex06/lb/Dockerfile

```
FROM nginx                # NGINX 이미지 사용
COPY nginx.conf /etc/nginx/conf.d/default.conf # 로컬의 nginx.conf를 컨테이너의 NGINX 설정 경로로 복사
ENTRYPOINT ["nginx", "-g", "daemon off;"]      # NGINX를 포그라운드로 실행
```

docker-compose.yml 파일은 다음과 같습니다. 하나의 **services** 옵션 안에 **app1**, **app2**, **lb** 세 개 서비스의 컨테이너 실행 설정이 작성되어 있습니다.

같은 **services** 옵션 안에 정의된 서비스들은 별도로 네트워크를 지정하지 않아도 자동으로 같은 네트워크에 연결됩니다.

[참고] [docker-compose.yml](#)

ex06/docker-compose.yml

```
services:
  app1:          # 서버 1
    build:
      context: ./app1 # Dockerfile 경로
    ports:
      - 8000:80      # localhost:8000으로 접근
  app2:          # 서버 2
    build:
      context: ./app2 # Dockerfile 경로
    ports:
      - 9000:80      # localhost:9000으로 접근
  lb:            # 로드밸런서
    image: lb        # 이미지 이름
    build:
      context: ./lb   # Dockerfile 경로
    ports:
      - 80:80        # localhost:80으로 접근
```

아래 명령어를 실행합니다. 빌드와 실행을 한 번에 수행합니다.

[실습] EX06 폴더로 이동 후, `docker compose up` 명령으로 모든 서비스를 한 번에 실행합니다.

docker compose up # 모든 컨테이너 한 번에 실행

```

$ docker compose up
[*] Running 4/4
✓ Network ex06_default Created
✓ Container ex06-app1-1 Created
✓ Container ex06-app2-1 Created
✓ Container ex06-lb-1 Created
Attaching to app1-1, app2-1, lb-1
  
```

그림 0-40: docker compose up 실행

Docker Desktop에서 컨테이너를 확인해보면 `ex06` 컨테이너 내부에 `app1`, `app2`, `lb` 컨테이너가 보입니다.

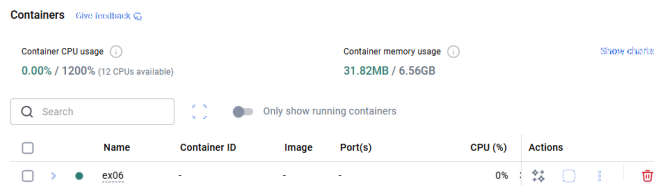


그림 0-41: Docker Desktop에서 Compose 컨테이너 확인



그림 0-42: Docker Desktop에서 Compose 로그 확인

EX01과 동일하게 브라우저에 `localhost:80/app1`, `localhost:80/app2` 를 입력하면 각 서버에 접근할 수 있습니다.

실습이 끝나면 다음 명령어로 컨테이너를 종료합니다.

[실습] 터미널에서 `docker compose down` 명령으로 모든 서비스를 종료합니다.

`docker compose down` # 모든 컨테이너 중지 및 삭제

2.5.3 docker compose : 주요 명령어

docker compose에서 자주 사용하는 하위 명령어를 정리하면 다음과 같습니다.

명령어	설명
<code>docker compose up</code>	모든 서비스를 빌드하고 실행
<code>docker compose up -d</code>	백그라운드에서 실행
<code>docker compose down</code>	모든 서비스를 중지하고 삭제
<code>docker compose ps</code>	실행 중인 서비스 목록 확인
<code>docker compose logs</code>	서비스 로그 확인
<code>docker compose build</code>	서비스 이미지만 빌드

2.6 종합 실습 : 웹 사이트 만들기

Dockerfile, NGINX, Redis, MySQL, Docker Compose까지, 지금까지 배운 모든 기술을 하나로 모아볼 시간입니다. 이번에는 Docker Compose를 활용해서 프론트 서버에서 발생한 요청이 백엔드 서버로 전달되고 DB 서버에서 데이터를 조회해 응답하는 웹 사이트를 만들어보겠습니다.

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex07> 에서 확인할 수 있습니다.

[EX07 패키지 구조]

```
ex07/
├── backend/      # 백엔드 서버 (Spring Boot)
│   ├── Dockerfile
│   └── entrypoint.sh # Git clone + 빌드 + 실행 스크립트
├── db/          # MySQL 데이터베이스
│   ├── Dockerfile
│   └── init.sql   # 초기 테이블 및 데이터 생성 SQL
└── frontend/    # 프론트엔드 (NGINX)
```

```

|   ├── Dockerfile
|   ├── index.html    # 화면 페이지
|   ├── nginx.conf    # 정적 파일 제공 + API 프록시 설정
|   └── docker-compose.yml # 전체 컨테이너 통합 실행

```

2.6.1 아키텍처 개요

이번에 만들 웹 사이트는 3개의 서비스로 구성됩니다.

- **Frontend (NGINX):** 브라우저에 HTML 페이지를 제공하고, `/api/` 요청을 백엔드로 프록시합니다.
- **Backend (Spring Boot):** `/api/users` API를 제공하여 DB에서 사용자 목록을 조회합니다.
- **DB (MySQL):** 사용자 데이터를 영구 저장합니다.

프론트 서버와 백엔드 서버, DB 서버는 서로 다른 포트를 가지고 있으며, 전체 요청 흐름은 다음과 같습니다.



그림 0-44: 풀스택 애플리케이션 아키텍처

2.6.2 MySQL : DB 서버 만들기

DB 서버는 2.4와 동일한 구조입니다.

[참고] [db/Dockerfile](#)

ex07/db/Dockerfile

```

FROM mysql                # MySQL 이미지 사용
COPY init.sql /docker-entrypoint-initdb.d # 로컬의 init.sql을 MySQL 초기화 폴더로 복사
ENV MYSQL_USER=metacoding  # 생성할 사용자 이름
ENV MYSQL_PASSWORD=metacoding1234 # 사용자 비밀번호
ENV MYSQL_ROOT_PASSWORD=root1234  # root 비밀번호
ENV MYSQL_DATABASE=metadb  # 생성할 데이터베이스 이름
CMD ["--character-set-server=utf8mb4", "--collation-server=utf8mb4_unicode_ci"] # 한글 인코딩 설정

```

2.6.3 Spring Boot : 백엔드 서버 만들기

GitHub에 있는 Spring Boot 프로젝트를 컨테이너에서 실행하려면 소스를 내려받고 빌드하는 과정이 필요합니다. 이런 초기 작업이 `entrypoint.sh`에 정의되어 있습니다. 컴퓨터에 비유하면 `Dockerfile`은 운영체제와 프로그램을 설치하는 과정이고, `entrypoint.sh`는 컴퓨터를 켜 때마다 자동으로 실행되는 시작 프로그램입니다.

`entrypoint.sh`는 컨테이너가 시작될 때 자동으로 실행되는 셸 스크립트입니다. 소스 코드 다운로드, 빌드, 서버 실행처럼 컨테이너가 커지자마자 해야 할 작업들을 순서대로 적어둡니다. - `Dockerfile` → 이미지 설치 단계 (한 번만 실행) - `entrypoint.sh` → 컨테이너 실행 단계 (시작할 때마다 실행)

`entrypoint.sh`는 다음과 같습니다.

[참고] 컨테이너가 시작되면 GitHub에서 Spring Boot 소스를 내려받은 후, Gradle로 빌드한 뒤, JAR 파일을 실행합니다.

ex07/backend/entrypoint.sh

```
#!/bin/bash
git clone https://github.com/metacoding-10-linux-docker/backend-server # 백엔드 서버 Github에서 내려받기
cd backend-server # 내려받은 폴더로 이동
chmod +x gradlew # 실행 권한 부여
./gradlew build # 스프링 프로젝트 빌드
java -jar -Dspring.profiles.active=prod build/libs/*.jar # 빌드된 파일 실행
```

스프링 서버 내부에는 `/api/users` 주소로 요청이 들어오면 회원 정보를 응답하는 컨트롤러가 존재합니다.

[참고] `entrypoint.sh`에서 내려받는 백엔드 서버의 핵심 API 코드는 다음과 같습니다.

UserController.java

```
@GetMapping("/api/users") // /api/users 요청 처리
public ResponseEntity<?> findAll() {
    List<User> users = userRepository.findAll(); // DB에서 회원 목록 조회
    return Resp.ok(users); // JSON 응답 반환
}
```

Dockerfile은 Java와 Git을 설치하고, 컨테이너가 시작되면 `entrypoint.sh`를 실행하도록 설정합니다.

[참고] `Dockerfile`

ex07/backend/Dockerfile

```

FROM eclipse-temurin:21-jdk      # JDK 21 베이스 이미지
WORKDIR /var/current/app         # 작업 디렉토리 설정
COPY entrypoint.sh /entrypoint.sh # 로컬의 entrypoint.sh를 컨테이너 루트 경로로 복사
RUN apt-get update && apt-get install -y git # Git 설치
ENTRYPOINT ["/entrypoint.sh"]    # 컨테이너 시작 시 실행

```

2.6.4 NGINX : 프론트 서버 만들기

Dockerfile은 nginx 이미지에 nginx.conf와 index.html을 복사합니다.

파일	설명
<code>frontend/Dockerfile</code>	nginx 이미지 기반, nginx.conf와 index.html 복사
<code>frontend/index.html</code>	fetch로 <code>/api/users</code> 요청 후 사용자 정보를 화면에 표시

[참고] [frontend/Dockerfile](#)

ex07/frontend/Dockerfile

```

FROM nginx                      # NGINX 이미지 사용
COPY nginx.conf /etc/nginx/nginx.conf # NGINX 설정 파일 복사
COPY index.html /usr/share/nginx/html/ # HTML 파일을 NGINX 기본 경로로 복사
CMD ["nginx", "-g", "daemon off;"]   # NGINX를 포그라운드로 실행

```

nginx.conf는 다음과 같습니다. 슬래시(/) 요청은 index.html을 응답하고 `/api/` 요청은 백엔드 서비스로 프록시합니다. `server backend:8080` 에서 `backend` 는 Docker Compose에서 정의한 서비스 이름입니다.

[참고] [frontend/nginx.conf](#)

ex07/frontend/nginx.conf

```

events {}

http {
    # 백엔드 서버 주소 (Docker Compose 서비스명)
    upstream backend {
        server backend:8080;
    }
}

```

```

}

server {
    listen 80;
    server_name _;

    # 정적 파일 제공
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # API 요청은 백엔드로 프록시
    location /api/ {
        proxy_pass http://backend;
    }
}
}

```

2.6.5 Docker Compose : 통합 구성

docker-compose.yml은 다음과 같습니다. `backend` , `db` , `frontend` 3개의 컨테이너를 생성합니다. `depends_on` 으로 기동 순서(db -> backend -> frontend)를 제어하고 `environment` 로 Spring Boot의 DB 접속 정보를 주입합니다.

[참고] `docker-compose.yml`

ex07/docker-compose.yml

```

services:
  backend:          # 백엔드 서비스 (Spring Boot)
    container_name: "backend" # 컨테이너 이름
    build:
      context: ./backend      # Dockerfile 경로
    ports:
      - "8080:8080"          # 호스트 8080 → 컨테이너 8080
    depends_on:
      - db                   # db가 먼저 시작된 후 실행
    environment:           # 환경 변수로 DB 접속 정보 전달
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/metadb?useSSL=false&serverTimezone=UTC&useLegacyDatetimeCode=false&allowPublicKeyRetrieval=true

```

```

SPRING_DATASOURCE_DRIVER_CLASS_NAME: com.mysql.cj.jdbc.Driver
SPRING_DATASOURCE_USERNAME: root
SPRING_DATASOURCE_PASSWORD: root1234

```

```

db:                                # 데이터베이스 서비스 (MySQL)
  container_name: "db"             # 컨테이너 이름
  build:
    context: ./db                  # Dockerfile 경로
  ports:
    - 3306:3306                   # 호스트 3306 → 컨테이너 3306

frontend:                          # 프론트엔드 서비스 (Nginx)
  container_name: "frontend"       # 컨테이너 이름
  build:
    context: ./frontend            # Dockerfile 경로
  ports:
    - "80:80"                     # 호스트 80 → 컨테이너 80
  depends_on:
    - backend                      # backend가 먼저 시작된 후 실행

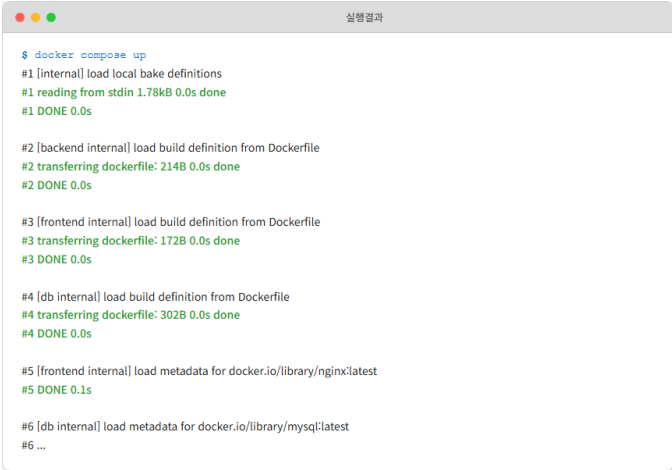
```

2.6.6 통합 실행

이제 명령어를 실행해 컨테이너를 실행합니다.

[실습] EX07 폴더로 이동 후, `docker compose up` 명령으로 전체 서비스를 실행합니다.

```
docker compose up # 풀스택 애플리케이션 실행
```

```

$ docker compose up
#1 [internal] load local bake definitions
#1 reading from stdin 1.78kB 0.0s done
#1 DONE 0.0s

#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 214B 0.0s done
#2 DONE 0.0s

#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 172B 0.0s done
#3 DONE 0.0s

#4 [db internal] load build definition from Dockerfile
#4 transferring dockerfile: 302B 0.0s done
#4 DONE 0.0s

#5 [frontend internal] load metadata for docker.io/library/nginx:latest
#5 DONE 0.1s

#6 [db internal] load metadata for docker.io/library/mysql:latest
#6 ...

```

그림 0-46: docker compose up 실행 결과

결과 확인

서버가 모두 실행되면 브라우저에서 결과를 확인합니다. 다음 항목을 순서대로 확인합니다.

1. 브라우저에서 `localhost` 또는 `localhost:80` 에 접속하여 “사용자 리스트” 페이지가 제대로 표시되는지 확인합니다.
2. 테이블에 ID와 이름 컬럼이 표시되고 `init.sql`에서 입력한 `ssar`, `cos` 데이터가 조회되는지 확인합니다.
3. 만약 데이터가 표시되지 않는다면 백엔드 서버의 빌드가 아직 진행 중일 수 있으므로 잠시 기다린 후 새로고침합니다.

서버와 통신이 성공하면 사용자 목록이 조회됩니다.

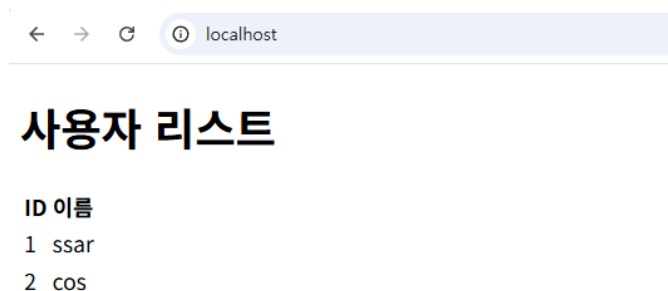


그림 0-47: 사용자 목록 조회 성공

실습이 끝나면 다음 명령어로 정리합니다.

[실습] 터미널에서 `docker compose down` 명령으로 전체 서비스를 종료하고 정리합니다.

`docker compose down` # 모든 컨테이너 중지 및 삭제

■ 이것만은 기억하자

- 레시피만 있으면 누구든 같은 요리를 만든다. `Dockerfile`에 환경 설정을 한 번 써두면 어디서든 동일한 컨테이너가 자동으로 만들어집니다. 더 이상 매번 패키지를 수동 설치할 필요가 없습니다.
- 안내데스크가 있으면 길을 잃지 않는다. `NGINX`는 URL 경로를 보고 적절한 서버로 요청을 보내 주고 트래픽이 몰려도 여러 서버에 골고루 나눠줍니다. 자주 찾는 파일은 캐시에 보관해 빠르게 응답합니다.
- 상자가 여러 개면, 악보 한 장으로 한꺼번에 실행하라. `Docker Compose`는 프론트엔드, 백엔드, DB 등 여러 컨테이너를 `docker-compose.yml` 하나에 정의하고 명령어 한 줄로 전부 실행합니다. 공용 사물함(Redis)으로 세션을 공유하고 영구 보관 창고(MySQL)로 데이터를 안전하게 저장할 수 있습니다.

`Docker Compose`로 풀스택 웹사이트까지 완성했습니다. 그런데 아직 해결하지 못한 문제가 남아 있습니다. 지금 실행 중인 백엔드 컨테이너를 `docker stop` 으로 종료해 보면, 사용자는 바로 오류 화면을 만나게 됩니다. 다시 서비스를 복구하려면 개발자가 직접 `docker compose up` 을 실행해야 합니다. 사용자가 몰리면 서버를 늘려야 하는데 컨테이너 수를 수동으로 조절하는 것이 전부입니다. 새벽에 컨테이너가 죽으면? 누군가 직접 다시 띄워야 합니다. 새 버전을 배포할 때마다 서비스가 잠시 멈추는 것도 피할 수 없습니다.

다음 챕터에서는 이 문제들을 해결합니다. 쿠버네티스로 원하는 수만큼 Pod을 자동으로 유지하고 컨테이너가 죽으면 자동으로 복구합니다. 롤링 업데이트로 서비스 중단 없이 새 버전을 배포합니다. 미니큐브 위에서 프론트엔드, 백엔드, DB, Redis가 연동되는 풀스택 웹사이트를 운영해 봅니다.

챕터 3. Kubernetes 시작하기

학습 목표

- 쿠버네티스의 구조와 동작 원리를 이해한다.
- Minikube로 로컬 환경에 쿠버네티스 클러스터를 구성한다.
- Deployment로 Pod을 관리하고, 자동 복구와 롤링 업데이트를 수행한다.
- Service로 Pod에 안정적으로 접근하는 네트워크를 구성한다.
- ConfigMap과 Secret으로 설정과 민감 정보를 외부에서 주입한다.
- Volume으로 영구 저장소를 구성하고, Ingress로 외부 요청을 라우팅한다.
- 미니큐브 위에서 풀스택 웹사이트를 배포한다.

3.1 쿠버네티스(Kubernetes)

3.1.1 쿠버네티스가 필요한 이유

Docker Compose로 여러 컨테이너를 실행할 수 있게 되었지만 운영 환경에서는 새로운 문제가 생깁니다. 예를 들어보겠습니다.

상황 1 – 새벽 3시, 컨테이너가 죽었다

백엔드 컨테이너가 메모리 부족으로 종료됐습니다. 사용자는 “서버 오류” 화면만 보게 됩니다. Docker Compose 환경이라면? 개발자가 알람을 확인하고 직접 `docker compose up` 을 다시 실행해야 합니다. 그 사이 서비스는 멈춰 있습니다.

상황 2 – 타임세일, 트래픽이 10배로 폭증

평소에는 컨테이너 1대로 충분했는데, 이벤트가 시작되자 응답 시간이 급격히 느려졌습니다. 수동으로 컨테이너 수를 늘리려면 서버를 준비하고 설정을 수정하고 다시 배포해야 합니다. 이벤트가 끝나면 또 줄여야 하는데, 이것을 매번 사람이 해야 합니다.

상황 3 – 새 버전 배포, 서비스가 잠시 멈춘다

결제 기능을 수정한 새 버전을 배포하는 상황입니다. 기존 컨테이너를 멈추고 새 컨테이너를 띄우는 그 짧은 순간, 결제 중이던 사용자는 오류를 만나게 됩니다.

컨테이너가 몇 개일 때는 수동 관리가 가능하지만, 수십~수백 개로 늘어나면 사실상 불가능합니다. 이것을 자동으로 해주는 게 바로 쿠버네티스입니다.

쿠버네티스(Kubernetes)는 구글에서 만든 대규모 컨테이너 관리 시스템입니다. 컨테이너들을 자동으로 배포, 확장, 복구, 관리해주는 운영 플랫폼입니다.

“어떻게 복구할지”를 일일이 지시하는 것이 아닌, “원하는 상태가 무엇인지”만 선언하면 나머지는 쿠버네티스가 알아서 맞춰 갑니다.

3.1.2 쿠버네티스의 핵심 리소스

쿠버네티스는 외부에서 요청이 들어오면 아래 그림처럼 쿠버네티스의 각 리소스를 거쳐 최종적으로 컨테이너(Pod)에 도달합니다. 지금은 전체 흐름만 가볍게 살펴보겠습니다.

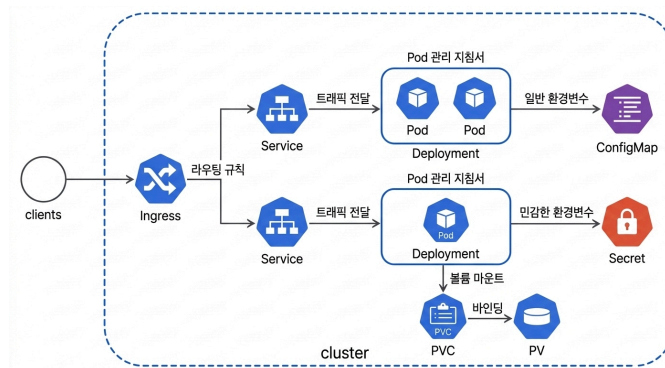


그림 0-1: 쿠버네티스 핵심 리소스 구조도

각 리소스의 역할은 다음과 같습니다. 상세한 내용은 실습을 통해 하나씩 다룹니다.

리소스	역할	이야기 속 비유
Ingress	외부 요청을 클러스터 내부로 라우팅하는 진입점	항구의 입구 게이트
Service	Pod의 IP가 바뀌어도 고정된 진입점을 제공하여 트래픽을 전달	대표 전화번호
Deployment	Pod의 생성, 개수 유지, 업데이트를 자동 관리하는 지침서	“이 앱을 3개 유지하라”는 지침서
Pod	컨테이너를 실행하는 가장 작은 단위	컨테이너를 담는 가장 작은 상자
ConfigMap	데이터베이스 주소 등 일반 설정값을 저장	환경 설정표
Secret	비밀번호, API 키 등 민감한 설정값을 암호화하여 저장	금고
PVC / PV	컨테이너가 삭제되어도 데이터를 유지하는 영구 저장소	창고 신청서 / 창고

3.1.3 쿠버네티스의 동작 원리

쿠버네티스는 크게 **컨트롤 플레인(Control Plane)** 과 **워커 노드(Worker Node)** 로 구성되어 있습니다. 이 둘을 하나의 시스템처럼 묶은 구조를 **클러스터(Cluster)** 라고 합니다.

회사에 비유하면 다음과 같습니다.

- **컨트롤 플레인** – 본사 관리팀. “서버 3대를 유지해라”, “이 서버가 죽으면 새로 띄워라” 같은 판단과 지시를 내립니다.
- **워커 노드** – 현장 작업자. 관리팀의 지시를 받아 실제로 컨테이너를 실행하고 관리합니다.



그림 0-3: 쿠버네티스 클러스터 구조

개발자가 명령어를 입력하면 어떤 일이 벌어지는지 살펴봅시다. 다음과 같은 순서로 동작합니다.

Step 1. 명령이 컨트롤 플레인의 **Kube API Server** 에 도달합니다.



그림 0-4: 개발자의 명령이 Kube API Server로 전달되는 흐름

Step 2. **Kube API Server** 는 컨트롤 플레인 내부의 구성 요소와 상호 작용합니다.

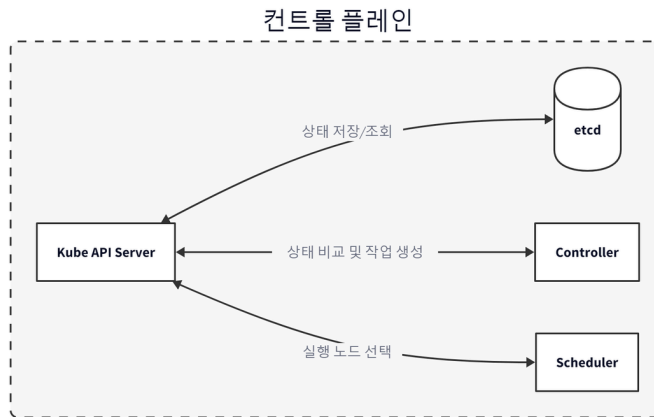


그림 0-5: 컨트롤 플레인 내부 구성 요소의 상호 작용

구성 요소	역할
etcd	상태 정보를 저장하는 저장소
Controller	원하는 상태와 실제 상태를 비교하여 필요한 작업을 자동 생성
Scheduler	명령이 실행될 노드를 자동 선택

Step 3. 실행할 작업과 노드가 정해지면 워커 노드의 **kubelet** 으로 전달됩니다.

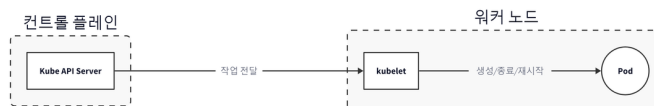


그림 0-7: kubelet의 컨테이너 관리

kubelet 은 컨트롤 플레인으로부터 명령을 받아 실제로 컨테이너를 관리하는 관리자입니다.

쿠버네티스가 무엇이고 어떻게 동작하는지 알아보았습니다. 이제 직접 실행해 볼 차례입니다. 다만 실제 쿠버네티스 환경을 로컬 PC에서 구현하기는 어렵기 때문에 로컬 PC 한 대에서도 쿠버네티스를 체험할 수 있는 미니큐브(**Minikube**)를 먼저 설치해보겠습니다.

3.2 미니큐브(Minikube)

3.2부터 3.7까지 작성하는 **YAML(yml)** 파일은 <https://github.com/metacoding-10-linux-docker/docker/tree/master/yaml>에서 확인할 수 있습니다.

3.2.1 미니큐브(Minikube)란?

미니큐브(**Minikube**)는 Mini + Kubernetes라는 의미로, 로컬 PC에서 쿠버네티스 환경을 손쉽게 구성할 수 있도록 해주는 개발용 프로그램입니다. Docker 컨테이너, VirtualBox 가상 머신 등을 사용해 미니큐브 환경을 구성할 수 있습니다.

미니큐브와 쿠버네티스의 기본 구조는 동일하지만, 미니큐브는 개발 환경에서 사용하는 용도이기 때문에 하나의 노드에 컨트롤 플레인과 워커 노드의 기능이 모여 있습니다.

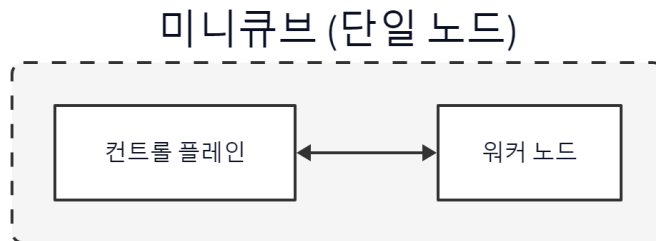


그림 0-8: 미니큐브의 단일 노드 구조

미니큐브는 단일 노드로 구성되기 때문에 구조가 단순하고 필요한 리소스도 적어 로컬 PC에서 손쉽게 활용할 수 있습니다. 다만 로드 밸런싱이나 오토 스케일링 같은 기능은 쓸 수 없습니다.

그럼에도 미니큐브를 쓰는 이유가 있습니다. 미니큐브에서 애플리케이션이 정상적으로 동작한다면, 동일한 설정과 구조를 그대로 실제 쿠버네티스 환경에도 적용할 수 있기 때문입니다.

3.2.2 미니큐브 기본 명령어

미니큐브 설치

[실습] OS에 맞는 패키지 관리자를 사용하여 미니큐브를 설치합니다.

Windows (터미널을 관리자 권한으로 실행)

choco install minikube

Mac (터미널에서 실행)

brew install minikube

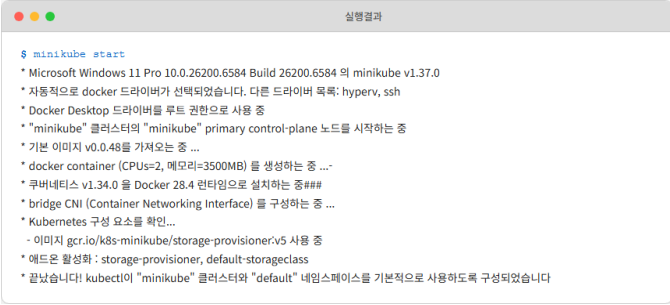
Windows는 **Chocolatey**, Mac은 **Homebrew** 패키지 관리자가 설치되어 있어야 동작하는 점 참고합니다.

미니큐브 실행

[실습] 미니큐브를 실행합니다.

minikube start

미니큐브 클러스터 시작



```
$ minikube start
* Microsoft Windows 11 Pro 10.0.26200.6584 Build 26200.6584의 minikube v1.37.0
* 자동적으로 docker 드라이버가 선택되었습니다. 다른 드라이버 목록: hyperv, ssh
* Docker Desktop 드라이버를 루트 권한으로 사용 중
* "minikube" 클러스터의 "minikube" primary control-plane 노드를 시작하는 중
* 기본 이미지 v0.0.48를 가져오는 중 ...
* docker container (CPUs=2, 메모리=3500MB)를 생성하는 중 ...
* 쿠버네티스 v1.34.0을 Docker 28.4 런타임으로 설치하는 중###
* bridge CNI (Container Networking Interface)를 구성하는 중 ...
* Kubernetes 구성 요소를 확인...
- 이미지 gcr.io/k8s-minikube/storage-provisioner/v5 사용 중
* 애드온 활성화: storage-provisioner, default-storageclass
* 끝났습니다! kubectli "minikube" 클러스터와 "default" 네임스페이스를 기본적으로 사용하도록 구성되었습니다
```

그림 0-9: minikube start 실행

미니큐브 명령어 요약

명령어	설명
<code>minikube start</code>	미니큐브 실행
<code>minikube stop</code>	미니큐브 종료
<code>minikube ip</code>	미니큐브 IP 확인
<code>minikube version</code>	미니큐브 버전 확인
<code>minikube dashboard</code>	대시보드 실행
<code>minikube service <서비스명> --url</code>	서비스 접근 URL 생성
<code>minikube addons enable ingress</code>	Ingress Controller 활성화
<code>minikube tunnel</code>	클러스터 외부에서 내부로 접근할 수 있도록 터널 생성

3.2.3 kubectl 명령어 : Pod 생성 및 관리

kubectl 은 쿠버네티스 내부의 클러스터 리소스를 관리하는 명령어입니다.

쿠버네티스를 다루기 위해서는 **kubectl** 명령어 학습이 필요합니다.

먼저 쿠버네티스의 가장 핵심 리소스인 Pod를 이용해 명령어를 실습해 보겠습니다.

Pod

Docker에서는 컨테이너를 직접 실행했지만, 쿠버네티스에서는 컨테이너를 Pod라는 껍데기에 담아서 실행합니다. 쿠버네티스가 관리하는 대상은 컨테이너가 아니라 Pod입니다.

Pod 는 쿠버네티스에서 컨테이너를 실행하는 가장 작은 단위입니다. Pod는 하나 이상의 컨테이너로 구성되어 있습니다.

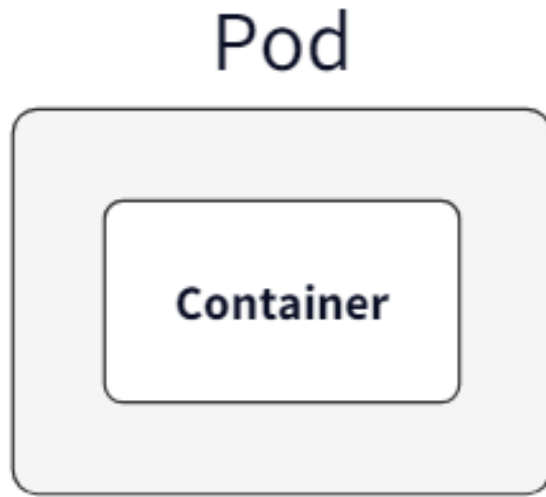


그림 0-11: Pod – 컨테이너를 감싸는 최소 실행 단위

Pod을 명령어로 생성하는 방법

`kubectrl run` 명령어를 사용하면 **Pod**를 손쉽게 생성할 수 있습니다.

`kubectrl run <pod명> --image=<이미지명>` 형태로 작성합니다.

[실습] nginx 이미지를 사용해 Pod를 생성하는 명령어입니다.

```
kubectrl run hello-pod1 --image=nginx # Dockerhub의 nginx 이미지로 Pod 생성
```

A terminal window titled 'kubectrl run' shows the command `$ kubectrl run hello-pod1 --image=nginx` being executed. The output is `pod/hello-pod1 created`.

그림 0-12: kubectrl run hello-pod1 실행 결과

Pod을 **yaml** 파일로 생성하는 방법

yaml 파일에 스크립트를 작성해서 **Pod**를 생성할 수도 있습니다. `hello-pod2.yaml`은 `nginx:1.20` 이미지를 사용하는 단일 **Pod**를 정의합니다.

[참고] Github 프로젝트의 [yaml/hello-pod2.yaml](#)을 참고합니다.

yaml/hello-pod2.yaml

```

apiVersion: v1          # API 버전
kind: Pod               # 리소스 종류
metadata:
  name: hello-pod2      # 리소스명
spec:                  # 상세 설정
  containers:          # 컨테이너 설정
  - name: hello-container # 컨테이너 이름
    image: nginx:1.20    # 사용할 이미지

```

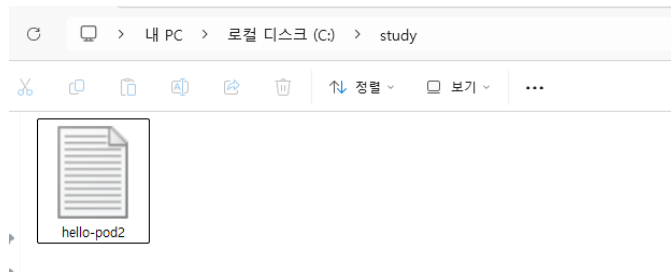


그림 0-13: hello-pod2.yaml 작성

[실습] 터미널 창을 **yaml** 폴더 로 이동 후 아래 명령어를 실행하면 **Pod** 가 생성됩니다.

```
kubectl apply -f hello-pod2.yaml # YAML 파일로 Pod 생성
```



그림 0-14: kubectl apply로 Pod 생성

Pod 조회

[실습] 생성된 **Pod** 목록을 조회합니다.

```
kubectl get pod # Pod 목록 조회
```

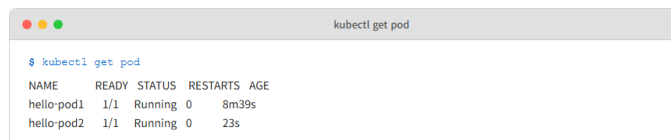


그림 0-15: Pod 목록 조회

kubectl describe pod <Pod이름> 명령어로 Pod의 상세 정보도 조회할 수 있습니다.

[실습] 생성된 Pod 목록을 조회합니다.

kubectl describe pod hello-pod2 # Pod 상세 정보 조회



```

$ kubectl describe pod hello-pod2
Name:          hello-pod2
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Sun, 15 Mar 2026 15:41:30 +0900
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.0.43
IPs:           IP: 10.244.0.43
Containers:
  hello-container:
    Container ID:  docker://65158d189989f41de190d076d953eb0da58c0f28cbfed088da7d91018bdc305
    Image:         nginx:1.20
    Image ID:      docker-pullable://nginx@sha256:38f8c1d9613f3f42e7969c3b1dd5c3277e635d4576713e6453c6193e66270a6d
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Sun, 15 Mar 2026 15:41:31 +0900
    Ready:         True
    Restart Count: 0
    Environment:   <none>
    Mounts:

```

그림 0-16: Pod 상세 조회

kubectl 명령어 요약

명령어	설명
<code>kubectl apply -f <파일></code>	YAML 파일로 리소스 생성/업데이트
<code>kubectl get <리소스></code>	리소스 목록 조회
<code>kubectl describe <리소스> <이름></code>	리소스 상세 정보 확인
<code>kubectl delete <리소스> <이름></code>	리소스 삭제
<code>kubectl exec -it <Pod명> -- bash</code>	Pod 내부 접속
<code>kubectl logs <Pod명></code>	Pod 로그 확인
<code>kubectl set image</code>	리소스 이미지 변경

3.3 Deployment, ReplicaSet

이전 챕터에서 명령어로 **Pod** 를 생성하는 방법을 배웠습니다. 그런데 **Pod** 를 직접 생성하면 한 가지 문제가 있습니다. **Pod** 가 오류로 종료되면 개발자가 직접 다시 실행해야 합니다.

이 문제를 해결해주는 게 **Deployment** 입니다. **Deployment** 는 “이 **Pod**를 3개 유지하고, 문제 생기면 자동 교체하라” 는 Pod 관리지침서와 같습니다.

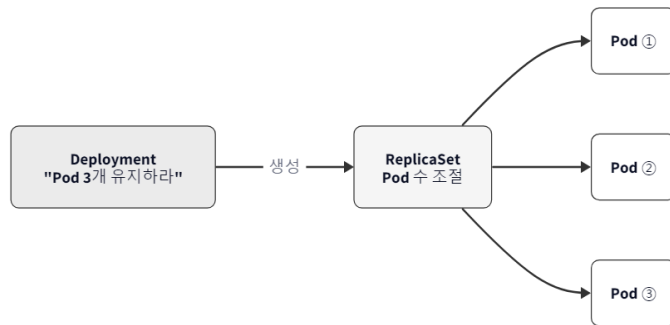


그림 0-18: Deployment가 ReplicaSet을 생성하고, ReplicaSet이 Pod 수를 유지한다

3.3.1 Deployment

Deployment 는 Pod를 자동으로 생성, 업데이트, 복구하는 관리 리소스입니다. Pod의 개수, 버전, 장애 여부를 지정된 상태에 맞게 자동으로 관리합니다.

이번에는 **Deployment** 로 Pod를 생성해 보겠습니다. nginx 이미지를 사용하는 `deploy-ex01.yml`을 작성합니다.

[참고] Github 프로젝트의 [yaml/deploy-ex01.yml](#) 을 참고합니다.

yaml/deploy-ex01.yml

```

apiVersion: apps/v1      # API 버전
kind: Deployment         # 리소스 종류
metadata:
  name: nginx-deploy     # 리소스 이름
spec:
  # pod에 대한 상태 지정
  replicas: 1            # 생성할 pod 수 지정
  selector:
    # 관리할 Pod 선택 조건
    matchLabels:
      # 라벨이 일치하는 Pod 선택
      app: nginx         # 라벨이 app : nginx인 pod를 관리
  
```

```

template:           # Pod 템플릿
metadata:
  labels:           # 라벨 지정
    app: nginx       # pod에 붙일 라벨
spec:              # 컨테이너 상세 설정
  containers:      # 컨테이너 설정
    - name: nginx-container # 컨테이너 이름
      image: nginx:1.20 # 사용할 이미지

```

Deployment의 **Selector** 는 특정 label을 가진 리소스를 선택하기 위한 조건입니다. Deployment나 Service와 같은 리소스의 Selector와 Pod의 labels가 일치하는 Pod를 찾아 관리합니다.

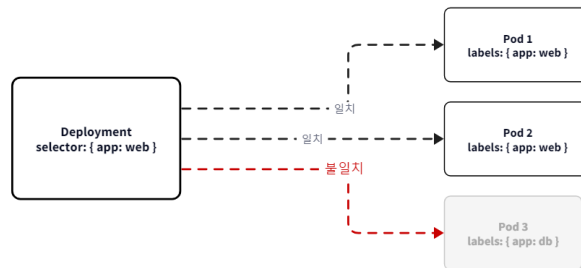


그림 0-19: Selector가 app: web인 Pod만 매칭하고, app: db인 Pod는 매칭하지 않음

아래 명령어로 **Deployment** 를 통해 Pod를 생성합니다.

[실습] **Deployment** 를 생성하고 **Pod** 를 확인합니다.

```

kubectl apply -f deploy-ex01.yml  # Deployment 생성
kubectl get pod                  # Pod 목록 조회

```

```

$ kubectl get pod
NAME                READY  STATUS   RESTARTS  AGE
hello-pod1          1/1    Running  0          27m
hello-pod2          1/1    Running  0          19m
nginx-deploy-756b46b54c-8q8p1  1/1    Running  0          5s

```

그림 0-20: Deployment와 Pod 생성 확인

아래 명령어를 통해 현재 생성된 Pod를 모두 제거합니다. 여기에는 3.2 챕터에서 생성한 hello-pod1, hello-pod2도 포함됩니다.

[실습] 전체 Pod 를 삭제한 뒤 다시 조회합니다.

```
kubectl delete pod --all # 전체 Pod 삭제
kubectl get pod # Pod 목록 조회
```

```
kubectl delete pod / get pod

$ kubectl delete pod --all
pod "hello-pod1" deleted from default namespace
pod "hello-pod2" deleted from default namespace
pod "nginx-deploy-756b46b54c-8q8p1" deleted from default namespace

$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deploy-756b46b54c-2sv8x       1/1     Running   0           3m11s
```

그림 0-21: Pod 제거 후 자동 재생성 확인

`kubectl run` 으로 직접 생성한 Pod 는 종료되었지만 `Deployment` 를 통해 생성된 Pod 는 자동으로 재시작되었습니다.

`Deployment` 로 Pod 를 생성하면 Pod 생성뿐만 아니라, Pod 의 개수 유지와 장애 복구까지 자동으로 이루어집니다. 그래서 Pod 는 직접 생성하기보다 `Deployment` 를 통해 생성하는 것이 일반적입니다.

[실습] 다음 실습을 위해 생성한 `Deployment` 를 제거합니다.

```
kubectl delete deployment nginx-deploy # Deployment 삭제
```

3.3.2 ReplicaSet

이번에는 Pod 의 개수를 원하는 만큼 유지하고, 원하는 상태를 관리하는 `ReplicaSet` 에 대해 알아보겠습니다.

에어컨의 자동 온도 조절을 떠올려 봅시다. 24도로 설정했을 때 현재 온도가 24도보다 올라가면 냉방을 틀고 내려가면 멈추면서 알아서 온도를 맞춰줍니다. `ReplicaSet` 은 이처럼 Pod 의 개수를 설정해두면, 현재 상태가 설정과 다를 때 자동으로 맞춰주는 역할을 합니다.

`ReplicaSet` 은 지정한 개수만큼 Pod이 항상 살아있도록 관리하는 컨트롤러입니다. Pod이 죽거나 예상보다 많아지는 상황이 발생하면 원래 설정된 수에 맞게 Pod을 자동으로 생성하거나 제거하여 상태를 일정하게 유지합니다.

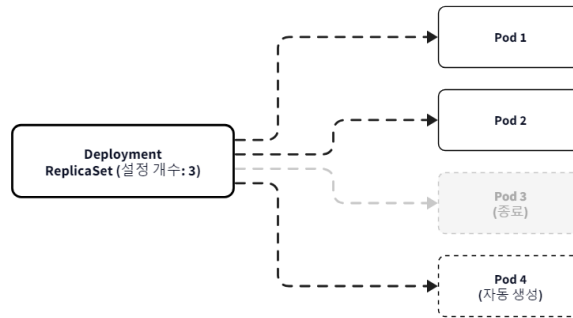


그림 0-22: Pod 3이 종료되면 ReplicaSet이 설정 개수를 맞추기 위해 Pod 4를 자동 생성

ReplicaSet 은 Deployment 에 설정합니다. 이번에는 Pod를 4개를 생성하는 Deployment를 작성합니다.

[참고] Github 프로젝트의 [yaml/deploy-ex02.yml](#) 을 참고합니다.

yaml/deploy-ex02.yml

```

apiVersion: apps/v1 # API 버전
kind: Deployment # 리소스 종류
metadata:
  name: nginx-replica # 리소스 이름
spec:
  replicas: 4 # pod 수 지정

  strategy:
    # Pod 교체 방식 설정
    type: RollingUpdate # 롤링 업데이트 전략
  rollingUpdate:
    maxSurge: 4 # 업데이트 중 최대 4개까지 추가 생성
    maxUnavailable: 4 # 업데이트 중 최대 4개까지 종료 허용

  selector:
    # 라벨이 app: nginx 인 pod를 관리
    matchLabels:
      # 라벨이 일치하는 Pod 선택
      app: nginx # app이 nginx인 Pod 선택
  template:
    # Pod 템플릿
    metadata:
      labels:
        # 라벨 지정

```



```

app: nginx    # pod에 붙일 라벨
spec:         # 컨테이너 상세 설정
containers:   # 컨테이너 설정
- name: nginx-container # 컨테이너 이름
  image: nginx:1.20     # 사용할 이미지

```

Deployment를 생성합니다.

[실습] Deployment 를 생성하고 Pod 개수를 확인합니다.

```

kubectl apply -f deploy-ex02.yml # Deployment 생성
kubectl get pod                 # Pod 목록 조회

```

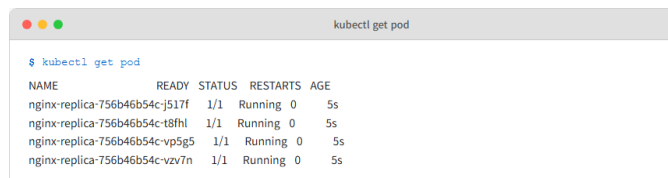


그림 0-23: replicas 설정으로 Pod 4개 생성

Deployment 에 설정한 replicas 의 수에 따라 Pod 가 4개 생성된 걸 확인할 수 있습니다.

3.3.3 롤링 업데이트(RollingUpdate)

deploy-ex02.yml을 보면 strategy 라는 설정이 type: RollingUpdate 로 되어 있습니다. RollingUpdate 옵션을 통해 새 버전을 배포할 때 Pod 을 한꺼번에 교체하지 않고 순차적으로 교체할 수 있습니다.

롤링 업데이트(RollingUpdate) 는 기존 Pod을 한꺼번에 내리지 않고, 하나씩 새 버전으로 바뀌어가는 무중단 배포 방식입니다. maxSurge 는 업데이트 중 추가로 띄울 수 있는 Pod 수, maxUnavailable 은 동시에 내릴 수 있는 Pod 수를 뜻합니다.

[실습] nginx 이미지를 1.21 버전으로 업데이트합니다.

```

kubectl set image deployment/nginx-replica nginx-container=nginx:1.21 # nginx 이미지를 1.21로 업데이트

```



그림 0-24: 이미지 버전 업데이트 실행

[실습] 업데이트 진행 상황을 실시간으로 확인합니다.

kubectl get pod -w

Pod 상태 실시간 감시

```

$ kubectl get pod -w
NAME                READY   STATUS    RESTARTS   AGE
nginx-replica-576bff5654-7gtrn  0/1    Pending   0          1s
nginx-replica-576bff5654-8bf7j  0/1    Pending   0          1s
nginx-replica-576bff5654-hkqwp  0/1    Pending   0          1s
nginx-replica-576bff5654-kcq4  0/1    Pending   0          1s
nginx-replica-576b46b54c-j5i7f  1/1    Terminating 0          9m20s
nginx-replica-576b46b54c-t8fhl  1/1    Terminating 0          9m20s
nginx-replica-576b46b54c-vp5g5  1/1    Terminating 0          9m20s
nginx-replica-576b46b54c-vzv7n  1/1    Terminating 0          9m20s
nginx-replica-576bff5654-hkqwp  0/1    ContainerCreating 0          1s
nginx-replica-576bff5654-7gtrn  0/1    ContainerCreating 0          1s
nginx-replica-576bff5654-8bf7j  0/1    ContainerCreating 0          1s
nginx-replica-576bff5654-kcq4  0/1    ContainerCreating 0          2s
nginx-replica-576b46b54c-vp5g5  0/1    Completed    0          9m23s
nginx-replica-576b46b54c-vzv7n  0/1    Completed    0          9m23s
nginx-replica-576b46b54c-t8fhl  0/1    Completed    0          9m23s
nginx-replica-576b46b54c-vp5g5  0/1    Completed    0          9m24s
nginx-replica-576b46b54c-j5i7f  0/1    Completed    0          9m24s
nginx-replica-576bff5654-8bf7j  1/1    Running       0          6s
nginx-replica-576bff5654-kcq4  1/1    Running       0          6s

```

그림 0-25: 롤링 업데이트 진행 상황

기존 Pod가 **Terminating** 상태로 종료되면서, 새로운 Pod가 **ContainerCreating** → **Running** 상태로 전환됩니다.

deploy-ex02.yml에 설정한 maxSurge와 maxUnavailable 값에 따라, 업데이트 과정에서 새로운 Pod 4개가 한 번에 생성되고 기존 Pod 또한 동시에 종료되었습니다. 이 설정을 조절하면 원하는 방식으로 업데이트를 진행할 수 있습니다.

[실습] **Deployment** 의 상세 정보를 확인하면 1.21 버전으로 변경된 것을 확인할 수 있습니다.

kubectl describe deployment nginx-replica # Deployment 상세 정보 조회

3.3.4 Rollback

[실습] **Deployment** 를 이전 상태로 롤백합니다.

kubectl rollout undo deployment/nginx-replica # 이전 버전으로 롤백

```

$ kubectl rollout undo deployment/nginx-replica
deployment.apps/nginx-replica rolled back

```

그림 0-26: Rollback 실행 결과

kubectl describe 명령어로 원복된 이미지 버전을 확인할 수 있습니다.

앞에서 만든 **Deployment** 는 다음 챕터에서도 계속 사용하니 삭제하지 않고 유지합니다.

3.4 Service

3.4.1 Service란?

이전에 **Deployment** 를 통해 **Pod** 를 생성하면 자동으로 관리된다는 것을 배웠습니다. 하지만 **Pod** 가 재시작될 때마다 IP가 바뀌기 때문에 외부에서 **Pod** 에 직접 접근하기가 어렵습니다. 이 문제를 해결해주는 게 **Service** 입니다.

콜센터의 대표 번호를 떠올려 봅시다. 고객은 상담사 개인 번호를 몰라도 대표 번호만 알면 상담사에게 연결할 수 있습니다. **Service** 는 이 대표 번호처럼, **Pod** 들 앞에 고정된 하나의 접근 주소를 만들어줍니다.

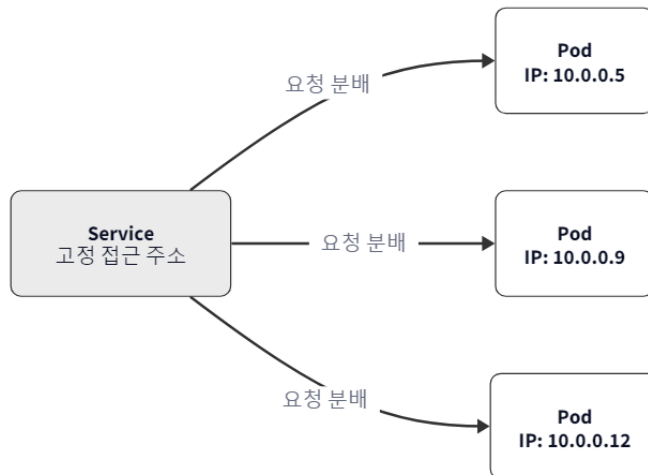


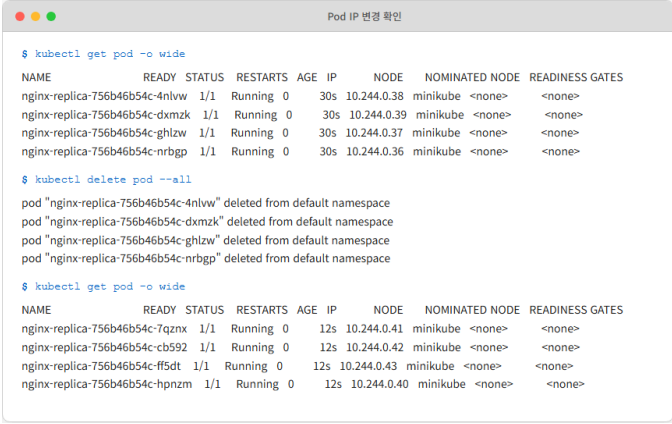
그림 0-27: Service는 고정 주소를 제공한다. Pod IP가 바뀌어도 Service 주소는 그대로다

Service 는 외부에서 Pod에 접근할 때, 고정 IP를 만들어 안정적으로 접근할 수 있게 만들어주는 리소스입니다.

Pod IP 변경

[실습] **Pod** 의 IP는 재시작 시 변경됩니다. IP를 확인한 후 **Pod** 를 삭제하고 다시 조회하면 IP가 달라진 것을 확인할 수 있습니다.

```
kubectl get pod -o wide      # IP 확인 (예: 10.244.0.7)
kubectl delete pod --all     # Pod 삭제
kubectl get pod -o wide      # IP 변경됨 (예: 10.244.0.8)
```



```
Pod IP 변경 확인

$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
nginx-replica-756b46b54c-4nlvw      1/1     Running   0          30s   10.244.0.38     minikube         <none>           <none>
nginx-replica-756b46b54c-dxmzk      1/1     Running   0          30s   10.244.0.39     minikube         <none>           <none>
nginx-replica-756b46b54c-ghlzw      1/1     Running   0          30s   10.244.0.37     minikube         <none>           <none>
nginx-replica-756b46b54c-nrbgp      1/1     Running   0          30s   10.244.0.36     minikube         <none>           <none>

$ kubectl delete pod --all
pod "nginx-replica-756b46b54c-4nlvw" deleted from default namespace
pod "nginx-replica-756b46b54c-dxmzk" deleted from default namespace
pod "nginx-replica-756b46b54c-ghlzw" deleted from default namespace
pod "nginx-replica-756b46b54c-nrbgp" deleted from default namespace

$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
nginx-replica-756b46b54c-7qzrx      1/1     Running   0          12s   10.244.0.41     minikube         <none>           <none>
nginx-replica-756b46b54c-cb592      1/1     Running   0          12s   10.244.0.42     minikube         <none>           <none>
nginx-replica-756b46b54c-ff5dt      1/1     Running   0          12s   10.244.0.43     minikube         <none>           <none>
nginx-replica-756b46b54c-hpnzm      1/1     Running   0          12s   10.244.0.40     minikube         <none>           <none>
```

그림 0-28: Pod 재시작 시 IP 변경 확인

이 문제를 해결해주는 게 **Service** 입니다.

Service 생성

Service 는 **Deployment** 와 마찬가지로 selector의 라벨이 일치하는 **Pod** 를 관리합니다. service-ex01.yml은 **NodePort** 타입의 **Service** 를 정의하며, 외부에서 30080 포트로 접속할 수 있도록 설정합니다.

[참고] Github 프로젝트의 [yaml/service-ex01.yml](#) 을 참고합니다.

yaml/service-ex01.yml

```
apiVersion: v1      # API 버전
kind: Service       # 리소스 종류
metadata:
  name: nginx-service # 서비스 이름
spec:
  type: NodePort     # 상세 설정
  selector:          # 연결할 Pod 선택 조건
    app: nginx       # app이 nginx인 Pod에 연결
  ports:             # 포트 설정
    - protocol: TCP   # 프로토콜
      port: 80        # 서비스가 클러스터 내부에서 열어둔 포트
```

```
targetPort: 80    # pod가 열어둔 포트
nodePort: 30080   # 외부에서 접속할 포트 지정
```

[실습] `Service` 를 생성합니다.

```
kubectl apply -f service-ex01.yml # Service 생성
```

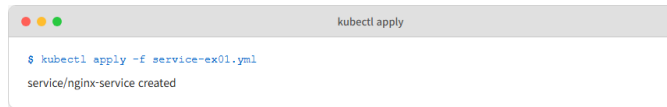


그림 0-29: Service 생성

Service 타입 비교

Service 타입에 따라 외부에서 접근할 수 있는지 여부가 달라집니다. 백엔드나 DB처럼 외부에 노출할 필요가 없는 서비스는 `ClusterIP` 로, 사용자가 직접 접속해야 하는 서비스는 `NodePort` 나 `LoadBalancer` 로 설정합니다.

서비스의 타입 종류는 다음과 같습니다.

타입	설명	접근 범위
ClusterIP	클러스터 내부에서만 접근 가능한 기본 타입	내부 전용
NodePort	노드의 특정 포트(30000~32767)를 통해 외부에서 접근 가능	외부 접근 가능
LoadBalancer	클라우드 환경에서 외부 로드밸런서를 자동 생성	외부 접근 가능 (클라우드)

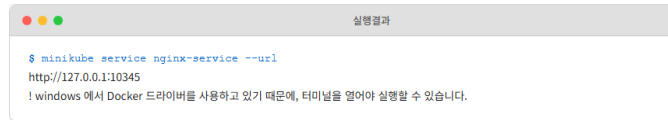
3.4.2 Networking

`Service` 를 생성했지만, 로컬 PC와 미니큐브는 서로 다른 네트워크에 있기 때문에 Service의 IP로 직접 접근할 수 없습니다. 로컬 PC에서 Service에 접근하려면 두 네트워크를 연결해주는 다리가 필요한데, `minikube service` 명령어가 그 역할을 합니다. 이 명령어를 실행하면 로컬 PC에서 접근할 수 있는 임시 경로가 생성됩니다.

minikube service 명령어는 특정 Service에 대해 임시 접근 경로를 생성하여, 로컬 PC에서 해당 Service에 바로 접속할 수 있도록 해줍니다.

[실습] 로컬 PC에서 **Service** 에 접속할 수 있는 URL을 생성합니다.

```
minikube service nginx-service --url # Service 접근 URL 생성
```



```
$ minikube service nginx-service --url
http://127.0.0.1:10345
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 0-31: minikube service URL 생성

생성된 URL로 접속하면, 미니쿠브 내부의 **Service** 를 통해 해당 **Pod** 서버로 요청이 전달됩니다.



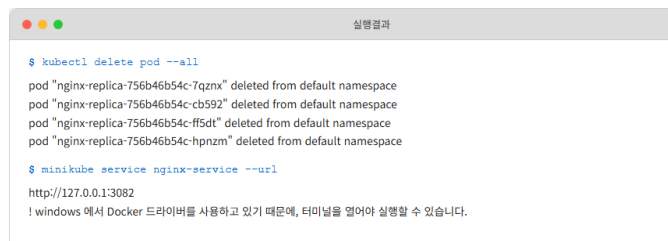
그림 0-32: 브라우저에서 nginx 접속 확인

확인 후 CTRL + C를 입력해 터미널을 빠져나옵니다.

이번에는 **Pod** 를 전부 삭제한 뒤 다시 접속해 보겠습니다.

[실습] **Pod** 를 삭제하고, **minikube service** 명령어를 통해 다시 접근합니다.

```
kubectl delete pod --all # 전체 Pod 삭제
minikube service nginx-service --url # Service 접근 URL 생성
```



```
$ kubectl delete pod --all
pod "nginx-replica-756b46b54c-7qznx" deleted from default namespace
pod "nginx-replica-756b46b54c-cb592" deleted from default namespace
pod "nginx-replica-756b46b54c-ff5dt" deleted from default namespace
pod "nginx-replica-756b46b54c-hpnzm" deleted from default namespace

$ minikube service nginx-service --url
http://127.0.0.1:3082
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 0-33: Pod 삭제 후 Service 접속

새롭게 생성된 URL로 접속하면, nginx 페이지가 정상적으로 표시됩니다. **Service** 가 고정된 접근 경로를 유지해주기 때문에 Pod가 재실행되어도 접근이 가능합니다.

[실습] 실습이 끝난 후 **Deployment** 와 **Service** 를 삭제합니다.

```
kubect! delete deployment nginx-replica # Deployment 삭제
```

```
kubect! delete service nginx-service # Service 삭제
```

3.4.3 Ingress

`minikube service` 명령어는 테스트용 임시 경로입니다. 실제 운영 환경에서는 <http://my-service.com> 같은 도메인으로 접속하는데, 이때 외부 요청을 클러스터 내부의 `Service` 로 연결해주는 역할을 하는 것이 `Ingress` 입니다.

건물에 비유하면, `Service` 가 각 부서의 내선 번호라면 `Ingress` 는 건물 1층의 안내 데스크입니다. 방문자(외부 요청)가 건물에 들어오면, 안내 데스크가 “어느 부서를 찾으세요?”라고 확인한 뒤 적절한 내선 번호(`Service`)로 연결해줍니다.



그림 0-34: 외부 요청은 Ingress를 통해 Service로 전달된다

Ingress 는 클러스터 외부의 HTTP/HTTPS 요청을 내부 `Service`로 라우팅하는 규칙을 정의하는 리소스입니다.

`Ingress` 가 동작하려면 두 가지가 필요합니다.

구성 요소	역할	비유
Ingress Controller	실제로 외부 요청을 받아 처리하는 소프트웨어	안내 데스크에 앉아있는 직원
Ingress 리소스	어떤 요청을 어떤 <code>Service</code> 로 보낼지 정의한 규칙	안내 데스크에 놓인 부서 안내판

`Ingress Controller` 는 보통 `Nginx`를 많이 사용하며, 미니큐브에서는 `minikube addons enable ingress` 명령어 한 줄로 사용할 수 있습니다.

`Ingress` 의 실습은 3.7절에서 실습을 진행합니다.

3.5 ConfigMap, Secret

`Service` 덕분에 `Pod`의 IP가 바뀌어도 안정적으로 접근할 수 있게 되었습니다. 그런데 실제 서비스를 운영하려면 데이터베이스 주소, API 키, 비밀번호 같은 설정 값이 필요합니다. 이런 값을 코드에 직

점 넣으면 변경할 때마다 이미지를 다시 빌드해야 합니다. 다음에는 이 문제를 해결하는 ConfigMap과 Secret을 알아보겠습니다.

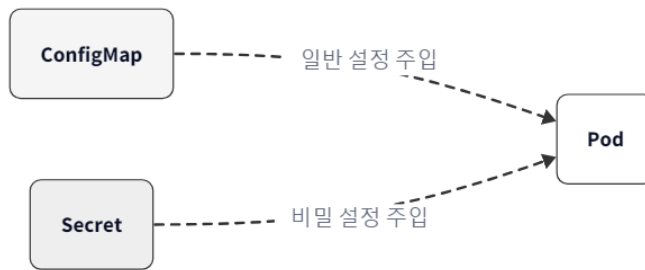


그림 0-36: ConfigMap과 Secret은 Pod에 설정과 민감 정보를 주입한다

3.5.1 ConfigMap

데이터베이스 주소나 접속 URL 같은 설정 값을 코드에 직접 넣으면, 값이 바뀔 때마다 이미지를 다시 빌드해야 합니다. **ConfigMap**은 이런 설정 값을 코드 바깥에서 따로 관리할 수 있게 해줍니다.

ConfigMap은 일반적인 설정 값을 외부에서 관리하는 리소스입니다. 환경 변수, 설정 파일 등 민감하지 않은 설정 정보를 저장하며, Pod는 이를 환경 변수로 전달받아 사용합니다.

[참고] Github 프로젝트의 [yaml/configmap-conn.yml](https://github.com/yamly/configmap-conn.yml)을 참고합니다.

yaml/configmap-conn.yml

```

apiVersion: v1          # API 버전
kind: ConfigMap          # 리소스 종류
metadata:
  name: configmap-conn    # ConfigMap 이름 지정
data:                    # 설정값 넣는 영역
  conn_info: "localhost:80" # 접속 정보
  conn_url: "config.test"  # 접속 URL
  
```

deploy-ex03.yml에서는 `envFrom.configMapRef`를 사용하여 **ConfigMap**에 정의된 모든 설정 값을 환경 변수 형태로 **Pod**에서 사용할 수 있습니다.

[참고] Github 프로젝트의 [yaml/deploy-ex03.yml](https://github.com/yamly/deploy-ex03.yml)을 참고합니다.

yaml/deploy-ex03.yml


```

apiVersion: apps/v1           # API 버전
kind: Deployment              # 리소스 종류
metadata:
  name: nginx-config-secret    # 리소스 이름
spec:                          # 상세 설정
  replicas: 1                 # pod 수 지정
  selector:                   # 관리할 Pod 선택 조건
    matchLabels:              # 라벨이 일치하는 Pod 선택
      app: nginx              # 라벨이 app : nginx인 pod를 관리
  template:                   # Pod 템플릿
    metadata:
      labels:                  # 라벨 지정
        app: nginx            # pod에 붙일 라벨
    spec:                      # 컨테이너 상세 설정
      containers:              # 컨테이너 설정
        - name: nginx-container # 컨테이너 이름
          image: nginx:1.20      # 사용할 이미지
          envFrom:                # 환경 변수 일괄 주입
            - configMapRef:      # ConfigMap 참조
              name: configmap-conn # ConfigMap 연결

```

[실습] ConfigMap 과 Deployment 를 생성합니다.

```

kubectl apply -f configmap-conn.yml # ConfigMap 생성
kubectl apply -f deploy-ex03.yml    # Deployment 생성

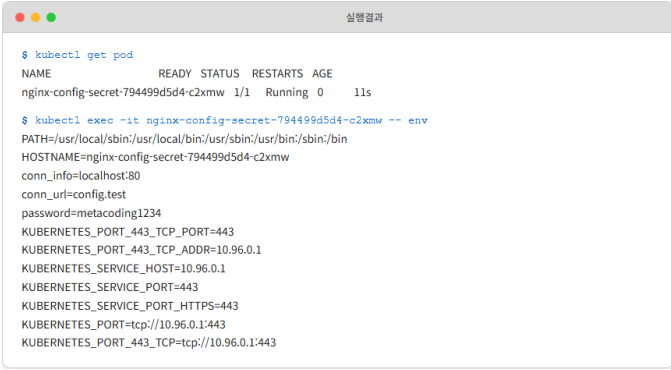
```

[실습] Pod 의 환경 변수를 조회하여 ConfigMap 설정이 적용되었는지 확인합니다.

```

kubectl get pod           # Pod 목록 조회
kubectl exec -it nginx-config-secret-794499d5d4-c2xmw -- env # Pod 환경 변수 조회

```



```

$ kubectl get pod
NAME                                READY  STATUS   RESTARTS  AGE
nginx-config-secret-79449d5d4-c2xmw  1/1    Running  0         11s

$ kubectl exec -it nginx-config-secret-79449d5d4-c2xmw -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=nginx-config-secret-79449d5d4-c2xmw
conn_info=localhost:80
conn_url=config.test
password=metacoding1234
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443

```

그림 0-37: Pod 환경 변수 조회

여러 개의 환경 변수 중 `ConfigMap` 에 설정한 `conn_info` 와 `conn_url` 을 확인할 수 있습니다.

3.5.2 Secret

`ConfigMap`이 일반적인 환경 설정이라면, `Secret`은 금고입니다. 비밀번호나 API 키처럼 민감한 정보는 설정표에 적어두면 안 됩니다. 금고에 따로 보관해야 합니다.

Secret 은 비밀번호, 토큰, 인증 키처럼 민감한 정보를 안전하게 저장하고 관리하기 위한 리소스입니다. `ConfigMap`과 구조는 비슷하지만 `Secret`의 값은 Base64로 인코딩되어 저장됩니다. 단, Base64는 암호화가 아닌 단순 인코딩이므로 보안을 보장하지는 않습니다. `Secret`은 평문이 설정 파일에 직접 노출되는 것을 방지하는 수준이며, 실제 운영 환경에서는 별도의 암호화 솔루션을 사용해야 합니다.

[참고] Github 프로젝트의 [yaml/secret-password.yml](#) 을 참고합니다.

yaml/secret-password.yml

```

apiVersion: v1          # API 버전
kind: Secret            # 리소스 종류
metadata:
  name: secret-password # Secret 이름
type: Opaque            # key-value 형태의 secret 타입
stringData:             # 평문을 자동으로 Base64 변환
  password: metacoding1234 # 비밀번호 설정

```

[실습] `Secret` 을 생성합니다. `stringData` 속성을 사용하면 평문을 자동으로 Base64로 변환해줍니다.

```
kubectl apply -f secret-password.yml # Secret 생성
```

[실습] `Secret` 의 내부를 YML 형태로 출력합니다.

```
kubectl get secret secret-password -o yaml # Secret 내용을 YAML 형태로 출력
```



```
$ kubectl get secret secret-password -o yaml
apiVersion: v1
data:
  password: bWV0YWNvZGluZzEyMzQ=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":"secret-password","namespace":"default"},"stringData":{"password":"meta...
  creationTimestamp: "2026-03-15T06:30:27Z"
name: secret-password
namespace: default
resourceVersion: "1387"
```

그림 0-38: Secret의 Base64 인코딩 확인

Secret 을 YAML 형태로 출력해보면, 환경 변수로 설정한 비밀번호가 Base64로 인코딩되어 있는 걸 확인할 수 있습니다.

3.5.3 환경 변수 추가

deploy-ex03.yaml에 **Secret** 을 추가해보겠습니다. **secretRef** 를 사용하면 **Secret** 에 정의된 값을 환경 변수로 **Pod** 에서 불러올 수 있습니다.

3.5.1에서 작성한 deploy-ex03.yaml의 **envFrom** 항목에 **secretRef** 를 추가합니다. 아래는 추가된 부분만 표시한 것입니다.

[참고] Github 프로젝트의 [yaml/deploy-ex03.yaml](#) 에서 **envFrom** 항목을 참고합니다.

yaml/deploy-ex03.yaml

```
# ... 생략

envFrom:
- configMapRef:
    name: configmap-conn    # ConfigMap 연결
- secretRef:
    name: secret-password   # Secret 연결 (추가)
```

[실습] 변경된 **Deployment** 를 적용합니다. yaml 파일 내부의 **template** 속성이 변경되면 **kubectl apply -f** 를 통해 적용할 수 있습니다.

```
kubectl apply -f deploy-ex03.yaml # 변경된 Deployment 적용
```

[실습] **Pod** 의 환경 변수를 조회하면 **Secret** 에 저장된 비밀번호가 주입되어 출력됩니다.

kubectl get pod

Pod 목록 조회

kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env # Pod 환경 변수 조회

```

$ kubectl get pod
NAME                                READY STATUS RESTARTS AGE
nginx-config-secret-7fbccb65f5-zq8nz 1/1   Running 0      51s

$ kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/bin
HOSTNAME=nginx-config-secret-7fbccb65f5-zq8nz
TERM=xterm
conn_url=config.test
password=metacoding1234
conn_info=localhost:80
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443

```

그림 0-39: Pod 환경 변수에 Secret 값 확인

여기서 주목할 점이 있습니다. **Secret** 에 Base64로 인코딩되어 저장된 비밀번호가, **Pod** 에서 사용할 때는 자동으로 평문 형태로 변경됩니다.

3.5.4 환경 변수 수정

ConfigMap 이나 **Secret** 에서 환경 변수 값을 변경한 후 **kubectl apply -f 파일명** 을 실행해 변경 내용을 **Kube API Server** 에 저장합니다. 그런데 저장된 환경 변수가 실행 중인 **Pod** 에 즉시 반영될까요? 아닙니다, 반영되지 않습니다.

Pod 나 **Deployment** 는 설정이 바뀌었다고 알아서 감지하지 않고, **Kube API Server** 도 변경된 설정을 기존 **Pod** 에 강제로 주입하지 않기 때문입니다.

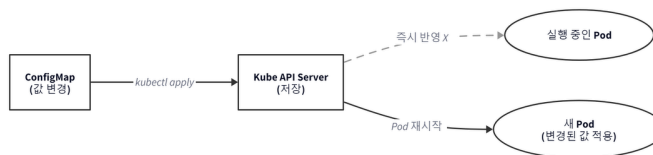


그림 0-40: ConfigMap 변경 후 Pod 재시작으로 반영되는 흐름

환경 변수 같은 실행 환경 설정은 프로세스가 시작될 때만 적용되고, 실행 중에는 변경할 수 없습니다. 따라서 변경된 설정을 반영하려면 **Pod**를 재시작해야 합니다.

실습을 위해 configmap-conn.yml의 **conn_info** 포트를 90으로 수정해보겠습니다.

[참고] Github 프로젝트의 [yaml/configmap-conn.yml](#) 에서 포트 수정 내용을 참고합니다.

yaml/configmap-conn.yml

... 생략

conn_info: "localhost:90" # 환경변수 수정

[실습] 변경된 **ConfigMap** 을 적용하고 **Pod** 를 재시작합니다.

kubectl apply -f configmap-conn.yml # 변경된 **ConfigMap** 적용

kubectl rollout restart deployment nginx-config-secret # 재시작

```
실행결과
$ kubectl apply -f configmap-conn.yml
configmap/configmap-conn configured
$ kubectl rollout restart deployment nginx-config-secret
deployment.apps/nginx-config-secret restarted
```

그림 0-41: ConfigMap 수정 후 Pod 재시작

[실습] 재시작된 **Pod** 의 환경 변수를 확인합니다.

kubectl exec -it nginx-config-secret-6655f67f4c-hsrrf -- env # Pod 환경 변수 조회

```
실행결과
$ kubectl get pod
NAME                                READY  STATUS   RESTARTS  AGE
nginx-config-secret-6655f67f4c-hsrrf  1/1    Running  0         8s

$ kubectl exec -it nginx-config-secret-6655f67f4c-hsrrf -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=nginx-config-secret-6655f67f4c-hsrrf
password=metacoding1234
conn_info=localhost:90
conn_url=config.test
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
```

그림 0-42: 변경된 환경 변수 확인

conn_info 변수의 포트가 90으로 변경되었습니다.

3.6 Volume

Pod 안에서 생성한 파일은 **Pod** 이 재시작되면 모두 사라집니다. 로그 파일이나 데이터베이스처럼 데이터를 보존해야 하는 경우에는 어떻게 해야 할까요? 이 문제를 해결해주는 것이 **Volume** 입니다.

볼륨(Volume)은 Pod 내부 컨테이너가 사용할 수 있는 외부 저장 공간을 의미합니다.

Volume에는 여러 종류가 있습니다.

종류	설명	데이터 유지
emptyDir	Pod 생성 시 만들어지는 임시 저장 공간. 같은 Pod 안의 컨테이너끼리 데이터를 공유할 때 사용	Pod 삭제 시 함께 삭제
hostPath	워커 노드(호스트)의 특정 경로를 Pod에 마운트	노드에 남아 있지만, Pod이 다른 노드로 이동하면 접근 불가
PV / PVC	클러스터 외부에 영구 저장소를 만들고, 요청서(PVC)를 통해 Pod에 연결	Pod이 삭제되어도 유지

이 중에서 실무에서 가장 많이 사용하는 **PV / PVC**를 실습해 보겠습니다.

3.6.1 Persistent storage

Persistent storage는 Pod이 종료되어도 데이터가 사라지지 않도록 외부에 저장하는 영구 저장소입니다. 실제 저장 공간인 **PV (PersistentVolume)**와 저장 공간을 요청하는 **PVC (PersistentVolumeClaim)**로 구성됩니다.



그림 0-44: PV는 실제 저장 공간, PVC는 요청서. PVC를 통해 Pod에 저장소를 연결한다

비유하면 **PV**는 창고 공간이고, **PVC**는 “10평짜리 창고가 필요합니다”라는 신청서입니다. Pod은 PVC를 통해 조건에 맞는 PV를 찾아 연결합니다. 실습은 PV 생성 → PVC 생성 → Pod에 연결 순서로 진행합니다.

PV(PersistentVolume)

PV(PersistentVolume)은 실제 데이터가 저장되는 저장소입니다.

이번 실습에서는 로컬 PC의 폴더를 저장소로 사용합니다.

[참고] Github 프로젝트의 [yaml/volume-pv.yml](https://github.com/yaml/volume-pv.yml)을 참고합니다.

yaml/volume-pv.yml

```

apiVersion: v1          # API 버전
kind: PersistentVolume  # 리소스 종류
metadata:
  name: volume-pv       # PV 이름
spec:
  # 상세 설정
  capacity:
    storage: 1Gi        # 1Gi 용량 할당
  accessModes:
    - ReadWriteOnce     # 읽기/쓰기 권한
  storageClassName: ""   # PVC가 볼륨을 자동으로 생성 못하게 방지
  persistentVolumeReclaimPolicy: Retain # PVC 삭제되어도 유지
  hostPath:
    path: /mnt/data     # 볼륨 경로 지정

```

YAML에서 `hostPath` 의 `/mnt/data` 는 미니큐브 내부 경로입니다. 로컬 PC의 폴더와 연결하려면 `minikube mount` 명령이 필요합니다.

먼저 데이터를 저장할 경로에 폴더를 생성합니다.

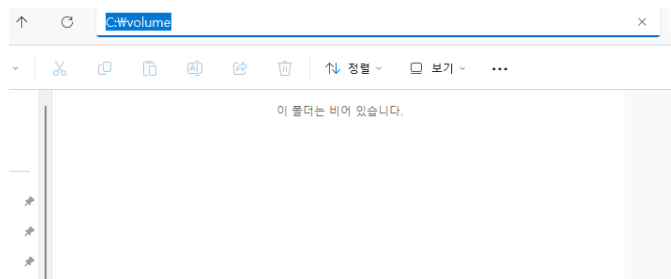


그림 0-45: 로컬 PC에 볼륨 폴더 생성

[실습] 로컬 PC의 `C:/volume` 경로와 미니큐브의 `/mnt/data` 경로를 마운트합니다.

```
minikube mount "C:/volume:/mnt/data" # 로컬 폴더를 미니큐브에 마운트
```



```

$ minikube mount "C:/volume:/mnt/data"
* Mounting host path C:/volume into VM as /mnt/data ...
- Mount type: 9p
- User ID:  docker
- Group ID:  docker
- 버전:      9p2000.L
- 메시지 사이즈: 262144
- 옵션:      map[]
- 연결된 주소: 127.0.0.1:9577
* Userspace file server:
ufs starting
* Successfully mounted C:/volume to /mnt/data

* NOTE: This process must stay alive for the mount to be accessible ...

```

그림 0-46: minikube mount 실행

`minikube mount` 명령어는 포그라운드로 실행됩니다. 터미널 창이 종료되면 로컬 PC와의 마운트도 함께 종료되기 때문에 새로운 터미널 창을 실행해야 합니다.

PVC(PersistentVolumeClaim)

저장 공간(PV)이 준비되었으니, 이제 저장 공간을 요청하는 신청서(PVC)를 만듭니다.

PVC(PersistentVolumeClaim) 는 Pod이 사용할 저장소의 조건(용량, 접근 모드 등)을 정의한 요청서입니다.

[참고] Github 프로젝트의 [yaml/volume-pvc.yml](#) 을 참고합니다.

yaml/volume-pvc.yml

```

apiVersion: v1          # API 버전
kind: PersistentVolumeClaim # 리소스 종류
metadata:
  name: volume-pvc      # pod가 참조하는 pvc명
spec:                   # 상세 설정
  accessModes:
    - ReadWriteOnce     # 읽기 쓰기 권한
  storageClassName: ""   # PVC가 자동으로 PV 생성하지 않도록 설정
  resources:
    requests:
      storage: 1Gi       # 용량
  volumeName: volume-pv # 참조할 PV명

```

PVC 가 PV 와 정상적으로 바인딩되려면 `accessModes` 와 `storageClassName` 설정이 서로 일치해야 하고 PVC 에서 요청한 용량이 PV 가 제공하는 용량보다 작거나 같아야 합니다.

PV와 PVC가 준비되었으니, 이제 Pod에 연결합니다.

[참고] Github 프로젝트의 [yaml/volume-pod.yml](#) 을 참고합니다.

yaml/volume-pod.yml

```

apiVersion: v1          # API 버전
kind: Pod               # 리소스 종류
metadata:
  name: volume-pod      # Pod 이름
spec:                  # 상세 설정
  containers:          # 컨테이너 설정
  - name: nginx-volume  # 컨테이너 이름
    image: nginx        # 사용할 이미지
    volumeMounts:       # 사용할 볼륨 마운트
    - name: storage     # 볼륨 이름
      mountPath: /mnt/data # 컨테이너에서 접근할 볼륨 위치
  volumes:             # 볼륨 정의
  - name: storage       # 볼륨 이름
    persistentVolumeClaim: # PVC 연결
    claimName: volume-pvc # 참조할 PVC

```

[실습] 새로운 터미널 창에서 **PV**, **PVC**, **Pod** 를 순서대로 생성합니다.

```

kubectl apply -f volume-pv.yml    # PV 생성
kubectl apply -f volume-pvc.yml   # PVC 생성
kubectl apply -f volume-pod.yml   # Pod 생성

```

[실습] **PV** 와 **PVC** 의 바인딩 상태를 확인합니다. **STATUS** 값이 **BOUND**이면 정상입니다.

```

kubectl get pv,pvc -o wide        # PV와 PVC 바인딩 상태 확인

```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATT
persistentvolume/volume-pv	1Gi	RWO	Retain	Bound	default/volume-pvc		<unset>

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATT	REBUTESCLASS	AG
persistentvolumeclaim/volume-pvc	Bound	volume-pv	1Gi	RWO		<unset>	9s	Filesystem

그림 0-47: PV와 PVC 바인딩 상태 확인

[실습] 컨테이너 내부에 접속하여 **/mnt/data** 경로에 파일을 생성합니다.

```
kubectl exec -it volume-pod -- /bin/bash # Pod 내부 접속
touch /mnt/data/c.txt # 볼륨 경로에 파일 생성
ls /mnt/data # 파일 목록 확인
```

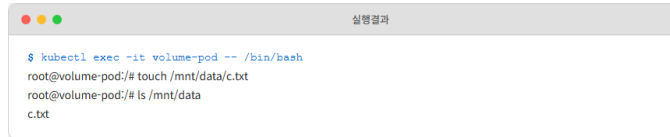


그림 0-48: 볼륨 경로에 파일 생성

`/mnt/data` 경로에 파일이 정상적으로 생성되었습니다.

그리고 연결된 로컬 PC에도 파일이 생성된 걸 확인할 수 있습니다.

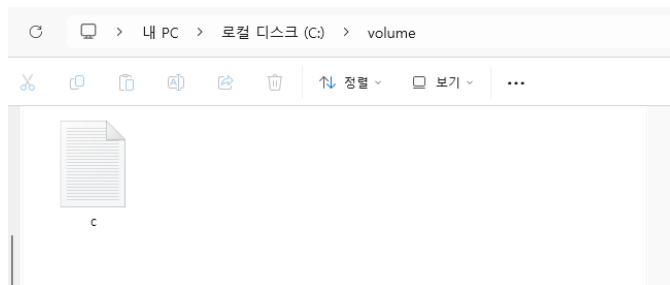


그림 0-49: 로컬 PC에서 마운트된 파일 확인

정리하면 다음과 같습니다.

- **인프라 운영자**는 PV(실제 스토리지 자원)를 생성하고 관리합니다.
- **개발자**는 PVC(스토리지 사용 요청)를 정의하여 필요한 저장 공간을 요청합니다.
- **Pod**는 애플리케이션이 실제로 실행되는 실행 단위입니다.
- **PVC**는 Pod가 사용할 스토리지를 요청하는 요청서입니다.
- **PV**는 그 요청에 의해 연결되는 실제 스토리지 자원입니다.

여기까지 쿠버네티스의 핵심 리소스를 하나씩 배워왔습니다. 이제 마지막으로, 이 모든 개념을 하나로 엮어 하나의 웹사이트를 미니큐브 위에 배포해보겠습니다.

3.7 미니큐브를 활용한 웹 사이트 만들기

우리는 챕터 2에서 프론트 서버 + 백엔드 서버 + DB 서버를 Docker Compose를 통해 실행했습니다. 그리고 챕터 3에서는 쿠버네티스의 리소스들을 하나씩 배워왔습니다. 마지막으로 이 모든 개념을 종합하여 Redis 서버를 추가한 뒤, 미니큐브 환경에서 실행해보겠습니다.

실습 코드는 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex08> 에서 확인할 수 있습니다.

3.7.1 아키텍처

이번 실습에서 배포할 애플리케이션의 전체 구조는 다음과 같습니다. 프론트엔드(Nginx), 백엔드(Spring Boot), DB(MySQL), Redis 총 4개의 서비스로 구성됩니다.

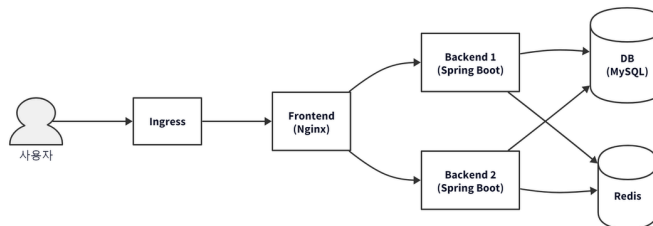


그림 0-50: ex08 풀스택 Kubernetes 아키텍처

챕터 2에서는 브라우저에서 `localhost` 로 바로 프론트엔드에 접속했습니다. 하지만 쿠버네티스에서는 외부 요청이 클러스터 내부로 바로 들어갈 수 없습니다. 그래서 `Ingress` 가 앞단에 위치합니다. 브라우저에서 요청을 보내면 `Ingress` 가 이를 받아 `Frontend Service` 로 전달하고, 이후 프론트엔드 → 백엔드 → DB/Redis 순으로 요청이 흘러갑니다.

3.7.2 이미지 폴더

[EX08 패키지 구조]

```

ex08/
├── backend/
│   ├── Dockerfile
│   └── entrypoint.sh
├── db/
│   ├── Dockerfile
│   └── init.sql
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── nginx.conf
├── redis/
│   └── Dockerfile
└── k8s/
  
```

EX08 폴더 내에는 이미지 생성을 위한 `backend`, `db`, `frontend`, `redis` 폴더가 있고, 이를 쿠버네티스 환경에서 실행하기 위한 `k8s` 폴더가 있습니다.

Backend, DB, Frontend, Redis 폴더는 EX07과 동일한 구조를 사용합니다. 설명이 필요한 부분만 코드로 표시하겠습니다.

우선 Redis를 추가하기 위해 Dockerfile을 작성해보겠습니다.

[참고] Github 프로젝트의 [ex08/redis/Dockerfile](#) 을 참고합니다.

ex08/redis/Dockerfile

```
FROM redis:7.4-alpine    # Redis 이미지 사용

CMD ["redis-server"]    # Redis 서버 실행
```

그리고 backend 폴더의 `entrypoint.sh`에서 `Git clone` 되는 주소를 수정합니다.

[참고] Github 프로젝트의 [ex08/backend/entrypoint.sh](#) 를 참고합니다.

ex08/backend/entrypoint.sh

```
#!/bin/bash

# 백엔드 서버 Github에서 내려받기 (Redis 버전으로 수정)
git clone https://github.com/metacoding-10-linux-docker/backend-redis-server
# 내려받은 폴더로 이동
cd backend-redis-server
# 실행 권한 부여
chmod +x gradlew
# 스프링 프로젝트 빌드
./gradlew build
# 빌드된 파일 실행
java -jar -Dspring.profiles.active=prod build/libs/*.jar
```

수정된 백엔드 서버는 API 요청 시 회원 정보와 함께 Redis에 저장된 방문 횟수를 함께 응답합니다.

[참고] `entrypoint.sh` 에서 내려받는 백엔드 서버의 핵심 API 코드는 다음과 같습니다.

UserController.java

```
@GetMapping("/api/users")
public ResponseEntity<?> findAll() {

    List<User> users = userRepository.findAll();    // DB에서 회원 목록 조회
```

```

Long count = redisTemplate.opsForValue()
    .increment("cnt:api/users:total");    // Redis 방문 횟수 증가

Map<String, Object> response = new HashMap<>();
response.put("users", users);            // 회원 목록
response.put("count", count);            // 방문 횟수

return Resp.ok(response);
}

```

프론트엔드의 index.html도 수정합니다. EX07과 비교해 방문 횟수를 표시하는 부분이 추가되었습니다.

[참고] Github 프로젝트의 [ex08/frontend/index.html](#) 을 참고합니다. (핵심 부분 발췌)

ex08/frontend/index.html

```

<h1>사용자 리스트</h1>
<h2>방문 횟수: <span id="visit-count">0</span></h2>

fetch('/api/users')          // nginx가 backend로 프록시
.then(response => response.json())
.then(data => {
    const users = data.body.users;    // 회원 목록
    const count = data.body.count;    // 방문 횟수

    users.forEach(user => {          // 응답 데이터를 테이블에 렌더링
        $("#user-list").append(render(user));
    });

    $("#visit-count").text(count);  // 방문 횟수 표시
});
// ... 생략
</script>

```

nginx.conf도 아래와 같이 수정합니다. upstream의 서버 주소를 Kubernetes Service명(backend-service)으로 변경합니다.

[참고] Github 프로젝트의 [ex08/frontend/nginx.conf](#) 를 참고합니다.

ex08/frontend/nginx.conf

```

events {}

http {
    # 백엔드 서버 주소 (K8s Service명으로 수정)
    upstream backend {
        server backend-service:8080;
    }

    server {
        listen 80;
        server_name _;

        # 정적 파일 제공
        location / {
            root /usr/share/nginx/html;
            index index.html;
        }

        # API 요청은 백엔드로 프록시
        location /api/ {
            proxy_pass http://backend;
        }
    }
}

```

3.7.3 k8s 폴더

ex08 폴더 내에 k8s 폴더가 있습니다.

k8s 폴더 는 Kubernetes 배포 설정 파일을 모아둔 폴더입니다.

[k8s 패키지 구조]

```

k8s/
├── backend/
│   ├── backend-configmap.yml
│   ├── backend-deploy.yml
│   ├── backend-secret.yml
│   └── backend-service.yml
├── db/
│   ├── db-deploy.yml
│   └── db-pv.yml

```

```

|   ├── db-pvc.yml
|   ├── db-secret.yml
|   └── db-service.yml
|   └── frontend/
|       ├── frontend-deploy.yml
|       ├── frontend-ingress.yml
|       └── frontend-service.yml
|   └── redis/
|       ├── redis-deploy.yml
|       └── redis-service.yml
|   └── namespace.yml

```

파일	설명
namespace.yml	리소스를 논리적으로 구분하는 Namespace 생성
*-deploy.yml	각 서버의 Deployment (Pod 생성 및 관리)
*-service.yml	각 서버의 Service (고정 IP로 Pod 접근)
frontend-ingress.yml	외부 요청을 프론트엔드 Service 로 라우팅하는 Ingress
backend-configmap.yml	백엔드 환경 변수 (DB 주소, Redis 주소)
backend-secret.yml	백엔드 민감 정보 (DB 계정/비밀번호)
db-secret.yml	DB 민감 정보 (MySQL 계정/비밀번호)
db-pv.yml / db-pvc.yml	DB 데이터 영구 저장을 위한 PV / PVC

Namespace

지금까지 실습에서 만든 Deployment, Service, ConfigMap 등은 모두 **default** 라는 기본 공간에 생성되었습니다. 리소스가 몇 개 안 될 때는 상관없지만, 여러 서비스의 리소스가 한 곳에 섞이면 관리가 어려워집니다.

Namespace 는 이 문제를 해결합니다. 회사 건물의 층 이라고 생각하면 됩니다. 1층은 프론트엔드 팀, 2층은 백엔드 팀, 3층은 데이터 팀처럼 층을 나누면 각 팀의 공간이 분리됩니다. 마찬가지로 **Namespace**를 나누면 리소스가 논리적으로 분리되어, 이름이 겹칠 걱정 없이 독립적으로 관리할 수 있습니다.

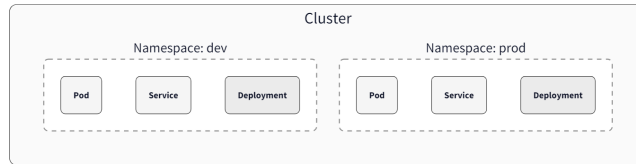


그림 0-52: 같은 Cluster 안에서 Namespace로 리소스를 분리한다

Namespace는 쿠버네티스 리소스를 논리적으로 구분하는 가상의 공간입니다. 별도로 지정하지 않으면 모든 리소스는 **default** 네임스페이스에 생성됩니다.

이번 실습에서는 `metacoding`이라는 Namespace를 만들어서, 모든 리소스를 이 안에 생성합니다. 각 YAML 파일의 `metadata`에 `namespace: metacoding`이 들어가 있는 이유가 바로 이것입니다.

[참고] Github 프로젝트의 [ex08/k8s/namespace.yml](#)을 참고합니다.

ex08/k8s/namespace.yml

```
apiVersion: v1      # API 버전
kind: Namespace     # 리소스 종류
metadata:
  name: metacoding  # 네임스페이스 이름
```

Namespace를 지정한 리소스는 조회할 때도 `-n` 옵션으로 Namespace를 명시해야 합니다.

```
kubectl get pod -n metacoding  # metacoding 네임스페이스의 Pod 조회
kubectl get pod                # default 네임스페이스의 Pod 조회 (metacoding 리소스 안 보임)
```

backend-deploy.yml

backend-deploy.yml은 이번 실습에서 가장 많은 리소스를 조합한 설정입니다. `replicas: 2`로 Pod를 2개 생성하고, `envFrom`으로 `ConfigMap`과 `Secret`을 동시에 연결합니다.

[참고] Github 프로젝트의 [ex08/k8s/backend/backend-deploy.yml](#)을 참고합니다.

ex08/k8s/backend/backend-deploy.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deploy
  namespace: metacoding  # namespace 설정
```



```
spec:
  replicas: 2                # pod 2개 생성
  selector:
    matchLabels:
      app: backend          # app: backend 라벨을 가진 pod 관리
  template:
    metadata:
      labels:
        app: backend        # pod에 app: backend 라벨 붙임
    spec:
      containers:
        - name: backend-server
          image: metacoding/backend:1
          ports:
            - containerPort: 8080    # 8080 포트 사용
          envFrom:
            - configMapRef:
                name: backend-configmap    # configmap 연결
            - secretRef:
                name: backend-secret      # secret 연결
```

`ConfigMap` 에는 DB 주소, Redis 주소 등 일반 설정을, `Secret` 에는 DB 계정/비밀번호를 저장합니다.

`Service` , `ConfigMap` , `Secret` 파일은 이전에 배운 구조와 동일하므로 Github을 참고합니다.

db-deploy.yml

db-deploy.yml은 `volumeMounts` 와 `PVC` 를 연결하여 DB 데이터를 영구 저장하는 설정입니다.

[참고] Github 프로젝트의 [ex08/k8s/db/db-deploy.yml](#) 을 참고합니다.

ex08/k8s/db/db-deploy.yml

```
apiVersion: apps/v1        # API 버전
kind: Deployment           # 리소스 종류
metadata:
  name: db-deploy          # 리소스 이름
  namespace: metacoding    # namespace 설정
spec:
  # 상세 설정
  replicas: 1              # Pod 1개 유지
  selector:
    matchLabels:
      app: db              # app: db 라벨을 가진 pod 관리
```

```

template:
  metadata:
    labels:
      app: db          # pod에 app: db 라벨 부여
  spec:
    containers:
      - name: db-server
        image: metacoding/db:1
        ports:
          - containerPort: 3306  # 3306 포트 사용
        envFrom:
          - secretRef:
              name: db-secret  # DB 접속을 위한 환경 변수 연결
        volumeMounts:
          - name: data
            mountPath: /var/lib/mysql # volume 경로 설정
    volumes:
      - name: data
        persistentVolumeClaim:
          claimName: db-pvc  # PVC 연결

```

`Secret`, `PV`, `PVC`, `Service` 파일은 이전에 배운 구조와 동일하므로 Github을 참고합니다.

나머지 설정

frontend, redis의 `Deployment` 와 `Service` 는 backend와 동일한 구조입니다. 이미지명, 라벨, 포트만 다릅니다. 아래 표를 참고합니다. 전체 코드는 Github을 참고합니다.

항목	frontend	redis
image	metacoding/frontend:1	metacoding/redis:1
containerPort	80	6379
replicas	1	1
Service port	80	6379

frontend-ingress.yml

이제 `Ingress` 리소스를 작성합니다. 모든 외부 요청을 `frontend-service` 의 80번 포트에 전달하는 규칙입니다.

[참고] Github 프로젝트의 [ex08/k8s/frontend/frontend-ingress.yml](#) 을 참고합니다.

ex08/k8s/frontend/frontend-ingress.yml

```

apiVersion: networking.k8s.io/v1    # API 버전
kind: Ingress                        # 리소스 종류
metadata:
  name: frontend-ingress            # Ingress 이름
  namespace: metacoding             # 네임스페이스 지정
spec:                               # 상세 설정
  rules:                            # 라우팅 규칙
    - http:                         # HTTP 규칙
      paths:                        # 경로 설정
        - path: /                  # 모든 경로
          pathType: Prefix          # 경로 매칭 방식
      backend:                      # 요청을 전달할 대상
        service:                   # 서비스 지정
          name: frontend-service   # 프론트엔드 Service로 연결
          port:                     # 포트 설정
            number: 80             # 서비스 포트 번호

```

path는 어떤 경로의 요청을 받을지 정하는 설정입니다. `/`로 설정하면 모든 경로의 요청을 받겠다는 의미이고, 해당 요청은 아래 **backend**에 지정한 **frontend-service**의 80번 포트로 전달됩니다.

전체 k8s 설정 파일은 <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex08/k8s>에서 확인할 수 있습니다.

3.7.4 실행하기

[실습] 미니큐브가 실행되어 있지 않다면 먼저 미니큐브를 실행합니다.

```
minikube start    # 미니큐브 클러스터 시작
```

Ingress Controller 활성화

Ingress 리소스가 동작하려면 **Ingress Controller**가 먼저 설치되어 있어야 합니다. 미니큐브에서는 애드온(addon)으로 간단하게 활성화할 수 있습니다.

[실습] Nginx Ingress Controller를 활성화합니다.

```
minikube addons enable ingress    # Ingress Controller 활성화
```

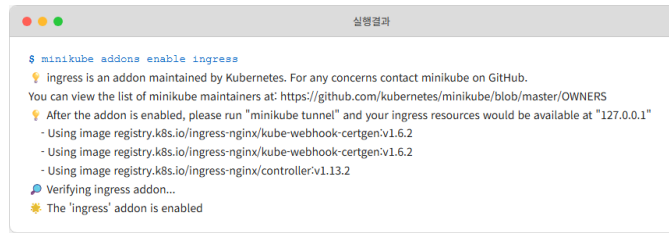


그림 0-54: Ingress Controller 활성화

[실습] Ingress Controller가 정상적으로 실행되고 있는지 확인합니다. `ingress-nginx-controller` Pod의 STATUS가 `Running` 이면 정상입니다.

`kubectl get pod -n ingress-nginx` # Ingress Controller Pod 상태 확인

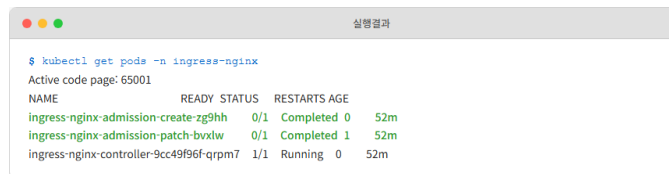


그림 0-55: Ingress Controller 실행 확인

이미지 빌드

EX08 폴더로 이동한 뒤, 각 서버의 Docker 이미지를 빌드합니다.

minikube image build: 미니큐브는 별도의 가상 환경(Docker 컨테이너 또는 VM) 안에서 동작합니다. 따라서 로컬 PC에서 `docker build` 로 만든 이미지는 미니큐브 내부에서 접근할 수 없습니다. `minikube image build` 명령을 사용하면 미니큐브 내부에 직접 이미지를 빌드하여, 별도의 이미지 레지스트리 없이도 Pod에서 바로 사용할 수 있습니다.

[실습] 각 서버의 Docker 이미지를 미니큐브 내부에 빌드합니다.

```

minikube image build -t metacoding/db:1 ./db          # DB 이미지 빌드
minikube image build -t metacoding/backend:1 ./backend # 백엔드 이미지 빌드
minikube image build -t metacoding/frontend:1 ./frontend # 프론트엔드 이미지 빌드
minikube image build -t metacoding/redis:1 ./redis    # Redis 이미지 빌드

```

```

실행결과

$ minikube image build -t metacoding/db:1 ./db
#7 DONE 0.1s

H133m2 warnings found (use docker --debug to expand):
H10m - SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_PASSWORD") (line 6)
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_ROOT_PASSWORD") (line 7)

$ minikube image build -t metacoding/backend:1 ./backend
#9 exporting layers
#9 exporting layers 1.2s done
#9 writing image sha256:1bdcc956d12db5a420f35886e8a52d66de0e0ce299374e8b1ab0915fc93d871 done
#9 naming to docker.io/metacoding/backend:1 0.0s done
#9 DONE 1.3s

$ minikube image build -t metacoding/frontend:1 ./frontend
#8 exporting to image
#8 exporting layers 0.1s done
#8 writing image sha256:c14a68b6cc45414e4d14d758090aa7f0032a7272db52591510a2c0a97ac7d818 done
#8 naming to docker.io/metacoding/frontend:1 0.0s done
#8 DONE 0.2s

$ minikube image build -t metacoding/redis:1 ./redis
#5 exporting to image
#5 exporting layers done
#5 writing image sha256:750007ed03b6df746d9afc354f887261e1da0476ce163077001599539a825e49 done
#5 naming to docker.io/metacoding/redis:1 0.0s done
#5 DONE 0.1s

```

그림 0-56: 미니큐브 이미지 빌드

리소스 생성

[실습] `Namespace` 를 생성합니다.

```
kubectl apply -f k8s/namespace.yml # Namespace 생성
```

```

실행결과

$ kubectl apply -f k8s/namespace.yml
namespace/metacoding created

```

그림 0-57: Namespace 생성

[실습] k8s 폴더의 모든 리소스를 생성합니다. `--recursive` 옵션은 하위 폴더의 YML 파일도 모두 적용합니다.

```
kubectl apply -f k8s/ --recursive # k8s 폴더의 모든 리소스 일괄 생성
```

```

$ kubectl apply -f k8s/ --recursive
configmap/backend-configmap created
deployment.apps/backend-deploy created
secret/backend-secret created
service/backend-service created
deployment.apps/db-deploy created
persistentvolume/db-pv created
persistentvolumeclaim/db-pvc created
secret/db-secret created
service/db-service created
deployment.apps/frontend-deploy created
ingress.networking.k8s.io/frontend-ingress created
service/frontend-service created
namespace/metacoding unchanged
deployment.apps/redis-deploy created
service/redis-service created

```

그림 0-58: k8s 리소스 일괄 생성

[실습] 생성된 리소스 정보를 확인합니다.

kubectl get deploy,pod,service -n metacoding # Deployment, Pod, Service 조회

```

$ kubectl get deploy,pod,service -n metacoding

```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/backend-deploy	2/2	2	2	5m9s
deployment.apps/db-deploy	1/1	1	1	5m9s
deployment.apps/frontend-deploy	1/1	1	1	5m9s
deployment.apps/redis-deploy	1/1	1	1	5m9s

NAME	READY	STATUS	RESTARTS	AGE
pod/backend-deploy-f7878cc5f-dnk5c	1/1	Running	0	5m9s
pod/backend-deploy-f7878cc5f-xsvjw	1/1	Running	0	5m9s
pod/db-deploy-66544b9896-rrccn	1/1	Running	0	5m9s
pod/frontend-deploy-6f567ffc4d-r9n85	1/1	Running	0	5m9s
pod/redis-deploy-579c655488-5ctz5	1/1	Running	0	5m9s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/backend-service	ClusterIP	10.108.17.238	<none>	8080/TCP	5m9s
service/db-service	ClusterIP	10.111.236.153	<none>	3306/TCP	5m9s
service/frontend-service	ClusterIP	10.106.199.126	<none>	80/TCP	5m9s
service/redis-service	ClusterIP	10.103.21.98	<none>	6379/TCP	5m9s

그림 0-59: Deployment, Pod, Service 조회

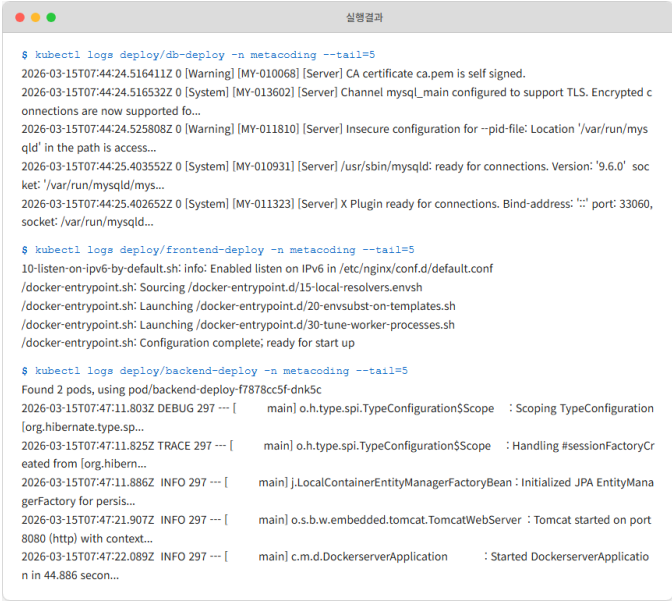
컨테이너 실행 확인

[실습] 각 서버의 로그를 확인합니다.

kubectl logs deploy/db-deploy -n metacoding --tail=5 # DB 서버 로그 확인

kubectl logs deploy/frontend-deploy -n metacoding --tail=5 # 프론트엔드 서버 로그 확인

kubectl logs deploy/backend-deploy -n metacoding --tail=5 # 백엔드 서버 로그 확인



```

$ kubectl logs deploy/db-deploy -n metacoding --tail=5
2026-03-15T07:44:24.516411Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-03-15T07:44:24.516532Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted c
onnections are now supported fo...
2026-03-15T07:44:24.525808Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mys
qlid' in the path is access...
2026-03-15T07:44:25.403552Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '9.6.0' soc
ket: '/var/run/mysqld/mys...
2026-03-15T07:44:25.402652Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060,
socket: '/var/run/mysqld...

$ kubectl logs deploy/frontend-deploy -n metacoding --tail=5
10-listen-on-ip-v6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up

$ kubectl logs deploy/backend-deploy -n metacoding --tail=5
Found 2 pods, using pod/backend-deploy-f7878cc5f-dnk5c
2026-03-15T07:47:11.803Z DEBUG 297 --- [main] o.h.type.spi.TypeConfiguration$Scope : Scoping TypeConfiguration
[org.hibernate.type.sp...
2026-03-15T07:47:11.825Z TRACE 297 --- [main] o.h.type.spi.TypeConfiguration$Scope : Handling #SessionFactoryCr
eated from [org.hibern...
2026-03-15T07:47:11.886Z INFO 297 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityMana
gerFactory for persis...
2026-03-15T07:47:21.907Z INFO 297 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port
8080 (http) with context...
2026-03-15T07:47:22.089Z INFO 297 --- [main] c.m.d.DockerserverApplication : Started DockerserverApplicatio
n in 44.886 secon...

```

그림 0-60: 서버 로그 확인

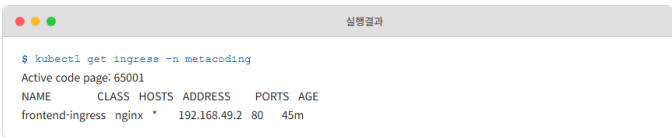
Pod 가 Running 상태라고 해서 서버가 바로 정상 동작하는 건 아닙니다. 실행에 시간이 필요할 수 있
으니 **kubectl logs** 명령어를 활용해 각 서버의 로그를 확인하며 정상적으로 동작하는지 확인합니다.

Ingress로 서버 연결

리소스가 모두 생성되었으니, **Ingress** 를 통해 프론트엔드에 접속해보겠습니다.

[실습] 생성된 **Ingress** 정보를 확인합니다. ADDRESS에 IP가 표시되면 정상입니다. (표시되기까지
1~2분 소요될 수 있습니다.)

```
kubectl get ingress -n metacoding # Ingress 리소스 조회
```



```

$ kubectl get ingress -n metacoding
Active code page: 65001
NAME          CLASS  HOSTS          ADDRESS          PORTS  AGE
frontend-ingress  nginx  *             192.168.49.2    80     45m

```

그림 0-61: Ingress 리소스 확인

[실습] **minikube tunnel** 명령어를 실행합니다. Docker Desktop 드라이버를 사용하는 경우 로컬 PC
에서 미니큐브 내부 IP로 직접 접근할 수 없기 때문에 터널이 필요합니다.

```
minikube tunnel # 로컬 PC에서 클러스터 접근을 위한 터널 생성
```

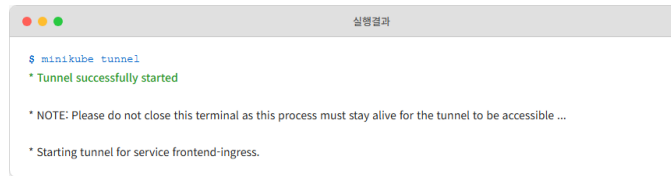


그림 0-62: minikube tunnel 실행

`minikube tunnel` 은 포그라운드로 실행됩니다. 터미널 창을 종료하면 터널도 함께 종료되니, 접속 확인이 끝날 때까지 유지합니다.

[실습] 브라우저에서 <http://127.0.0.1:80> 로 접속합니다.

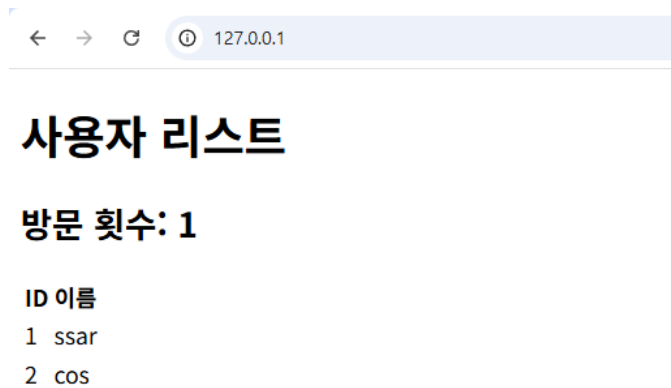
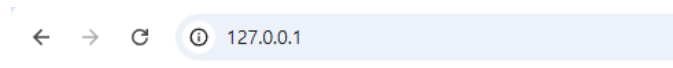


그림 0-63: Ingress를 통한 웹사이트 접속

DB에서 조회된 데이터가 화면에 표시됩니다. 여러 번 요청을 보내면 방문 횟수가 증가하는 걸 볼 수 있습니다.



사용자 리스트

방문 횟수: 20

ID 이름

1 ssar

2 cos

그림 0-64: 방문 횟수 증가 확인

이전에 `minikube service` 명령어로 임시 URL을 만들어 접속했던 것과 비교해보면 `Ingress` 는 도메인 기반 라우팅을 지원하여 실제 운영 환경과 더 유사한 구조를 제공합니다.

이제 `Pod` 내부를 확인해보겠습니다. `minikube tunnel` 이 실행 중인 터미널은 유지하고, 새로운 터미널 창을 실행합니다.

[실습] 전체 `Pod` 목록을 확인합니다.

`kubectl get pod -n metacoding` # `metacoding` 네임스페이스의 `Pod` 목록 조회



그림 0-65: 전체 Pod 목록 확인

[실습] 각 백엔드 서버의 로그를 확인합니다.

`kubectl logs backend-deploy-f7878cc5f-rbs74 -n metacoding --tail=10` # 백엔드 서버 1 로그 확인

`kubectl logs backend-deploy-f7878cc5f-wt2cd -n metacoding --tail=10` # 백엔드 서버 2 로그 확인

`backend-deploy-f7878cc5f-rbs74`



```

$ kubectl logs backend-deploy-f7878cc5f-rbs74 -n metacoding --tail=10
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0

```

그림 0-66: 백엔드 서버 1 로그

backend-deploy-f7878cc5f-wt2cd



```

$ kubectl logs backend-deploy-f7878cc5f-wt2cd -n metacoding --tail=10
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0
Hibernate: select u1_0.id,u1_0.name from user_tb u1_0

```

그림 0-67: 백엔드 서버 2 로그

로그에서 회원 정보를 조회하기 위해 실행된 **SELECT문** 이 출력됩니다. **backend-service** 가 로드밸런싱을 수행해서 요청이 두 서버로 분산 처리된 것을 확인할 수 있습니다.

이것만은 기억하자

- 컨테이너가 많아지면, 관제탑이 필요하다. 컨테이너 몇 개는 수동으로 관리할 수 있지만, 수십~수백 개가 되면 자동으로 배포하고, 복구하고, 확장하는 쿠버네티스가 필수입니다.
- **Deployment + Service + Ingress** = 안정적인 서비스. Deployment가 Pod의 개수와 상태를 자동으로 유지하고, Service가 고정된 접근점을 제공하며, Ingress가 외부 요청을 적절한 Service로 라우팅합니다.
- 설정은 밖에서, 데이터는 영구히. ConfigMap과 Secret으로 코드와 설정을 분리하고, PV/PVC로 컨테이너가 사라져도 데이터를 보존합니다.

이것으로 Docker와 Kubernetes를 향한 오픈이의 여정이 마무리되었습니다. 챕터 1에서 컨테이너 하나를 띄우는 것부터 시작해 챕터 2에서 여러 컨테이너를 한 번에 관리하는 Docker Compose를 배웠고 챕터 3에서는 쿠버네티스로 자동화된 운영 환경까지 구성했습니다. Ingress를 통해 외부 요청을 도메인 기반으로 연결하는 것까지 해봤으니, 실무 환경과의 거리도 한층 좁혀졌습니다. 물론 실무에는 Helm, CI/CD 파이프라인 같은 주제가 더 기다리고 있습니다. 하지만 여기까지 온 여러분이라면, 그 어떤 기술도 두렵지 않을 것입니다.

맷음말

오픈이의 여정이 끝났습니다.

새벽 3시에 올린 전화 한 통에서 시작해서, 상자 하나를 만드는 법을 배우고, 여러 상자를 화물선에 싣는 법을 익히고, 마침내 항구 전체를 관제 시스템에 맡기는 데까지 왔습니다.

여기까지 따라오신 여러분의 머릿속에는 이제 하나의 그림이 그려져 있을 겁니다.

- 환경이 달라서 안 돌아간다 → **Docker**로 상자에 담는다.
- 상자가 여러 개라 관리가 안 된다 → **Docker Compose**로 한꺼번에 싣는다.
- 항구가 커져서 사람 손으로 안 된다 → **Kubernetes**가 자동으로 관리한다.

이 그림이 그려진다면, 이 책은 제 역할을 다한 겁니다.

하지만 솔직히 말하면, 이 책에서 다룬 건 시작에 가깝습니다.

실무에서는 이 책에서 다루지 않은 것들이 기다리고 있습니다. CI/CD 파이프라인으로 배포를 자동화하고, 모니터링 도구로 서비스 상태를 감시하고, 클라우드 환경에서 실제 운영을 경험하게 될 겁니다.

그때 이 책에서 잡은 전체 그림이 빛을 발합니다. 새로운 도구를 만났을 때 “이게 뭐지?” 대신 “아, 이건 이 부분을 해결하는 거구나” 하고 자리를 잡아줄 겁니다. 개념이 잡혀 있으면 새로운 기술을 배우는 속도가 완전히 달라집니다.

이 책이 여러분의 실무에서 든든한 기초 체력이 되었으면 좋겠습니다. 컨테이너 기술의 전체 그림을 머릿속에 품고, 거기서부터 한 걸음씩 더 깊이 나아가 보세요.

여러분의 다음 여정을 응원합니다.

부록: 더 깊이 공부하기

이 책에서 전체 그림을 잡았다면, 이제 각 기술을 더 깊이 파고들 차례입니다. 아래 자료들은 다음 단계로 나아가는 데 도움이 됩니다.

Docker

자료	설명
Docker 공식 문서	docs.docker.com – 명령어 레퍼런스, Dockerfile 작성법, 네트워크 설정 등 가장 정확한 자료
Docker Hub	hub.docker.com – 공식 이미지 검색 및 사용법 확인
Docker Compose 공식 문서	docs.docker.com/compose – Compose 파일 문법, 멀티 컨테이너 구성 심화

이 책에서 다루지 않은 **Docker** 심화 주제: - Docker 네트워크 종류 (bridge, host, overlay) - 멀티 스테이지 빌드 (이미지 크기 최적화) - Docker 보안 (rootless 모드, 이미지 스캐닝) - .dockerignore 활용

Kubernetes

자료	설명
Kubernetes 공식 문서	kubernetes.io/docs – 개념 설명, 튜토리얼, API 레퍼런스
Kubernetes 공식 튜토리얼	kubernetes.io/docs/tutorials – 단계별 실습 가이드
kubectl 치트시트	kubernetes.io/docs/reference/kubectl/cheatsheet – 자주 쓰는 명령어 모음

이 책에서 다루지 않은 **K8s** 심화 주제: - Ingress (외부 트래픽 라우팅) - HPA (수평 자동 확장) - Helm (패키지 매니저) - RBAC (역할 기반 접근 제어) - 클라우드 배포 (AWS EKS, Google GKE, Azure AKS)

이 책의 예제 코드

이 책에서 사용한 모든 예제 코드는 아래 저장소에서 확인할 수 있습니다.

- **GitHub:** 저장소 URL (추후 추가)

각 장별 예제 폴더 구조:

ex01~ex03/ → 2장 NGINX 실습
ex04/ → 2장 Redis 실습
ex05/ → 2장 DB Server 실습
ex06/ → 2장 Docker Compose 실습
ex07/ → 2장 폴스택 웹사이트
ex08/ → 3장 K8s 폴스택 웹사이트
yaml/ → 3장 K8s 리소스 실습

주요 명령어 모음

Docker 명령어

명령어	설명
<code>docker pull [이미지]</code>	이미지 다운로드
<code>docker run [옵션] [이미지]</code>	컨테이너 실행
<code>docker ps</code>	실행 중인 컨테이너 목록
<code>docker ps -a</code>	모든 컨테이너 목록
<code>docker stop [컨테이너]</code>	컨테이너 중지
<code>docker rm [컨테이너]</code>	컨테이너 삭제
<code>docker images</code>	이미지 목록
<code>docker rmi [이미지]</code>	이미지 삭제
<code>docker build -t [태그] .</code>	Dockerfile로 이미지 빌드
<code>docker commit [컨테이너] [이미지명]</code>	컨테이너를 이미지로 저장
<code>docker push [이미지]</code>	이미지를 레지스트리에 업로드
<code>docker exec -it [컨테이너] bash</code>	실행 중인 컨테이너에 접속
<code>docker compose up -d</code>	Compose로 서비스 실행
<code>docker compose down</code>	Compose 서비스 중지 및 삭제

kubectl 명령어

명령어	설명
<code>kubectl get pods</code>	Pod 목록 조회
<code>kubectl get deployments</code>	Deployment 목록 조회
<code>kubectl get services</code>	Service 목록 조회
<code>kubectl apply -f [파일]</code>	YAML 파일로 리소스 생성/수정
<code>kubectl delete -f [파일]</code>	YAML 파일의 리소스 삭제
<code>kubectl describe [리소스] [이름]</code>	리소스 상세 정보
<code>kubectl logs [Pod명]</code>	Pod 로그 확인
<code>kubectl exec -it [Pod명] -- bash</code>	Pod 내부 접속
<code>kubectl set image deployment/[이름] [컨테이너]=[이미지]</code>	이미지 업데이트
<code>kubectl rollout undo deployment/[이름]</code>	이전 버전으로 롤백
<code>kubectl rollout restart deployment/[이름]</code>	Deployment 재시작
<code>minikube start</code>	미니큐브 클러스터 시작
<code>minikube service [서비스명] --url</code>	서비스 접근 URL 확인